

```
>>> nombre = "Estudiante"
>>> for paso in range(1, 31):
...     aprender(paso)
>>> print(f"¡Hola, {nombre}!")
¡Hola, Estudiante!

>>> _
```

Python desde Cero

Guía de estudio: de los operadores a las APIs, microservicios
y simulación 3D

Núcleo Lógica de Programación · CEIPA
Septiembre de 2026

Contenido

Cómo usar esta guía	4
ETAPA 1 · FUNDAMENTOS	
Módulo 1: Qué es Python y cómo prepararse	6
Módulo 2: Primer programa y reglas de sintaxis	9
Módulo 3: Variables y tipos de datos	11
Módulo 4: Operadores	13
Módulo 5: Textos (strings), entrada y salida	17
ETAPA 2 · LÓGICA Y CONTROL	
Módulo 6: Condicionales	20
Módulo 7: Ciclos	23
Módulo 8: Listas, tuplas y conjuntos	26
Módulo 9: Diccionarios	29
Módulo 10: Funciones, recursividad y lambdas	33
Módulo 11: Algoritmos sencillos resueltos	37
ETAPA 3 · ORIENTACIÓN A OBJETOS	
Módulo 12: Programación Orientada a Objetos I	41
Módulo 13: Programación Orientada a Objetos II	44
Módulo 14: Manejo de excepciones y archivos	48
Módulo 15: Módulos, paquetes, entornos virtuales y librería estándar	51
ETAPA 4 · PROYECTOS PRÁCTICOS	
Módulo 16: Proyecto — Calculadora de finanzas personales	54
Módulo 17: Proyecto — Reproductor de música	59
Módulo 18: Proyecto — Juegos sencillos	63
ETAPA 5 · DISEÑO PROFESIONAL	
Módulo 19: Principios KISS, DRY, YAGNI y SOLID	68
Módulo 20: Patrones de diseño	71
Módulo 21: Patrones arquitectónicos	77
ETAPA 6 · APLICACIONES REALES	
Módulo 22: Consumo de APIs REST	81
Módulo 23: Backend — API REST con FastAPI y SQL	85
Módulo 24: Frontend con Python	91

Módulo 25: Microservicios	94
Módulo 26: Simulación 3D	98
Módulo 27: Pruebas unitarias con pytest	102
ETAPA 7 · EVALUACIÓN	
Módulo 28: Banco de preguntas y respuestas	104
Módulo 29: Python Developer Test	109
Módulo 30: Soluciones de los retos y proyecto integrador	115
Recursos recomendados	123
Anexo: Hoja de referencia rápida	125

Cómo usar esta guía

Esta guía lleva a una persona que nunca ha programado hasta construir una API REST, un frontend, microservicios, juegos y simulaciones 3D con Python. Está organizada en **30 módulos** que suben de dificultad poco a poco. Cada módulo tiene teoría corta, código que se puede copiar y ejecutar, un **caso real**, **retos** y **preguntas de repaso**.

Etapa	Módulos	Qué aprende el estudiante	Tiempo sugerido
1. Fundamentos	1 a 5	Instalar Python, sintaxis, variables, operadores, textos, entrada y salida	2 semanas
2. Lógica y control	6 a 11	Decisiones, ciclos, listas, diccionarios , funciones y algoritmos	3 semanas
3. Orientación a objetos	12 a 15	Clases, herencia, excepciones, archivos, módulos y entornos virtuales	3 semanas
4. Proyectos prácticos	16 a 18	Finanzas personales, reproductor de música, juegos (Snake y rompecabezas)	2 semanas
5. Diseño profesional	19 a 21	Principios SOLID, patrones de diseño y patrones arquitectónicos	2 semanas
6. Aplicaciones reales	22 a 27	APIs REST, backend, frontend, microservicios, simulación 3D y pruebas	3 semanas
7. Evaluación	28 a 30	Banco de preguntas, <i>Python Developer Test</i> y soluciones de retos	1 semana

Método recomendado para cada módulo:

1. Leer la teoría y **escribir el código a mano**, sin copiar y pegar. Así se aprende la sintaxis.
2. Ejecutarlo, cambiar un valor y **predecir el resultado** antes de volver a ejecutar.
3. Responder las preguntas de repaso sin mirar la respuesta.
4. Resolver el reto del módulo y compararlo con la solución del Módulo 30.

Convenciones:

- El código usa **Python 3.10 o superior** (se recomienda la versión estable más reciente, 3.14 a septiembre de 2026).
- Los comentarios `# Salida:` muestran lo que imprime la consola.
- Las líneas que empiezan con `>>>` se escriben en la consola interactiva de Python (REPL).
- Todos los proyectos completos están en la carpeta `proyectos/` del repositorio y las soluciones en `retos/soluciones/`.

Para el docente: la evaluación del Módulo 29 (*Python Developer Test*) se entrega sin respuestas. La clave de respuestas va en un archivo aparte que no se publica en el repositorio.

Módulo 1: Qué es Python y cómo prepararse

Python es un lenguaje de programación **de alto nivel, interpretado, de tipado dinámico y multiparadigma** (permite programar de forma estructurada, orientada a objetos y funcional). Fue creado por Guido van Rossum y publicado en 1991. Su filosofía privilegia la **legibilidad**: el código de Python se parece mucho al pseudocódigo que se escribe en la clase de lógica.

Se usa en ciencia de datos, inteligencia artificial, automatización, desarrollo web (backend), robótica, simulación, videojuegos, ciberseguridad y en scripts para casi cualquier tarea repetitiva.

Conceptos clave

Concepto	Qué significa	Analogía
Lenguaje interpretado	El intérprete lee y ejecuta el programa línea por línea; no hay un paso de compilación manual	Un traductor simultáneo en una conferencia
Tipado dinámico	El tipo de una variable se decide al ejecutar y puede cambiar	Una caja que acepta cualquier objeto
Tipado fuerte	Python no mezcla tipos sin permiso: <code>"3" + 4</code> produce error	No se suman peras con manzanas
CPython	La implementación oficial de Python, escrita en C	El motor original del carro
pip	El gestor de paquetes: instala librerías de terceros desde PyPI	La tienda de aplicaciones de Python
Entorno virtual (venv)	Carpeta aislada con su propio Python y sus librerías	Un cuarto propio para cada proyecto

¿Cómo se ejecuta un programa? Escribes `hola.py` → el intérprete lo convierte internamente en *bytecode* (archivos `.pyc` en la carpeta `__pycache__`) → la máquina virtual de Python ejecuta ese *bytecode*. Es un principio parecido a la JVM de Java, pero automático e invisible para el programador.

Python frente a Java (para quien viene de la guía de Java):

Aspecto	Java	Python
Tipos	Se declaran: <code>int edad = 20;</code>	Se infieren: <code>edad = 20</code>
Bloques	Llaves <code>{ }</code>	Indentación (4 espacios)
Fin de instrucción	Punto y coma <code>;</code>	Salto de línea
Punto de entrada	<code>public static void main</code>	El archivo se ejecuta de arriba hacia abajo
Incremento	<code>x++</code>	<code>x += 1</code> (no existe <code>++</code>)

Qué versión instalar

Python publica una versión nueva cada año (en octubre) y cada versión recibe soporte durante cinco años. A septiembre de 2026, la página oficial de descargas indica que **Python 3.14** es la serie estable más reciente (3.14.7, agosto de 2026) y que **Python 3.15** está planeado para el 1 de octubre de 2026. Para aprender, instala la **última versión estable 3.14.x**. Todo el código de esta guía funciona desde Python 3.10.

Instalación paso a paso

1. Entra a **python.org/downloads** y descarga la versión estable para tu sistema operativo.
2. **Windows:** ejecuta el instalador (o el *Python install manager* que ofrece la página). Si aparece la casilla "**Add python.exe to PATH**", márcala.
3. **macOS:** usa el instalador `.pkg` de python.org. **Linux:** casi siempre ya viene instalado; verifica con `python3 --version`.
4. Abre una terminal y verifica:

```
python --version      # Windows (también funciona: py --version)
python3 --version     # macOS y Linux
pip --version         # el gestor de paquetes
```

5. Instala un editor. Recomendado: **Visual Studio Code** con la extensión oficial **Python** de Microsoft. Alternativas: **PyCharm Community** (gratuito) o **Thonny** (muy sencillo, ideal para el primer día).
6. Sin instalar nada: **Google Colab** o **Replit** permiten programar en el navegador.

Herramienta para experimentar: la consola interactiva (REPL)

Escribe `python` (o `python3`) en la terminal. Aparece el indicador `>>>` y puedes probar instrucciones sin crear archivos:

```
>>> edad = 20
>>> edad * 2
40
>>> type(edad)
<class 'int'>
>>> exit()
```

Preguntas de repaso

- **¿Qué significa que Python sea interpretado?** Que el intérprete ejecuta el código directamente, sin un paso de compilación manual previo como `javac`.

- **¿Qué diferencia hay entre tipado dinámico y tipado débil?** Dinámico: el tipo se decide al ejecutar. Débil: el lenguaje mezcla tipos solo. Python es dinámico pero **fuerte**.
- **¿Para qué sirve pip?** Para instalar librerías de terceros, por ejemplo `pip install requests`.

Módulo 2: Primer programa y reglas de sintaxis

Crea un archivo llamado `hola.py` con el siguiente contenido:

```
# Mi primer programa en Python
print("¡Hola, mundo!")
print("Sin salto", end=" ")      # end cambia el salto de línea por un espacio
print("de línea")
print("Python", "es", "genial", sep="-") # sep cambia el separador
# Salida:
# ¡Hola, mundo!
# Sin salto de línea
# Python-es-genial
```

Para ejecutarlo, abre la terminal en la carpeta del archivo:

```
python hola.py      # Windows
python3 hola.py     # macOS y Linux
```

Reglas de sintaxis que no se negocian

- **La indentación es obligatoria.** Los bloques (dentro de un `if`, `for`, `def`, `class`) se marcan con **4 espacios**, no con llaves. Una indentación incorrecta produce `IndentationError`.
- Los bloques se abren con **dos puntos** `:` al final de la línea.
- **No se usa punto y coma:** cada instrucción va en su propia línea.
- Python **distingue mayúsculas de minúsculas:** `edad`, `Edad` y `EDAD` son tres variables distintas.
- **Comentarios:** `#` una línea. Para documentar funciones y clases se usan *docstrings* entre triples comillas `"""..."""`.
- Los textos van entre comillas **simples o dobles:** `'hola'` y `"hola"` son iguales.

```
edad = 20
if edad >= 18:      # los dos puntos abren el bloque
    print("Mayor de edad") # 4 espacios de indentación
    print("Puede votar")  # sigue dentro del if
print("Fin")         # fuera del if (sin indentación)
```

Convenciones de nombres (PEP 8)

PEP 8 es la guía oficial de estilo de Python. Seguirla hace que tu código se vea profesional.

Elemento	Estilo	Ejemplo
Variables y funciones	snake_case (minúsculas con guion bajo)	saldo_actual , calcular_total()
Clases	PascalCase	CuentaBancaria , Producto
Constantes	MAYUSCULAS_CON_GUION	IVA , TASA_MAXIMA
Módulos (archivos)	minúsculas cortas	finanzas.py , utilidades.py
Atributos "privados"	un guion bajo inicial	_saldo

Palabras reservadas que no se pueden usar como nombres: False , None , True , and , as , assert , async , await , break , class , continue , def , del , elif , else , except , finally , for , from , global , if , import , in , is , lambda , nonlocal , not , or , pass , raise , return , try , while , with , yield . Puedes verlas con `import keyword` ;
`print(keyword.kwlist)` .

Curiosidad: escribe `import this` en la consola. Aparece *El Zen de Python*, 19 principios del lenguaje. Uno de ellos resume toda la filosofía: la legibilidad cuenta.

Errores típicos del principiante

Mensaje de error	Causa probable
IndentationError: expected an indented block	Falta indentar la línea después de :
SyntaxError: expected ':'	Faltan los dos puntos al final de if , for , def ...
NameError: name 'x' is not defined	Variable mal escrita o usada antes de crearla
TypeError: can only concatenate str (not "int") to str	Se intentó unir texto con número: usa str() o f-strings
SyntaxError: unterminated string literal	Falta cerrar una comilla

Cómo leer un error (traceback): Python muestra el error de abajo hacia arriba. La **última línea** dice el tipo de error y la línea anterior indica el **archivo y número de línea** donde ocurrió. Empieza siempre leyendo la última línea.

Reto 2.1

Escribe un programa que imprima tu nombre, tu ciudad y tu carrera, cada dato en una línea, y luego los tres en una sola línea separados por | usando el parámetro sep .

Módulo 3: Variables y tipos de datos

Una **variable** es un nombre que apunta a un valor guardado en memoria. En Python no se declara el tipo: se infiere del valor asignado, y la misma variable puede apuntar luego a otro tipo.

```
edad = 20 # int
precio = 45_900.50 # float (el _ mejora la lectura: 45900.50)
nombre = "Ana" # str
es_estudiante = True # bool
sin_valor = None # NoneType: "ausencia de valor"

print(type(precio)) # Salida: <class 'float'>
edad = "veinte" # válido: ahora edad apunta a un str (tipado dinámico)

# Asignación múltiple e intercambio sin variable auxiliar
a, b = 1, 2
a, b = b, a
print(a, b) # Salida: 2 1
```

Tipos básicos

Tipo	Qué guarda	Ejemplos	Notas
int	Enteros de tamaño ilimitado	42, -7, 10**100	No hay desbordamiento como en Java
float	Decimales (doble precisión, 64 bits)	3.14, 1e-3, 2.0	Pueden tener errores de redondeo
complex	Números complejos	3 + 4j	Útiles en ingeniería eléctrica
str	Texto (inmutable)	"hola", 'A'	No existe tipo char: un carácter es un str de largo 1
bool	Verdadero o falso	True, False	Con mayúscula inicial
NoneType	Ausencia de valor	None	Equivale a null

Conversión de tipos (casting)

```
print(int("25") + 5) # Salida: 30
print(float("3.5") * 2) # Salida: 7.0
print(int(3.99)) # Salida: 3 (trunca, no redondea)
print(round(3.567, 2)) # Salida: 3.57
print(str(100) + " pesos") # Salida: 100 pesos
print(bool(0), bool(""), bool([]), bool(None)) # Salida: False False False False
print(bool("False")) # Salida: True (cualquier texto no vacío es True)
```

Valores "falsos" (falsy): 0, 0.0, "", [], {}, (), set() y None. Todo lo demás es verdadero. Esto permite escribir `if lista:` en lugar de `if len(lista) > 0:`.

Constantes

Python no tiene una palabra reservada para constantes. Por convención se escriben en MAYÚSCULAS y nadie las modifica:

```
IVA = 0.19
DESCUENTO_ESTUDIANTE = 0.10
```

Caso real: el problema de los decimales en dinero

```
print(0.1 + 0.2)           # Salida: 0.30000000000000004
print(0.1 + 0.2 == 0.3)    # Salida: False

from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2")) # Salida: 0.3 (exacto)
```

Los `float` se guardan en binario y algunos decimales no tienen representación exacta. En una tienda en línea se puede usar `float` para practicar, pero los sistemas bancarios y contables usan `Decimal` (o guardan los valores en centavos como enteros).

Reto 3.1

Pide al usuario una temperatura en grados Celsius y muéstrala convertida a Fahrenheit ($F = C \times 9/5 + 32$) y a Kelvin ($K = C + 273.15$).

Preguntas de repaso

- **¿Qué imprime `type(10 / 2)`?** `<class 'float'>`: la división con `/` siempre devuelve `float`, aunque sea exacta.
- **¿Qué es `None`?** Un valor especial que representa "nada"; por ejemplo, lo que devuelve una función sin `return`.
- **¿`int("3.5")` funciona?** No: produce `ValueError`. Primero `float("3.5")` y luego `int(...)`.

Módulo 4: Operadores

Los operadores permiten calcular, comparar y combinar valores. Este módulo es la base de todas las preguntas de "¿qué imprime este código?" de las evaluaciones.

Operadores aritméticos

Operador	Operación	Ejemplo	Resultado
+	Suma	7 + 2	9
-	Resta	7 - 2	5
*	Multiplicación	7 * 2	14
/	División (siempre float)	7 / 2	3.5
//	División entera (piso)	7 // 2	3
%	Módulo (residuo)	7 % 2	1
**	Potencia	2 ** 10	1024

```
print(10 / 5)      # Salida: 2.0   (float aunque la división sea exacta)
print(-7 // 2)     # Salida: -4    (redondea hacia abajo, no hacia cero)
print(-7 % 3)      # Salida: 2     (en Python el residuo toma el signo del divisor)
print(2 ** 0.5)    # Salida: 1.4142135623730951 (raíz cuadrada)
print("ja" * 3)    # Salida: jajaja (* también repite textos y listas)
print(divmod(17, 5)) # Salida: (3, 2) -> cociente y residuo al tiempo
```

Usos típicos de %: saber si un número es par (`n % 2 == 0`), obtener el último dígito (`n % 10`), hacer ciclos que "dan la vuelta" (`indice % largo`).

Operadores de asignación

```
x = 10
x += 5    # x = x + 5    -> 15
x -= 3    # x = x - 3    -> 12
x *= 2    # x = x * 2    -> 24
x /= 4    # x = x / 4     -> 6.0
x //= 4   # x = x // 4    -> 1.0
x **= 3   # x = x ** 3    -> 1.0
print(x)  # Salida: 1.0
```

Importante: Python **no tiene** `++` ni `--`. Escribir `x++` en una línea produce `SyntaxError`. Curiosamente, `++x` sí es válido pero **no incrementa nada**: se lee como `+(x)`, el mismo valor. Y `z = x++ + y` tampoco falla: Python lo lee como `x + (+(+y))`, así que `z` recibe `x + y` y `x` **no cambia**. Para incrementar se usa `x += 1`.

Operadores de comparación (devuelven `bool`)

Operador	Significado	Ejemplo	Resultado
<code>==</code>	Igual a	<code>5 == 5.0</code>	<code>True</code>
<code>!=</code>	Distinto de	<code>5 != 3</code>	<code>True</code>
<code>></code> / <code><</code>	Mayor / menor que	<code>3 > 7</code>	<code>False</code>
<code>>=</code> / <code><=</code>	Mayor o igual / menor o igual	<code>7 >= 7</code>	<code>True</code>

```

nota = 3.8
print(3.0 <= nota <= 5.0)  # Salida: True (comparaciones encadenadas, como en
                             matemáticas)
print("abc" < "abd")        # Salida: True (los textos se comparan letra por letra)
print("Zorro" < "abeja")    # Salida: True (las mayúsculas van antes que las minúsculas)

```

Operadores lógicos: `and`, `or`, `not`

a	b	a and b	a or b	not a
<code>True</code>	<code>True</code>	<code>True</code>	<code>True</code>	<code>False</code>
<code>True</code>	<code>False</code>	<code>False</code>	<code>True</code>	<code>False</code>
<code>False</code>	<code>True</code>	<code>False</code>	<code>True</code>	<code>True</code>
<code>False</code>	<code>False</code>	<code>False</code>	<code>False</code>	<code>True</code>

Cortocircuito: `and` deja de evaluar en cuanto encuentra un valor falso; `or`, en cuanto encuentra uno verdadero. Además, **devuelven uno de los operandos**, no necesariamente `True` o `False`:

```

print(0 or "valor por defecto")  # Salida: valor por defecto
print("" and 5)                  # Salida: (texto vacío)
print(3 and 5)                   # Salida: 5

lista = []
if lista and lista[0] > 10:      # no falla: si la lista está vacía no evalúa lista[0]
    print("Primer elemento grande")

nombre = input("Nombre: ") or "Invitado"  # si el usuario no escribe nada

```

Operadores de identidad y pertenencia

Operador	Pregunta	Ejemplo
<code>is</code> / <code>is not</code>	¿Es el mismo objeto en memoria?	<code>x is None</code>
<code>in</code> / <code>not in</code>	¿Está contenido?	<code>"a" in "casa"</code> , <code>3 in [1, 2, 3]</code>

```

a = [1, 2]
b = [1, 2]

```

```
c = a
print(a == b)    # Salida: True    (mismo contenido)
print(a is b)    # Salida: False   (objetos distintos)
print(a is c)    # Salida: True    (c apunta al mismo objeto que a)
```

Regla práctica: usa `==` para comparar valores y `is` solo para comparar con `None`, `True` o `False`.

Operadores a nivel de bits

`&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` y `>>` (desplazamientos). Se usan en robótica y electrónica para manejar registros y sensores. `&` y `|` **no hacen cortocircuito**. Con conjuntos, `&` es intersección y `|` es unión.

```
print(6 & 3, 6 | 3, 6 ^ 3)    # Salida: 2 7 5    (110 & 011 = 010, etc.)
print(1 << 3)                 # Salida: 8        (desplazar un bit tres posiciones = × 8)
```

Precedencia (de mayor a menor)

1. `()` paréntesis
2. `**` potencia (se asocia de derecha a izquierda: `2 ** 3 ** 2 = 2 ** 9 = 512`)
3. `+`, `-` (signo)
4. `*`, `/`, `//`, `%`
5. `+`, `-`
6. Comparaciones: `==`, `!=`, `<`, `>`, `<=`, `>=`, `is`, `in`
7. `not`
8. `and`
9. `or`

```
print(2 + 3 * 4)            # Salida: 14
print((2 + 3) * 4)          # Salida: 20
print(-2 ** 2)              # Salida: -4    (la potencia va antes que el signo)
print(not True or True)     # Salida: True  -> (not True) or True
```

Caso real: préstamo con interés simple y cuotas

```
monto = 2_000_000
tasa_anual = 0.24
meses = 12

interes = monto * tasa_anual * (meses / 12)
total = monto + interes
cuota = total / meses
print(f"Interés: ${interes:,.0f}")    # Salida: Interés: $480,000
print(f"Cuota:   ${cuota:,.2f}")      # Salida: Cuota:   $206,666.67
```

Reto 4.1

Pide un número de segundos y muéstralo en formato `HH:MM:SS` usando solo `//` y `%`.

Ejemplo: `3725` → `01:02:05`.

Preguntas de repaso

- ¿Qué devuelve `7 // 2` y qué devuelve `7 / 2`? `3` (división entera) y `3.5` (división real).
- ¿Qué imprime `print(10 % 3 == 1 and not 5 > 8)`? `True`.
- ¿Qué diferencia hay entre `and` y `&`? `and` es lógico y hace cortocircuito; `&` opera bit a bit (o intersección de conjuntos) y evalúa siempre ambos lados.

Módulo 5: Textos (strings), entrada y salida

Un `str` es una **secuencia inmutable de caracteres**: se puede recorrer y leer por posición, pero no se puede modificar un carácter; cualquier "cambio" crea un texto nuevo.

Índices y rebanadas (slicing)

```
texto = "Python"
#      P y t h o n
#      0 1 2 3 4 5      (índices positivos)
#     -6 -5 -4 -3 -2 -1  (índices negativos)
print(texto[0], texto[-1]) # Salida: P n
print(texto[0:3])          # Salida: Pyt   [inicio:fin) el fin NO se incluye
print(texto[2:])           # Salida: thon
print(texto[:2])           # Salida: Pto   (de 2 en 2)
print(texto[::-1])         # Salida: nohtyP (invertido)
print(len(texto))          # Salida: 6
```

Métodos más usados

Método	Qué hace	Ejemplo → Resultado
<code>upper()</code> / <code>lower()</code>	Mayúsculas / minúsculas	<code>"Hola".upper()</code> → <code>"HOLA"</code>
<code>title()</code> / <code>capitalize()</code>	Primera letra en mayúscula	<code>"ana pÉrez".title()</code> → <code>"Ana Pérez"</code>
<code>strip()</code>	Quita espacios al inicio y al final	<code>" hola ".strip()</code> → <code>"hola"</code>
<code>replace(a, b)</code>	Reemplaza	<code>"1.500".replace(".", "")</code> → <code>"1500"</code>
<code>split(sep)</code>	Divide en una lista	<code>"a,b,c".split(",")</code> → <code>['a', 'b', 'c']</code>
<code>sep.join(lista)</code>	Une una lista	<code>"-".join(["a", "b"])</code> → <code>"a-b"</code>
<code>find(x)</code>	Posición de <code>x</code> (o <code>-1</code>)	<code>"casa".find("s")</code> → <code>2</code>
<code>count(x)</code>	Cuántas veces aparece	<code>"banana".count("a")</code> → <code>3</code>
<code>startswith()</code> / <code>endswith()</code>	¿Empieza/termina con...?	<code>"foto.png".endswith(".png")</code> → <code>True</code>
<code>isdigit()</code> / <code>isalpha()</code>	¿Solo dígitos / solo letras?	<code>"123".isdigit()</code> → <code>True</code>

f-strings: la forma moderna de formatear

```
nombre = "Laura"
precio = 1234567.891
print(f"Hola {nombre}, tienes {len(nombre)} letras")
```

```
print(f"{precio:,.2f}")      # Salida: 1,234,567.89 (miles con coma, 2 decimales)
print(f"{0.1567:.1%}")      # Salida: 15.7%
print(f"[{'izq':<8}]")      # Salida: [izq      ]
print(f"[{'der':>8}]")      # Salida: [          der]
print(f"{7:03d}")           # Salida: 007
print(f"{precio=}")         # Salida: precio=1234567.891 (útil para depurar)
```

Para mostrar pesos colombianos con punto de miles: `f"${valor:,.0f}".replace(",", ".")`.

Caracteres especiales y textos de varias líneas

```
print("Línea 1\nLínea 2")    # \n salto de línea
print("Columna\tTabulada")  # \t tabulación
print("Ella dijo \"hola\"")  # comillas escapadas
ruta = r"C:\nueva\tabla"    # r"" = texto crudo: no interpreta las barras
mensaje = """Este texto
ocupa varias
líneas"""
```

Entrada de datos con `input()`

`input()` **siempre devuelve un `str`**. Si necesitas un número, hay que convertirlo.

```
nombre = input("¿Cómo te llamas? ")
edad = int(input("¿Cuántos años tienes? "))
print(f"{nombre}, en 10 años tendrás {edad + 10}")
```

Caso real: factura simple

```
producto = "Audífonos inalámbricos"
cantidad = 2
precio_unitario = 89_900
IVA = 0.19

subtotal = cantidad * precio_unitario
iva = subtotal * IVA
total = subtotal + iva

print("=" * 40)
print(f"{'FACTURA DE VENTA':^40}")
print("=" * 40)
print(f"{producto:<25}{cantidad:>3} x {precio_unitario:>9,}")
print(f"{'Subtotal:':<25}{subtotal:>15,.0f}")
print(f"{'IVA 19%:':<25}{iva:>15,.0f}")
print(f"{'TOTAL:':<25}{total:>15,.0f}")
```

Reto 5.1

Pide un nombre completo y muestra: cantidad de letras (sin espacios), el nombre en mayúsculas, las iniciales, el nombre invertido y cuántas vocales tiene.

Preguntas de repaso

- **¿Por qué `texto[0] = "J"` produce error?** Porque los `str` son inmutables. Se crea uno nuevo: `"J" + texto[1:]`.
- **¿Qué imprime `"Hola Mundo".split()` ?** `['Hola', 'Mundo']` : sin argumento, divide por espacios.
- **¿Qué tipo devuelve `input()` ?** Siempre `str`.

Módulo 6: Condicionales

Los condicionales permiten que el programa **tome decisiones**. Es la traducción directa de los diagramas de flujo con rombos que se hacen en Lógica de Programación.

if, elif y else

```
nota = float(input("Nota (0.0 a 5.0): "))

if nota < 0 or nota > 5:
    print("Nota inválida")
elif nota >= 4.5:
    print("Excelente")
elif nota >= 3.0:
    print("Aprobado")
else:
    print("Reprobado")
```

- Solo se ejecuta **el primer bloque** cuya condición sea verdadera; los demás se saltan.
- `elif` es la contracción de *else if*. Puede haber tantos como se necesiten.
- `else` es opcional y no lleva condición.

Condicionales anidados y condiciones compuestas

```
edad = 20
tiene_cedula = True

# Anidado
if edad >= 18:
    if tiene_cedula:
        print("Puede votar")

# Equivalente y más legible con and
if edad >= 18 and tiene_cedula:
    print("Puede votar")
```

Operador ternario (expresión condicional)

```
edad = 16
estado = "mayor" if edad >= 18 else "menor"
print(estado)  # Salida: menor
```

match / case (Python 3.10+)

Es el equivalente moderno del `switch` de otros lenguajes, pero más poderoso: compara valores y también **estructuras**.

```
comando = "subir"
match comando:
    case "subir" | "arriba":      # | significa "o"
        print("Moviendo hacia arriba")
    case "bajar":
        print("Moviendo hacia abajo")
    case _:
        print("Comando desconocido")

punto = (0, 5)
match punto:
    case (0, 0):
        print("Origen")
    case (0, y):
        print(f"Sobre el eje Y en {y}")    # Salida: Sobre el eje Y en 5
    case (x, y):
        print(f"Punto ({x}, {y})")
```

Caso real: tarifa de parqueadero

Un parqueadero cobra \$3.000 la primera hora, \$2.000 cada hora adicional o fracción, y tiene un tope diario de \$25.000. Los vehículos eléctricos tienen 50 % de descuento.

```
import math

minutos = 200
es_electrico = True

horas = math.ceil(minutos / 60)    # toda fracción cuenta como hora completa
if horas <= 1:
    valor = 3_000
else:
    valor = 3_000 + (horas - 1) * 2_000

valor = min(valor, 25_000)         # tope diario
if es_electrico:
    valor *= 0.5

print(f"{horas} horas -> ${valor:,.0f}")    # Salida: 4 horas -> $4,500
```

Reto 6.1

Pide tres números y muestra el mayor **sin usar** la función `max()`.

Reto 6.2

Pide un año y di si es bisiesto. Regla: divisible entre 4, excepto los divisibles entre 100 que no lo sean entre 400 (2000 sí, 1900 no, 2024 sí).

Reto 6.3

Calcula el Índice de Masa Corporal ($IMC = \text{peso} / \text{estatura}^2$) y clasifícalo: menor a 18.5 bajo peso, 18.5 a 24.9 normal, 25 a 29.9 sobrepeso, 30 o más obesidad.

Preguntas de repaso

- ¿Cuántos bloques se ejecutan en una cadena `if / elif / else` ? Como máximo uno.
- ¿Qué hace `case _` ? Atrapa cualquier valor que no coincidió con los casos anteriores.

Módulo 7: Ciclos

Un ciclo repite un bloque de código. Python tiene dos: `while` (repite **mientras** se cumpla una condición) y `for` (recorre **cada elemento** de una secuencia).

`while`

```
contador = 1
while contador <= 5:
    print(contador, end=" ")
    contador += 1          # si olvidas esta línea, el ciclo es infinito
# Salida: 1 2 3 4 5
```

Validar entradas es el uso típico de `while`:

```
clave = ""
while clave != "python123":
    clave = input("Contraseña: ")
print("Acceso concedido")
```

Python no tiene `do-while`. Se imita con `while True` y `break`:

```
while True:
    opcion = input("¿Continuar? (s/n): ").lower()
    if opcion in ("s", "n"):
        break
```

`for` y `range()`

```
for i in range(5):          # 0, 1, 2, 3, 4   (el fin no se incluye)
    print(i, end=" ")
print()
for i in range(2, 11, 2):    # desde 2 hasta 10, de 2 en 2
    print(i, end=" ")
print()
for i in range(10, 0, -3):   # cuenta regresiva: 10 7 4 1
    print(i, end=" ")
print()
for letra in "Hola":        # también recorre textos, listas, diccionarios...
    print(letra, end="-")
# Salida:
# 0 1 2 3 4
# 2 4 6 8 10
# 10 7 4 1
# H-o-l-a-
```

break, continue y else en ciclos

```
for n in range(1, 10):
    if n % 2 == 0:
        continue      # salta al siguiente número (ignora los pares)
    if n > 7:
        break          # termina el ciclo por completo
    print(n, end=" ")
# Salida: 1 3 5 7

# El else de un ciclo se ejecuta solo si el ciclo NO terminó con break
buscado = 7
for n in [2, 4, 6]:
    if n == buscado:
        print("Encontrado")
        break
else:
    print("No encontrado")    # Salida: No encontrado
```

Recorridos con enumerate y zip

```
frutas = ["mango", "lulo", "guayaba"]
precios = [3000, 1500, 2000]

for posicion, fruta in enumerate(frutas, start=1):
    print(posicion, fruta)      # 1 mango / 2 lulo / 3 guayaba

for fruta, precio in zip(frutas, precios):
    print(f"{fruta}: ${precio}")    # recorre dos listas en paralelo
```

Acumuladores y contadores

```
notas = [4.2, 3.1, 2.5, 4.8, 3.9]
suma = 0      # acumulador
aprobados = 0 # contador
for nota in notas:
    suma += nota
    if nota >= 3.0:
        aprobados += 1
print(f"Promedio: {suma / len(notas):.2f} Aprobados: {aprobados}")
# Salida: Promedio: 3.70 Aprobados: 4
```

Ciclos anidados

```
for fila in range(1, 4):
    for col in range(1, 4):
        print(f"{fila * col:3}", end=" ")
    print()
# Salida:
# 1 2 3
```

```
# 2 4 6
# 3 6 9
```

Caso real: menú de un cajero automático

```
saldo = 500_000
while True:
    print("\n1. Consultar 2. Depositar 3. Retirar 0. Salir")
    opcion = input("Opción: ")
    if opcion == "1":
        print(f"Saldo: ${saldo:,}")
    elif opcion == "2":
        saldo += int(input("Valor a depositar: "))
    elif opcion == "3":
        valor = int(input("Valor a retirar: "))
        if valor > saldo:
            print("Fondos insuficientes")
        elif valor % 10_000 != 0:
            print("El cajero solo entrega múltiplos de $10.000")
        else:
            saldo -= valor
    elif opcion == "0":
        break
    else:
        print("Opción inválida")
```

Reto 7.1

Pide un número y muestra su tabla de multiplicar del 1 al 10 con formato alineado.

Reto 7.2

Pide números hasta que el usuario escriba 0 y muestra el promedio de los números ingresados (sin contar el cero).

Reto 7.3 (FizzBuzz)

Imprime los números del 1 al 100, pero en los múltiplos de 3 escribe "Fizz", en los de 5 "Buzz" y en los de ambos "FizzBuzz". Es una de las preguntas más comunes en entrevistas.

Preguntas de repaso

- ¿Qué imprime `for i in range(3): pass` seguido de `print(i)`? 2: la variable del `for` conserva el último valor.
- ¿Cuándo usar `while` y cuándo `for`? `for` cuando sabes qué vas a recorrer o cuántas veces; `while` cuando depende de una condición (validar datos, menús, juegos).

Módulo 8: Listas, tuplas y conjuntos

Las colecciones guardan varios valores en una sola variable.

Colección	Sintaxis	¿Ordenada?	¿Modificable?	¿Duplicados?	Uso típico
<code>list</code>	<code>[1, 2, 3]</code>	Sí	Sí	Sí	Datos que cambian: carrito de compras
<code>tuple</code>	<code>(1, 2, 3)</code>	Sí	No	Sí	Datos fijos: coordenadas, fechas
<code>set</code>	<code>{1, 2, 3}</code>	No	Sí	No	Eliminar duplicados, pertenencia rápida
<code>dict</code>	<code>{"a": 1}</code>	Sí (orden de inserción)	Sí	Claves únicas	Buscar por clave (Módulo 9)

Listas

```
compras = ["arroz", "leche", "huevos"]
compras.append("café")           # agrega al final
compras.insert(0, "pan")         # agrega en una posición
compras.extend(["sal", "azúcar"])
compras.remove("leche")          # elimina por valor (el primero que encuentre)
ultimo = compras.pop()           # elimina y devuelve el último
print(compras)                   # Salida: ['pan', 'arroz', 'huevos', 'café', 'sal']
print(ultimo)                    # Salida: azúcar
print(len(compras), compras[1], compras[-1]) # Salida: 5 arroz sal
print("café" in compras)         # Salida: True
print(compras.index("huevos"))    # Salida: 2
```

Ordenar: `sort()` frente a `sorted()`

```
numeros = [5, 2, 9, 1]
ordenada = sorted(numeros)       # devuelve una lista NUEVA
numeros.sort(reverse=True)       # modifica la lista ORIGINAL y devuelve None
print(ordenada, numeros)         # Salida: [1, 2, 5, 9] [9, 5, 2, 1]

palabras = ["kiwi", "uva", "banano"]
print(sorted(palabras, key=len)) # Salida: ['uva', 'kiwi', 'banano']
```

Cuidado: asignar no es copiar

```
a = [1, 2, 3]
b = a                            # b y a son EL MISMO objeto
b.append(4)
print(a)                         # Salida: [1, 2, 3, 4]

c = a.copy()                     # copia real (también: list(a) o a[:])
c.append(5)
print(a)                         # Salida: [1, 2, 3, 4]
```

Funciones útiles con listas

```
ventas = [120, 340, 90, 510]
print(sum(ventas), max(ventas), min(ventas))    # Salida: 1060 510 90
print(sum(ventas) / len(ventas))                # Salida: 265.0
```

Comprensión de listas (list comprehension)

Una forma compacta de crear listas a partir de otras. Sintaxis: `[expresión for elemento in iterable if condición]`.

```
cuadrados = [n ** 2 for n in range(1, 6)]
pares = [n for n in range(20) if n % 2 == 0]
nombres = ["ana", "luis", "sofía"]
mayus = [n.title() for n in nombres]
print(cuadrados)    # Salida: [1, 4, 9, 16, 25]
print(mayus)        # Salida: ['Ana', 'Luis', 'Sofía']
```

Matrices (listas de listas)

```
matriz = [
    [1, 2, 3],
    [4, 5, 6],
]
print(matriz[1][2])    # Salida: 6 (fila 1, columna 2)
transpuesta = [[fila[c] for fila in matriz] for c in range(3)]
print(transpuesta)     # Salida: [[1, 4], [2, 5], [3, 6]]
```

Tuplas

```
punto = (4, -2)
x, y = punto            # desempaquetado
print(x, y)             # Salida: 4 -2
unico = (5,)            # tupla de un elemento: la coma es obligatoria
print(type((5)), type(unico)) # Salida: <class 'int'> <class 'tuple'>
```

Las tuplas son más livianas que las listas y, al ser inmutables, pueden usarse como **claves de diccionario**. Una función que "devuelve varios valores" en realidad devuelve una tupla.

Conjuntos (sets)

```
correos = ["a@x.com", "b@x.com", "a@x.com"]
unicos = set(correos)
print(len(unicos))      # Salida: 2

python = {"Ana", "Luis", "Sara"}
java = {"Luis", "Pedro"}
print(python & java)     # Salida: {'Luis'} intersección
print(python | java)    # unión (el orden al imprimir puede variar)
print(python - java)    # diferencia: están en python pero no en java
```

Ojo: `{}` crea un **diccionario vacío**, no un conjunto. El conjunto vacío es `set()`.

Reto 8.1

Guarda las temperaturas de una semana en una lista y muestra la máxima, la mínima, el promedio y los días que estuvieron por encima del promedio.

Reto 8.2

Escribe una función que elimine los duplicados de una lista **conservando el orden original**:

`[3, 1, 3, 2, 1, 5] → [3, 1, 2, 5]`.

Preguntas de repaso

- **¿Qué devuelve `lista.sort()`?** `None` : ordena la lista en su lugar. Un error clásico es escribir `lista = lista.sort()`.
- **¿Qué diferencia hay entre lista y tupla?** La lista se puede modificar; la tupla no.
- **¿Cómo se verifica si un valor está en una colección?** Con `in`. En un `set` esa búsqueda es muchísimo más rápida que en una lista.

Módulo 9: Diccionesarios

Un diccionario (`dict`) guarda pares **clave** → **valor**. En lugar de buscar por posición (como en una lista), se busca por una clave con significado: la cédula de una persona, el código de un producto, el nombre de una ciudad. Es la estructura más usada en Python profesional: el JSON que devuelven las APIs se convierte directamente en diccionarios.

Crear y acceder

```
estudiante = {
    "nombre": "Valentina",
    "edad": 19,
    "carrera": "Ingeniería de Sistemas",
    "notas": [4.2, 3.8, 4.5],
}
print(estudiante["nombre"])           # Salida: Valentina
print(estudiante.get("correo"))        # Salida: None    (no falla si la clave no existe)
print(estudiante.get("correo", "N/D")) # Salida: N/D    (valor por defecto)
# print(estudiante["correo"])          # KeyError: la clave no existe
print(len(estudiante))                 # Salida: 4 (cantidad de pares)
print("edad" in estudiante)            # Salida: True (in busca en las CLAVES)
```

Agregar, modificar y eliminar

```
producto = {"codigo": "P01", "precio": 12_000}
producto["stock"] = 30           # agrega (la clave no existía)
producto["precio"] = 11_500      # modifica (la clave ya existía)
del producto["codigo"]           # elimina
stock = producto.pop("stock")    # elimina y devuelve el valor
producto.update({"precio": 10_000, "marca": "Genérica"}) # varias a la vez
print(producto)                 # Salida: {'precio': 10000, 'marca': 'Genérica'}
```

Recorrer un diccionario

```
inventario = {"manzana": 50, "pera": 0, "uva": 120}

for clave in inventario:           # recorre las claves
    print(clave)

for valor in inventario.values():  # solo los valores
    print(valor)

for fruta, cantidad in inventario.items(): # clave y valor al tiempo (lo más común)
    estado = "agotado" if cantidad == 0 else f"{cantidad} unidades"
    print(f"{fruta}: {estado}")
```

Métodos principales

Método	Qué hace
<code>d.get(k, defecto)</code>	Devuelve el valor o <code>defecto</code> si no existe la clave
<code>d.keys()</code> / <code>d.values()</code> / <code>d.items()</code>	Vistas de claves, valores y pares
<code>d.update(otro)</code>	Agrega o sobrescribe con los pares de otro diccionario
<code>d.pop(k, defecto)</code>	Elimina y devuelve el valor
<code>d.setdefault(k, v)</code>	Devuelve <code>d[k]</code> ; si no existe, la crea con <code>v</code>
<code>d.copy()</code>	Copia superficial
<code>d1 d2</code>	Une dos diccionarios en uno nuevo (Python 3.9+)
<code>dict.fromkeys(lista, v)</code>	Crea un diccionario con esas claves y el mismo valor

Reglas importantes

- Las **claves deben ser inmutables** (hashables): `str`, `int`, `float`, `tuple`. Una lista no puede ser clave.
- Las claves son **únicas**: si repites una, el último valor reemplaza al anterior.
- Desde Python 3.7, los diccionarios **conservan el orden de inserción**.
- Los valores pueden ser de cualquier tipo, incluso otros diccionarios o listas.

Patrón clásico: contar ocurrencias

```
votos = ["azul", "rojo", "azul", "verde", "azul", "rojo"]
conteo = {}
for color in votos:
    conteo[color] = conteo.get(color, 0) + 1
print(conteo)           # Salida: {'azul': 3, 'rojo': 2, 'verde': 1}

ganador = max(conteo, key=conteo.get)
print(ganador)          # Salida: azul
```

Patrón clásico: agrupar

```
estudiantes = [("Ana", "Sistemas"), ("Luis", "Robótica"), ("Sara", "Sistemas")]
por_carrera = {}
for nombre, carrera in estudiantes:
    por_carrera.setdefault(carrera, []).append(nombre)
print(por_carrera)      # Salida: {'Sistemas': ['Ana', 'Sara'], 'Robótica': ['Luis']}
```

Comprensión de diccionarios

```
precios = {"cuaderno": 8_000, "lápiz": 1_500, "morrall": 95_000}
con_iva = {prod: round(p * 1.19) for prod, p in precios.items()}
```

```
baratos = {prod: p for prod, p in precios.items() if p < 10_000}
print(baratos)           # Salida: {'cuaderno': 8000, 'lápiz': 1500}
```

Diccionarios anidados y JSON

```
import json

pedido = {
    "id": 1024,
    "cliente": {"nombre": "Carlos", "ciudad": "Medellín"},
    "items": [
        {"producto": "Mouse", "cantidad": 1, "precio": 45_000},
        {"producto": "Pad", "cantidad": 2, "precio": 15_000},
    ],
}

print(pedido["cliente"]["ciudad"])           # Salida: Medellín
total = sum(i["cantidad"] * i["precio"] for i in pedido["items"])
print(total)                                 # Salida: 75000

texto_json = json.dumps(pedido, ensure_ascii=False) # dict -> texto JSON
de_vuelta = json.loads(texto_json)             # texto JSON -> dict
print(de_vuelta == pedido)                    # Salida: True
```

Caso real: inventario de una tienda

```
inventario = {
    "A01": {"nombre": "Arroz 1 kg", "precio": 4_800, "stock": 40},
    "B07": {"nombre": "Aceite 1 L", "precio": 12_500, "stock": 3},
    "C15": {"nombre": "Café 500 g", "precio": 18_900, "stock": 0},
}

def vender(codigo, cantidad):
    producto = inventario.get(codigo)
    if producto is None:
        return "Código inexistente"
    if producto["stock"] < cantidad:
        return f"Stock insuficiente de {producto['nombre']}"
    producto["stock"] -= cantidad
    return f"Venta: {cantidad} x {producto['nombre']} = ${producto['precio'] * cantidad:,}"

print(vender("A01", 3))    # Salida: Venta: 3 x Arroz 1 kg = $14,400
print(vender("C15", 1))    # Salida: Stock insuficiente de Café 500 g

por_reponer = [p["nombre"] for p in inventario.values() if p["stock"] < 5]
print(por_reponer)        # Salida: ['Aceite 1 L', 'Café 500 g']
valor_total = sum(p["precio"] * p["stock"] for p in inventario.values())
print(f"Inventario valorizado: ${valor_total:,}") # Salida: Inventario valorizado:
$215,100
```

Reto 9.1

Cuenta cuántas veces aparece cada palabra en un texto y muestra las dos más frecuentes.

Reto 9.2

Crea una agenda de contactos con un diccionario de diccionarios y funciones para agregar, buscar (sin importar mayúsculas) y eliminar contactos.

Reto 9.3

Invierte un diccionario (las claves pasan a ser valores y viceversa) usando una comprensión de diccionario.

Preguntas de repaso

- **¿Qué diferencia hay entre `d["x"]` y `d.get("x")`?** Si la clave no existe, el primero lanza `KeyError` y el segundo devuelve `None`.
- **¿Una lista puede ser clave de un diccionario?** No, porque es mutable. Una tupla sí.
- **¿Qué imprime `{"a": 1, "a": 2}`?** `{'a': 2}`: la clave repetida se sobrescribe.

Módulo 10: Funciones, recursividad y lambdas

Una **función** es un bloque de código con nombre que recibe datos (parámetros), hace una tarea y opcionalmente devuelve un resultado. Aplican el principio **DRY** (*Don't Repeat Yourself*): escribir una vez y reutilizar muchas.

Definir y llamar

```
def saludar(nombre):
    """Devuelve un saludo personalizado.""" # docstring: documentación
    return f"Hola, {nombre}"

mensaje = saludar("Camila")
print(mensaje) # Salida: Hola, Camila
print(saludar.__doc__) # Salida: Devuelve un saludo personalizado.
```

- `def` define la función; `return` devuelve el valor y **termina** la función.
- Una función sin `return` devuelve `None`.

Tipos de parámetros

```
def calcular_precio(valor, descuento=0.0, iva=0.19): # parámetros con valor por defecto
    base = valor * (1 - descuento)
    return base * (1 + iva)

print(calcular_precio(100_000)) # posicional
print(calcular_precio(100_000, 0.1)) # posicional
print(calcular_precio(100_000, iva=0.05)) # por nombre (keyword)

def promedio(*numeros): # *args: cantidad variable de argumentos (llega como tupla)
    return sum(numeros) / len(numeros)

def ficha(**datos): # **kwargs: argumentos con nombre (llega como diccionario)
    for clave, valor in datos.items():
        print(f"{clave}: {valor}")

print(promedio(4, 5, 3)) # Salida: 4.0
ficha(nombre="Luis", edad=22)
```

Devolver varios valores

```
def estadisticas(datos):
    return min(datos), max(datos), sum(datos) / len(datos) # devuelve una tupla

minimo, maximo, prom = estadisticas([3, 8, 1, 6])
print(minimo, maximo, prom) # Salida: 1 8 4.5
```

Anotaciones de tipo (type hints)

No son obligatorias ni cambian la ejecución, pero documentan el código y permiten que el editor detecte errores. Todo el código profesional moderno las usa.

```
def area_rectangulo(base: float, altura: float) -> float:
    return base * altura

def buscar(nombres: list[str], objetivo: str) -> int | None:
    return nombres.index(objetivo) if objetivo in nombres else None
```

Alcance de variables (scope)

```
contador = 0                                # variable global

def incrementar():
    global contador                          # sin esta línea, Python crearía una variable local
    contador += 1

def local():
    x = 10                                  # solo existe dentro de la función
    return x

incrementar()
print(contador)                             # Salida: 1
```

Evita `global` en lo posible: es mejor recibir parámetros y devolver resultados.

Error clásico: valores por defecto mutables

```
def agregar_mal(item, lista=[]):            # la lista se crea UNA sola vez
    lista.append(item)
    return lista

print(agregar_mal("a"))                    # Salida: ['a']
print(agregar_mal("b"))                    # Salida: ['a', 'b'] ¡sorpresa!

def agregar_bien(item, lista=None):
    if lista is None:
        lista = []
    lista.append(item)
    return lista
```

Recursividad

Una función recursiva **se llama a sí misma**. Siempre necesita un **caso base** que detenga las llamadas y un **caso recursivo** que acerque el problema al caso base.

```
def factorial(n: int) -> int:
    if n <= 1:                             # caso base
        return 1
```

```

    return n * factorial(n - 1)  # caso recursivo

def fibonacci(n: int) -> int:
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(factorial(5))  # Salida: 120
print([fibonacci(i) for i in range(10)])  # Salida: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

```

Python limita la profundidad de recursión (por defecto unas 1.000 llamadas); si se supera, lanza `RecursionError`.

Funciones lambda y funciones de orden superior

Una `lambda` es una función anónima de una sola expresión. Se usa como argumento de otras funciones.

```

cuadrado = lambda x: x ** 2
print(cuadrado(7))  # Salida: 49

estudiantes = [("Ana", 4.1), ("Luis", 3.2), ("Sara", 4.8)]
por_nota = sorted(estudiantes, key=lambda e: e[1], reverse=True)
print(por_nota[0])  # Salida: ('Sara', 4.8)

numeros = [1, 2, 3, 4, 5, 6]
print(list(map(lambda n: n * 10, numeros)))  # Salida: [10, 20, 30, 40, 50, 60]
print(list(filter(lambda n: n % 2 == 0, numeros)))  # Salida: [2, 4, 6]

```

En Python moderno se prefieren las comprensiones de listas a `map` y `filter` porque se leen mejor.

Caso real: validador de datos de registro

```

import re

def validar_registro(nombre: str, correo: str, edad: int) -> list[str]:
    errores = []
    if len(nombre.strip()) < 3:
        errores.append("El nombre debe tener al menos 3 letras")
    if not re.fullmatch(r"[\w.+-]+@[ \w-]+\.[\w.]+", correo):
        errores.append("Correo inválido")
    if not 14 <= edad <= 100:
        errores.append("Edad fuera de rango")
    return errores

print(validar_registro("Jo", "jo@correo", 12))
# Salida: ['El nombre debe tener al menos 3 letras', 'Correo inválido', 'Edad fuera de rango']

```

Reto 10.1

Escribe `es_primo(n)` y úsala para listar los primos hasta `n`.

Reto 10.2

Escribe una función **recursiva** que sume los dígitos de un número: `suma_digitos(2026) → 10`.

Reto 10.3

Valida una contraseña: mínimo 8 caracteres, una mayúscula, un número y un símbolo. La función debe devolver la lista de reglas que no se cumplen.

Preguntas de repaso

- **¿Qué diferencia hay entre parámetro y argumento?** El parámetro es la variable en la definición (`def f(x)`); el argumento es el valor enviado en la llamada (`f(5)`).
- **¿Qué devuelve una función sin `return`?** `None`.
- **¿Qué es `*args`?** Un parámetro que recibe cualquier cantidad de argumentos posicionales en forma de tupla.

Módulo 11: Algoritmos sencillos resueltos

Un **algoritmo** es una secuencia finita de pasos para resolver un problema. Estos son los algoritmos clásicos que aparecen en clases, parciales y entrevistas. Intenta resolver cada uno antes de leer la solución.

1. Máximo de una lista sin usar `max()`

```
def maximo(numeros: list[float]) -> float:
    mayor = numeros[0]
    for n in numeros[1:]:
        if n > mayor:
            mayor = n
    return mayor

print(maximo([3, 17, 8, 21, 5])) # Salida: 21
```

2. Palíndromo

```
def es_palindromo(texto: str) -> bool:
    limpio = "".join(c.lower() for c in texto if c.isalnum())
    return limpio == limpio[::-1]

print(es_palindromo("Anita lava la tina")) # Salida: True
```

3. Contar vocales

```
def contar_vocales(texto: str) -> int:
    return sum(1 for letra in texto.lower() if letra in "aeiouáéíóú")

print(contar_vocales("Programación")) # Salida: 5
```

4. Números primos (criba de Eratóstenes)

```
def criba(n: int) -> list[int]:
    es_primo = [True] * (n + 1)
    es_primo[0] = es_primo[1] = False
    for i in range(2, int(n ** 0.5) + 1):
        if es_primo[i]:
            for multiplo in range(i * i, n + 1, i):
                es_primo[multiplo] = False
    return [i for i, primo in enumerate(es_primo) if primo]

print(criba(30)) # Salida: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

5. Máximo común divisor (algoritmo de Euclides)

```
def mcd(a: int, b: int) -> int:
    while b != 0:
        a, b = b, a % b
    return a

print(mcd(48, 18))          # Salida: 6
print(48 * 18 // mcd(48, 18)) # Salida: 144 (mínimo común múltiplo)
```

6. Conversión de decimal a binario

```
def a_binario(n: int) -> str:
    if n == 0:
        return "0"
    digitos = ""
    while n > 0:
        digitos = str(n % 2) + digitos
        n //= 2
    return digitos

print(a_binario(13), bin(13), int("1101", 2)) # Salida: 1101 0b1101 13
```

7. Búsqueda lineal y búsqueda binaria

```
def busqueda_lineal(lista, objetivo):
    for i, valor in enumerate(lista):
        if valor == objetivo:
            return i
    return -1

def busqueda_binaria(lista_ordenada, objetivo):
    izquierda, derecha = 0, len(lista_ordenada) - 1
    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2
        if lista_ordenada[medio] == objetivo:
            return medio
        if lista_ordenada[medio] < objetivo:
            izquierda = medio + 1
        else:
            derecha = medio - 1
    return -1

datos = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(busqueda_lineal(datos, 23), busqueda_binaria(datos, 23)) # Salida: 5 5
```

La búsqueda lineal revisa uno por uno (hasta n pasos). La binaria descarta la mitad en cada paso (unos $\log_2 n$ pasos): en un millón de datos, unos 20 pasos en lugar de un millón. Requiere la lista ordenada.

8. Ordenamiento burbuja (bubble sort)

```
def burbuja(lista: list) -> list:
    datos = lista.copy()
    n = len(datos)
    for i in range(n - 1):
        hubo_cambio = False
        for j in range(n - 1 - i):
            if datos[j] > datos[j + 1]:
                datos[j], datos[j + 1] = datos[j + 1], datos[j]
                hubo_cambio = True
        if not hubo_cambio:      # si no hubo intercambios, ya está ordenada
            break
    return datos

print(burbuja([64, 34, 25, 12, 22, 11, 90]))  # Salida: [11, 12, 22, 25, 34, 64, 90]
```

En la práctica se usa `sorted()`, que implementa **Timsort**, mucho más eficiente. Burbuja se estudia para entender cómo se ordena.

9. Anagramas

```
def son_anagramas(a: str, b: str) -> bool:
    return sorted(a.lower().replace(" ", "")) == sorted(b.lower().replace(" ", ""))

print(son_anagramas("amor", "roma"))  # Salida: True
```

10. Segundo mayor recorriendo una sola vez

```
def segundo_mayor(numeros: list[int]) -> int | None:
    primero = segundo = None
    for n in numeros:
        if primero is None or n > primero:
            primero, segundo = n, primero
        elif n != primero and (segundo is None or n > segundo):
            segundo = n
    return segundo

print(segundo_mayor([4, 9, 2, 9, 7]))  # Salida: 7
```

Complejidad algorítmica (notación Big O)

Notación	Nombre	Ejemplo
O(1)	Constante	Acceder a <code>lista[5]</code> o <code>diccionario["clave"]</code>
O(log n)	Logarítmica	Búsqueda binaria
O(n)	Lineal	Recorrer una lista, búsqueda lineal
O(n log n)	Lineal-logarítmica	<code>sorted()</code>

Notación	Nombre	Ejemplo
$O(n^2)$	Cuadrática	Burbuja, dos ciclos anidados sobre los mismos datos

Preguntas de repaso

- **¿Por qué buscar en un diccionario es más rápido que en una lista?** Porque el diccionario usa una tabla hash: calcula la posición a partir de la clave ($O(1)$ en promedio) en lugar de revisar elemento por elemento ($O(n)$).
- **¿Qué condición exige la búsqueda binaria?** Que los datos estén ordenados.

Módulo 12: Programación Orientada a Objetos I

La **Programación Orientada a Objetos (POO)** organiza el código alrededor de **objetos** que combinan datos (atributos) y comportamiento (métodos). Modela el mundo real: una cuenta bancaria tiene un saldo (dato) y se le puede depositar (comportamiento).

Concepto	Definición	Ejemplo
Clase	Plantilla o molde que define atributos y métodos	<code>CuentaBancaria</code>
Objeto / instancia	Un elemento concreto creado a partir de la clase	La cuenta de Ana, la cuenta de Luis
Atributo	Dato que guarda el objeto (su estado)	<code>titular</code> , <code>saldo</code>
Método	Función que pertenece a la clase (su comportamiento)	<code>depositar()</code> , <code>retirar()</code>
Constructor	Método que inicializa el objeto al crearlo	<code>__init__</code>
<code>self</code>	Referencia al propio objeto dentro de sus métodos	<code>self.saldo</code>

Un objeto se define por tres elementos: **estado** (valores de sus atributos), **comportamiento** (sus métodos) e **identidad** (es único en memoria, aunque otro tenga los mismos datos).

Primera clase

```
class CuentaBancaria:
    banco = "Banco CEIPA"  # atributo de CLASE: compartido por todas las
                           # cuentas

    def __init__(self, titular: str, saldo: float = 0):
        self.titular = titular  # atributos de INSTANCIA: propios de cada objeto
        self.saldo = saldo

    def depositar(self, valor: float) -> None:
        if valor <= 0:
            raise ValueError("El depósito debe ser positivo")
        self.saldo += valor

    def retirar(self, valor: float) -> None:
        if valor > self.saldo:
            raise ValueError("Fondos insuficientes")
        self.saldo -= valor

    def __str__(self) -> str:  # cómo se ve el objeto al imprimirlo
        return f"{self.titular}: ${self.saldo:,.0f}"

cuenta_ana = CuentaBancaria("Ana", 100_000)  # crear un objeto (instanciar)
cuenta_luis = CuentaBancaria("Luis")
cuenta_ana.depositar(50_000)
```

```
cuenta_ana.retirar(30_000)
print(cuenta_ana)           # Salida: Ana: $120,000
print(cuenta_luis.saldo)    # Salida: 0
print(cuenta_luis.banco)    # Salida: Banco CEIPA
```

Encapsulamiento en Python

Python no tiene `private` como Java. Se usan **convenciones**:

- `_saldo` (un guion bajo): "atributo interno, no lo toques desde fuera".
- `__saldo` (dos guiones bajos): Python cambia el nombre internamente (*name mangling*) para dificultar el acceso.
- `@property`: permite exponer un atributo con **validación**, sin cambiar la forma de usarlo.

```
class Producto:
    def __init__(self, nombre: str, precio: float):
        self.nombre = nombre
        self.precio = precio          # pasa por el setter de abajo

    @property
    def precio(self) -> float:        # getter
        return self._precio

    @precio.setter
    def precio(self, valor: float) -> None: # setter con validación
        if valor <= 0:
            raise ValueError("El precio debe ser positivo")
        self._precio = valor

p = Producto("Mouse", 45_000)
p.precio = 39_900          # se usa como atributo, pero se valida
print(p.precio)            # Salida: 39900
# p.precio = -5            # ValueError: El precio debe ser positivo
```

Métodos de instancia, de clase y estáticos

```
class Temperatura:
    def __init__(self, celsius: float):
        self.celsius = celsius

    def a_fahrenheit(self) -> float:    # de instancia: usa self
        return self.celsius * 9 / 5 + 32

    @classmethod
    def desde_fahrenheit(cls, f: float): # de clase: constructor alternativo
        return cls((f - 32) * 5 / 9)

    @staticmethod
    def es_congelacion(celsius: float) -> bool: # estático: no usa self ni cls
        return celsius <= 0

t = Temperatura.desde_fahrenheit(212)
print(t.celsius, Temperatura.es_congelacion(-3)) # Salida: 100.0 True
```

Métodos especiales (dunder methods)

Los métodos con doble guion bajo permiten que tus objetos funcionen con operadores y funciones de Python.

```
class Vector:
    def __init__(self, x: float, y: float):
        self.x, self.y = x, y

    def __add__(self, otro):
        # permite v1 + v2
        return Vector(self.x + otro.x, self.y + otro.y)

    def __eq__(self, otro):
        # permite v1 == v2
        return self.x == otro.x and self.y == otro.y

    def __repr__(self):
        # representación técnica
        return f"Vector({self.x}, {self.y})"

print(Vector(1, 2) + Vector(3, 4))    # Salida: Vector(4, 6)
print(Vector(1, 2) == Vector(1, 2))  # Salida: True
```

Reto 12.1

Crea la clase `Producto` con nombre, precio (validado con `@property`) y stock. Agrega los métodos `precio_con_iva()`, `vender(cantidad)` (que lance error si no hay stock) y `__str__`.

Preguntas de repaso

- **¿Qué es `self`?** El primer parámetro de los métodos de instancia; representa al objeto que está ejecutando el método.
- **¿Qué diferencia hay entre atributo de clase y de instancia?** El de clase es compartido por todos los objetos; el de instancia es propio de cada uno.
- **¿Cuándo se ejecuta `__init__`?** Automáticamente al crear el objeto.

Módulo 13: Programación Orientada a Objetos II

Los cuatro pilares de la POO

Pilar	Idea	En Python
Encapsulamiento	Ocultar los detalles internos y proteger el estado	<code>_atributo</code> , <code>@property</code>
Herencia	Una clase hija reutiliza y extiende a una clase padre	<code>class Hija(Padre):</code>
Polimorfismo	El mismo mensaje produce respuestas distintas según el objeto	Métodos con el mismo nombre en varias clases
Abstracción	Mostrar solo lo esencial; definir "qué" sin decir "cómo"	Clases abstractas con <code>abc</code>

Herencia y `super()`

```
class Empleado:
    def __init__(self, nombre: str, salario: float):
        self.nombre = nombre
        self.salario = salario

    def pago_mensual(self) -> float:
        return self.salario

    def __str__(self) -> str:
        return f"{self.nombre}: ${self.pago_mensual():,.0f}"

class Vendedor(Empleado):
    # Vendedor hereda de Empleado
    def __init__(self, nombre: str, salario: float, ventas: float):
        super().__init__(nombre, salario) # reutiliza el constructor del padre
        self.ventas = ventas

    def pago_mensual(self) -> float:
        # sobrescribe (override) el método
        return self.salario + self.ventas * 0.05

class Gerente(Empleado):
    def pago_mensual(self) -> float:
        return super().pago_mensual() * 1.3 # bonificación del 30 %

nomina = [Empleado("Ana", 2_000_000), Vendedor("Luis", 1_800_000, 20_000_000),
Gerente("Sara", 5_000_000)]
for persona in nomina:
    print(persona) # polimorfismo: cada objeto responde a su manera
print(isinstance(nomina[1], Empleado)) # Salida: True (un Vendedor ES un Empleado)
```

Clases abstractas

Una clase abstracta **no se puede instanciar**; define un contrato que las hijas deben cumplir.

```
from abc import ABC, abstractmethod
import math

class Figura(ABC):
    @abstractmethod
    def area(self) -> float: ...

class Circulo(Figura):
    def __init__(self, radio: float):
        self.radio = radio
    def area(self) -> float:
        return math.pi * self.radio ** 2

class Rectangulo(Figura):
    def __init__(self, base: float, altura: float):
        self.base, self.altura = base, altura
    def area(self) -> float:
        return self.base * self.altura

figuras: list[Figura] = [Circulo(1), Rectangulo(3, 4)]
print([round(f.area(), 2) for f in figuras]) # Salida: [3.14, 12]
# Figura() -> TypeError: Can't instantiate abstract class Figura ...
```

Duck typing: "si camina como pato..."

En Python el polimorfismo no exige herencia: basta con que el objeto tenga el método que se necesita.

```
class Perro:
    def hablar(self): return "Guau"

class Robot:
    def hablar(self): return "Bip bip"

for ser in (Perro(), Robot()):
    print(ser.hablar()) # no comparten clase padre y funciona igual
```

dataclass: clases de datos en pocas líneas

```
from dataclasses import dataclass, field

@dataclass
class Estudiante:
    nombre: str
    codigo: str
    notas: list[float] = field(default_factory=list)

    def promedio(self) -> float:
        return sum(self.notas) / len(self.notas) if self.notas else 0.0
```

```
e = Estudiante("Mateo", "2026-001", [4.0, 3.5, 4.5])
print(e)                # Salida: Estudiante(nombre='Mateo', codigo='2026-001', notas=
[4.0, 3.5, 4.5])
print(e.promedio())     # Salida: 4.0
```

`@dataclass` genera automáticamente `__init__`, `__repr__` y `__eq__`. Con `@dataclass(frozen=True)` el objeto se vuelve inmutable.

Enumeraciones (Enum)

```
from enum import Enum

class EstadoPedido(Enum):
    CREADO = "creado"
    PAGADO = "pagado"
    ENVIADO = "enviado"

estado = EstadoPedido.PAGADO
print(estado.name, estado.value)    # Salida: PAGADO pagado
print(estado is EstadoPedido.PAGADO) # Salida: True
```

Composición: "tiene un" frente a "es un"

La herencia expresa "**es un**" (un `Vendedor` es un `Empleado`). La composición expresa "**tiene un**" (un `Carro` tiene un `Motor`). Regla de diseño: **prefiere la composición** cuando no hay una relación "es un" clara.

```
class Motor:
    def encender(self): return "Motor encendido"

class Carro:
    def __init__(self):
        self.motor = Motor()    # composición
    def arrancar(self):
        return self.motor.encender()

print(Carro().arrancar())      # Salida: Motor encendido
```

Reto 13.1

Crea una clase abstracta `Vehiculo` con el método abstracto `peaje()`, y las clases `Moto` (\$0), `Carro` (\$13.000) y `Camion` (\$20.000 más \$8.000 por cada eje adicional a 2). Calcula el total recaudado de una fila de vehículos.

Preguntas de repaso

- ¿Qué hace `super().__init__(...)`? Llama al constructor de la clase padre para no repetir código.

- **¿Qué es sobrescribir un método?** Definir en la clase hija un método con el mismo nombre que en la padre para cambiar su comportamiento.
- **¿Python permite herencia múltiple?** Sí: `class C(A, B):`. El orden de búsqueda de métodos se llama MRO (`C.__mro__`).

Módulo 14: Manejo de excepciones y archivos

Excepciones

Una **excepción** es un error que ocurre durante la ejecución. Si no se maneja, el programa se detiene. Manejarlas permite responder al error de forma controlada.

```
try:
    numero = int(input("Número: "))
    resultado = 100 / numero
except ValueError:
    print("Debes escribir un número entero")
except ZeroDivisionError:
    print("No se puede dividir entre cero")
else:
    print(f"Resultado: {resultado}")      # solo si NO hubo error
finally:
    print("Operación terminada")         # SIEMPRE se ejecuta
```

Excepciones más comunes

Excepción	Cuándo ocurre	Ejemplo
ValueError	Valor con el tipo correcto pero contenido inválido	<code>int("abc")</code>
TypeError	Operación con tipos incompatibles	<code>"a" + 1</code>
ZeroDivisionError	División entre cero	<code>5 / 0</code>
IndexError	Índice fuera de rango	<code>[1, 2][5]</code>
KeyError	Clave inexistente en un diccionario	<code>{"a": 1}["b"]</code>
AttributeError	Atributo o método inexistente	<code>"texto".push()</code>
FileNotFoundError	Archivo que no existe	<code>open("no_existe.txt")</code>
NameError	Variable no definida	<code>print(x_no_definida)</code>

Lanzar excepciones y crear las propias

```
class SaldoInsuficienteError(Exception):
    """Error de negocio: el retiro supera el saldo."""

def retirar(saldo: float, valor: float) -> float:
    if valor <= 0:
        raise ValueError("El valor debe ser positivo")
    if valor > saldo:
        raise SaldoInsuficienteError(f"Faltan ${valor - saldo:,.0f}")
    return saldo - valor
```

```
try:
    retirar(50_000, 80_000)
except SaldoInsuficienteError as error:
    print("No se pudo retirar:", error)    # Salida: No se pudo retirar: Faltan $30,000
```

Buenas prácticas: captura excepciones **específicas** (nunca un `except:` vacío que oculta todos los errores), muestra mensajes útiles y no uses excepciones para controlar el flujo normal del programa.

Archivos de texto

Siempre se abren con `with`: el archivo se cierra solo, incluso si ocurre un error.

Modo	Significado
"r"	Leer (por defecto). Error si no existe
"w"	Escribir. Borra el contenido anterior o crea el archivo
"a"	Agregar al final
"x"	Crear; error si ya existe

```
with open("bitacora.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Inicio del sistema\n")
    archivo.write("Sensor 1: OK\n")

with open("bitacora.txt", "a", encoding="utf-8") as archivo:
    archivo.write("Sensor 2: FALLA\n")

with open("bitacora.txt", encoding="utf-8") as archivo:
    for numero, linea in enumerate(archivo, start=1):
        print(numero, linea.strip())
```

Archivos CSV y JSON

```
import csv
import json

ventas = [
    {"producto": "Teclado", "cantidad": 3, "precio": 80000},
    {"producto": "Mouse", "cantidad": 5, "precio": 45000},
]

# Escribir y leer CSV (se abre en Excel)
with open("ventas.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.DictWriter(f, fieldnames=["producto", "cantidad", "precio"])
    escritor.writeheader()
    escritor.writerows(ventas)

with open("ventas.csv", encoding="utf-8") as f:
    total = sum(int(fila["cantidad"]) * int(fila["precio"]) for fila in csv.DictReader(f))
    print(f"Total vendido: ${total:,}")    # Salida: Total vendido: $465,000

# Escribir y leer JSON (formato de las APIs)
```

```

with open("ventas.json", "w", encoding="utf-8") as f:
    json.dump(ventas, f, indent=2, ensure_ascii=False)

with open("ventas.json", encoding="utf-8") as f:
    datos = json.load(f)
print(datos[1]["producto"])           # Salida: Mouse

```

Rutas con `pathlib`

```

from pathlib import Path

carpeta = Path("reportes")
carpeta.mkdir(exist_ok=True)           # crea la carpeta si no existe
archivo = carpeta / "enero.txt"        # une rutas con / (funciona en Windows y
Linux)
archivo.write_text("Reporte de enero", encoding="utf-8")
print(archivo.exists(), archivo.suffix) # Salida: True .txt
print([p.name for p in carpeta.glob("*.txt")]) # Salida: ['enero.txt']

```

Reto 14.1

Lee un archivo `notas.csv` con columnas `estudiante,nota1,nota2,nota3`, calcula el promedio de cada estudiante e indica si aprobó (≥ 3.0). Si una fila tiene una nota inválida, muestra un aviso con el número de línea y continúa con las demás.

Preguntas de repaso

- ¿Cuándo se ejecuta el bloque `else` de un `try`? Solo si no ocurrió ninguna excepción.
- ¿Por qué usar `with open(...)`? Porque cierra el archivo automáticamente.
- ¿Qué diferencia hay entre los modos `"w"` y `"a"`? `"w"` borra el contenido; `"a"` agrega al final.

Módulo 15: Módulos, paquetes, entornos virtuales y librería estándar

Módulos e importaciones

Un **módulo** es cualquier archivo `.py`. Un **paquete** es una carpeta con módulos (tradicionalmente con un archivo `__init__.py`). Importar permite reutilizar código.

```
import math                # importa el módulo completo
from random import randint, choice  # importa solo lo que necesitas
import datetime as dt      # con alias

print(math.sqrt(144), math.pi)  # Salida: 12.0 3.141592653589793
print(randint(1, 6))            # número aleatorio entre 1 y 6 (incluidos)
print(choice(["piedra", "papel", "tijera"]))
print(dt.date.today().year)
```

Supón que creas `utilidades.py`:

```
# utilidades.py
def formato_pesos(valor: float) -> str:
    return "$" + f"{valor:,.0f}".replace(",", ".")

if __name__ == "__main__":
    # Este bloque solo se ejecuta si corres "python utilidades.py",
    # NO cuando otro archivo hace "import utilidades".
    print(formato_pesos(1500000))
```

Y en otro archivo de la misma carpeta: `from utilidades import formato_pesos`.

`if __name__ == "__main__":` es la forma estándar de separar "lo que hace el módulo al ejecutarse" de "lo que ofrece al ser importado". Todos los proyectos de esta guía la usan.

Librería estándar: "baterías incluidas"

Python trae cientos de módulos listos para usar sin instalar nada:

Módulo	Para qué sirve	Ejemplo
<code>math</code>	Funciones matemáticas	<code>math.ceil(4.1) → 5</code>
<code>random</code>	Números aleatorios, barajar	<code>random.shuffle(lista)</code>
<code>datetime</code>	Fechas y horas	<code>datetime.now()</code>
<code>os / pathlib</code>	Sistema operativo y rutas	<code>Path.home()</code>
<code>json / csv</code>	Formatos de datos	<code>json.loads(texto)</code>
<code>collections</code>	<code>Counter</code> , <code>deque</code> , <code>defaultdict</code>	<code>Counter("banana")</code>

Módulo	Para qué sirve	Ejemplo
<code>statistics</code>	Media, mediana, desviación	<code>statistics.median([3, 1, 2])</code>
<code>re</code>	Expresiones regulares	<code>re.findall(r"\d+", "alb22")</code>
<code>sqlite3</code>	Base de datos SQL en un archivo	Módulo 23
<code>unittest</code>	Pruebas unitarias	Módulo 27
<code>tkinter</code>	Interfaces gráficas de escritorio	Módulo 24
<code>threading</code>	Hilos (tareas concurrentes)	Ver abajo

```

from collections import Counter, deque
from datetime import datetime, timedelta
import statistics

print(Counter("mississippi").most_common(2))    # Salida: [('i', 4), ('s', 4)]
cola = deque([1, 2, 3])
cola.appendleft(0); cola.pop()
print(list(cola))                               # Salida: [0, 1, 2]
entrega = datetime(2026, 10, 5) + timedelta(days=15)
print(entrega.strftime("%d/%m/%Y"))             # Salida: 20/10/2026
print(statistics.mean([4.5, 3.0, 4.0]))         # Salida: 3.8333333333333335

```

Hilos (threads) en una mirada

Un **hilo** es una línea de ejecución dentro de un programa; permite hacer varias tareas "al mismo tiempo", por ejemplo descargar varios archivos mientras la interfaz sigue respondiendo.

```

import threading
import time

def descargar(nombre: str, segundos: float) -> None:
    time.sleep(segundos)           # simula una espera de red
    print(f"{nombre} descargado")

hilos = [threading.Thread(target=descargar, args=(f"archivo{i}", 0.2)) for i in range(3)]
for h in hilos:
    h.start()                     # inicia el hilo
for h in hilos:
    h.join()                      # espera a que termine
print("Todo listo")              # tarda unos 0.2 s en total, no 0.6 s

```

En CPython tradicional, el *GIL* (Global Interpreter Lock) permite que solo un hilo ejecute código Python a la vez, por eso los hilos ayudan sobre todo en tareas de **espera** (red, disco). Para cálculo intensivo se usa `multiprocessing`. Desde Python 3.14, la versión *free-threaded* (sin GIL) tiene soporte oficial.

pip y PyPI

PyPI (pypi.org) es el repositorio público con cientos de miles de paquetes. `pip` los instala:

```

pip install requests          # instala
pip install "fastapi[standard]" # instala con extras
pip list                      # lista lo instalado
pip freeze > requirements.txt # guarda las versiones exactas del proyecto
pip install -r requirements.txt # instala todo lo del archivo (en otro computador)

```

Entornos virtuales (venv)

Cada proyecto debería tener su propio entorno, para que las versiones de librerías de un proyecto no choquen con las de otro.

```

python -m venv .venv          # crea el entorno en la carpeta .venv
.venv\Scripts\activate        # activar en Windows
source .venv/bin/activate     # activar en macOS / Linux
pip install requests          # se instala SOLO en este entorno
deactivate                    # salir del entorno

```

Cuando el entorno está activo, la terminal muestra `(.venv)` al inicio. La carpeta `.venv` **no se sube a GitHub** (se ignora con `.gitignore`); se sube el `requirements.txt`.

Herramientas modernas de gestión de proyectos: `pyproject.toml` es el archivo estándar para describir un proyecto; **Poetry** y **uv** gestionan dependencias y entornos automáticamente (uv es muy rápido y cada vez más popular). Cumplen el papel que Maven y Gradle cumplen en Java.

Preguntas de repaso

- **¿Para qué sirve `if __name__ == "__main__":` ?** Para que cierto código solo se ejecute cuando el archivo se corre directamente y no cuando se importa.
- **¿Por qué usar entornos virtuales?** Para aislar las dependencias de cada proyecto y evitar conflictos de versiones.
- **¿Qué contiene `requirements.txt` ?** La lista de paquetes (y versiones) que necesita el proyecto.

Módulo 16: Proyecto — Calculadora de finanzas personales

Objetivo: construir una herramienta de consola que registre gastos, los agrupe por categoría, genere alertas y simule créditos y ahorros. Integra todo lo visto: variables, condicionales, ciclos, diccionarios, funciones, clases, excepciones y archivos.

Ubicación en el repositorio: `proyectos/01_finanzas_personales/finanzas.py`

Fórmulas financieras que usa

Cálculo	Fórmula	Significado
Interés compuesto	$VF = C \times (1 + r/n)^{(n \cdot t)}$	Cuánto vale hoy un ahorro dentro de t años con tasa anual r capitalizada n veces al año
Cuota fija (sistema francés)	$\text{cuota} = P \times i / (1 - (1 + i)^{-n})$	Cuota mensual de un crédito de monto P , tasa mensual i y n meses
Regla 50/30/20	50 % necesidades, 30 % deseos, 20 % ahorro	Guía sencilla para distribuir el ingreso

Diseño

- `Gasto` es una `dataclass` que **valida** sus datos en `__post_init__` (valor positivo, categoría válida).
- `Presupuesto` guarda los gastos en una lista y calcula total, saldo, resumen por categoría (un **diccionario**), alertas y exportación a CSV.
- Las fórmulas son **funciones puras** (no dependen de variables externas), lo que facilita probarlas.
- El menú usa `match / case` y valida cada número con `try / except`.

Código completo

 **Archivo:** `proyectos/01_finanzas_personales/finanzas.py`

```
"""
Calculadora de finanzas personales.
Módulos de la guía que aplica: 3 a 16 (tipos, diccionarios, funciones, clases, archivos).

Ejecutar: python finanzas.py
"""
from __future__ import annotations

import csv
from dataclasses import dataclass, field
from datetime import date
from pathlib import Path
```

```
CATEGORIAS = ("vivienda", "alimentacion", "transporte", "educacion", "ocio", "ahorro",
"otros")
```

```
@dataclass
```

```
class Gasto:
```

```
    descripcion: str
```

```
    valor: float
```

```
    categoria: str = "otros"
```

```
    fecha: date = field(default_factory=date.today)
```

```
    def __post_init__(self) -> None:
```

```
        if self.valor <= 0:
```

```
            raise ValueError("El valor del gasto debe ser positivo")
```

```
        if self.categoria not in CATEGORIAS:
```

```
            raise ValueError(f"Categoría inválida: {self.categoria}")
```

```
class Presupuesto:
```

```
    """Registra ingresos y gastos de un mes y genera reportes."""
```

```
    def __init__(self, ingreso_mensual: float) -> None:
```

```
        if ingreso_mensual <= 0:
```

```
            raise ValueError("El ingreso debe ser positivo")
```

```
        self.ingreso_mensual = ingreso_mensual
```

```
        self.gastos: list[Gasto] = []
```

```
    def agregar(self, gasto: Gasto) -> None:
```

```
        self.gastos.append(gasto)
```

```
    def total_gastado(self) -> float:
```

```
        return sum(g.valor for g in self.gastos)
```

```
    def saldo(self) -> float:
```

```
        return self.ingreso_mensual - self.total_gastado()
```

```
    def por_categoria(self) -> dict[str, float]:
```

```
        resumen: dict[str, float] = {}
```

```
        for g in self.gastos:
```

```
            resumen[g.categoria] = resumen.get(g.categoria, 0) + g.valor
```

```
        # Ordenado de mayor a menor gasto
```

```
        return dict(sorted(resumen.items(), key=lambda par: par[1], reverse=True))
```

```
    def regla_50_30_20(self) -> dict[str, float]:
```

```
        """Sugerencia clásica: 50 % necesidades, 30 % deseos, 20 % ahorro."""
```

```
        return {
```

```
            "necesidades": self.ingreso_mensual * 0.50,
```

```
            "deseos": self.ingreso_mensual * 0.30,
```

```
            "ahorro": self.ingreso_mensual * 0.20,
```

```
        }
```

```
    def alertas(self) -> list[str]:
```

```
        mensajes = []
```

```
        if self.saldo() < 0:
```

```
            mensajes.append(f"Gastaste {formato_pesos(-self.saldo())} más de lo que
```

```
ganas.")
```

```
        for categoria, valor in self.por_categoria().items():
```

```
            porcentaje = valor / self.ingreso_mensual * 100
```

```
            if categoria == "ocio" and porcentaje > 30:
```

```
                mensajes.append(f"Ocio consume el {porcentaje:.1f} % del ingreso (sugerido:
```

```
30 % o menos).")
```

```

        return mensajes

    def exportar_csv(self, ruta: str | Path) -> Path:
        ruta = Path(ruta)
        with ruta.open("w", newline="", encoding="utf-8") as archivo:
            escritor = csv.writer(archivo)
            escritor.writerow(["fecha", "descripcion", "categoria", "valor"])
            for g in self.gastos:
                escritor.writerow([g.fecha.isoformat(), g.descripcion, g.categoria,
g.valor])
        return ruta

# ----- Fórmulas financieras -----

def formato_pesos(valor: float) -> str:
    """1234567.8 -> '$1.234.568' (formato colombiano, sin decimales)."""
    return "$" + f"{valor:,.0f}".replace(",", ".")

def interes_compuesto(capital: float, tasa_anual: float, anios: int, capitalizaciones: int
= 12) -> float:
    """Valor futuro:  $VF = C * (1 + r/n)^{(n * t)}$ ."""
    return capital * (1 + tasa_anual / capitalizaciones) ** (capitalizaciones * anios)

def cuota_credito(monto: float, tasa_mensual: float, meses: int) -> float:
    """Cuota fija (sistema francés):  $C = P * i / (1 - (1 + i)^{-n})$ ."""
    if tasa_mensual == 0:
        return monto / meses
    return monto * tasa_mensual / (1 - (1 + tasa_mensual) ** -meses)

def tabla_amortizacion(monto: float, tasa_mensual: float, meses: int) -> list[dict[str,
float]]:
    cuota = cuota_credito(monto, tasa_mensual, meses)
    saldo = monto
    filas = []
    for mes in range(1, meses + 1):
        interes = saldo * tasa_mensual
        abono = cuota - interes
        saldo -= abono
        filas.append({"mes": mes, "cuota": cuota, "interes": interes, "abono": abono,
"saldo": max(saldo, 0)})
    return filas

def meses_para_meta(meta: float, ahorro_mensual: float) -> int:
    """Cuántos meses ahorrando una cuota fija (sin intereses) para alcanzar una meta."""
    if ahorro_mensual <= 0:
        raise ValueError("El ahorro mensual debe ser positivo")
    meses = 0
    acumulado = 0.0
    while acumulado < meta:
        acumulado += ahorro_mensual
        meses += 1
    return meses

# ----- Menú -----

def pedir_numero(mensaje: str) -> float:

```

```

"""Pide un número; acepta coma o punto decimal (1,8 o 1.8). Sin separador de miles."""
while True:
    try:
        return float(input(mensaje).strip().replace(",", "."))
    except ValueError:
        print(" Escribe un número válido sin puntos de miles, por ejemplo 150000 o
1.8")

def menu() -> None:
    print("=== Calculadora de finanzas personales ===")
    presupuesto = Presupuesto(pedir_numero("Ingreso mensual: "))
    opciones = {
        "1": "Registrar gasto",
        "2": "Ver resumen",
        "3": "Simular crédito",
        "4": "Simular ahorro (interés compuesto)",
        "5": "Exportar a CSV",
        "0": "Salir",
    }
    while True:
        print()
        for clave, texto in opciones.items():
            print(f" {clave}. {texto}")
        opcion = input("Opción: ").strip()
        match opcion:
            case "1":
                descripcion = input("Descripción: ")
                valor = pedir_numero("Valor: ")
                print("Categorías:", ", ".join(CATEGORIAS))
                categoria = input("Categoría: ").strip().lower() or "otros"
                try:
                    presupuesto.agregar(Gasto(descripcion, valor, categoria))
                    print(" Gasto registrado.")
                except ValueError as error:
                    print(" Error:", error)
            case "2":
                print(f"Ingreso: {formato_pesos(presupuesto.ingreso_mensual)}")
                print(f"Gastado: {formato_pesos(presupuesto.total_gastado())}")
                print(f"Saldo: {formato_pesos(presupuesto.saldo())}")
                for categoria, valor in presupuesto.por_categoria().items():
                    print(f" {categoria:<13} {formato_pesos(valor):>12}")
                for alerta in presupuesto.alertas():
                    print(" ALERTA:", alerta)
            case "3":
                monto = pedir_numero("Monto del crédito: ")
                tasa = pedir_numero("Tasa mensual en % (ej. 1.8): ") / 100
                meses = int(pedir_numero("Plazo en meses: "))
                cuota = cuota_credito(monto, tasa, meses)
                print(f" Cuota mensual: {formato_pesos(cuota)}")
                print(f" Total pagado: {formato_pesos(cuota * meses)}")
            case "4":
                capital = pedir_numero("Capital inicial: ")
                tasa = pedir_numero("Tasa efectiva anual en % (ej. 9): ") / 100
                anios = int(pedir_numero("Años: "))
                print(f" Valor futuro: {formato_pesos(interés_compuesto(capital, tasa,
anios))}")
            case "5":
                ruta = presupuesto.exportar_csv("gastos.csv")
                print(f" Archivo creado: {ruta.resolve()}")
            case "0":
                print("¡Hasta pronto!")

```

```

        break
    case _:
        print(" Opción no válida")

if __name__ == "__main__":
    menu()

```

Ejemplo de uso

```

from finanzas import Presupuesto, Gasto, cuota_credito, formato_pesos

p = Presupuesto(3_000_000)
p.agregar(Gasto("Arriendo", 1_200_000, "vivienda"))
p.agregar(Gasto("Salidas", 1_000_000, "ocio"))
p.agregar(Gasto("Mercado", 600_000, "alimentacion"))
print(p.por_categoria())    # {'vivienda': 1200000, 'ocio': 1000000, 'alimentacion':
600000}
print(p.alertas())          # ['Ocio consume el 33.3 % del ingreso (sugerido: 30 % o
menos).']
print(formato_pesos(cuota_credito(10_000_000, 0.018, 24)))    # $516.808

```

Retos de ampliación

1. Agrega metas de ahorro con fecha límite e indica cuánto hay que ahorrar al mes.
2. Guarda y carga los gastos desde un archivo JSON para no perderlos al cerrar.
3. Muestra la tabla de amortización del crédito mes a mes (la función `tabla_amortizacion` ya existe).
4. Con `matplotlib`, dibuja un gráfico de torta de los gastos por categoría.

Aviso: las cifras son para aprender a programar; no son asesoría financiera. Las tasas reales de los créditos dependen de cada entidad.

Módulo 17: Proyecto — Reproductor de música

Objetivo: modelar un reproductor con lista de reproducción, siguiente, anterior, pausa, modo aleatorio, repetición y búsqueda. Es un excelente ejercicio de **estructuras de datos**: cada función del reproductor corresponde a una estructura clásica.

Ubicación: `proyectos/02_reproductor_musica/reproductor.py`

Funcionalidad	Estructura de datos	Por qué
Cola de reproducción	<code>deque</code> (cola doble)	Sacar del inicio con <code>popleft()</code> es $O(1)$; en una lista es $O(n)$
Botón "anterior"	Lista usada como pila (LIFO)	La última canción escuchada es la primera en volver
Modos de repetición	<code>Enum</code>	Valores fijos y con nombre: <code>NO</code> , <code>UNA</code> , <code>TODAS</code>
Canción	<code>dataclass(frozen=True)</code>	Inmutable y comparable
Aleatorio	<code>random.shuffle</code>	Baraja la cola
Búsqueda	Comprensión de listas	Filtra por título o artista

Código completo

 **Archivo:** `proyectos/02_reproductor_musica/reproductor.py`

```
"""
Reproductor de música (simulado en consola).
Conceptos: listas, deque, dataclasses, enum, pilas (historial), random, patrón State
simple.
Si instalas pygame (pip install pygame) y las canciones tienen una ruta a un .mp3/.ogg
real,
el reproductor también suena.

Ejecutar: python reproductor.py
"""
from __future__ import annotations

import random
from collections import deque
from dataclasses import dataclass
from enum import Enum

try: # Audio opcional
    import pygame
    pygame.mixer.init()
    AUDIO = True
except Exception: # pygame no instalado o sin tarjeta de sonido
    AUDIO = False
```

```

@dataclass(frozen=True)
class Cancion:
    titulo: str
    artista: str
    duracion_seg: int
    ruta: str | None = None

    def duracion_texto(self) -> str:
        minutos, segundos = divmod(self.duracion_seg, 60)
        return f"{minutos}:{segundos:02d}"

    def __str__(self) -> str:
        return f"{self.titulo} - {self.artista} ({self.duracion_texto()})"

class Repetir(Enum):
    NO = "no"
    UNA = "una"
    TODAS = "todas"

class Reproductor:
    def __init__(self, canciones: list[Cancion]) -> None:
        if not canciones:
            raise ValueError("La lista de reproducción está vacía")
        self.original = list(canciones)
        self.colas: deque[Cancion] = deque(canciones)
        self.historial: list[Cancion] = [] # funciona como PILA
        self.actual: Cancion | None = None
        self.reproduciendo = False
        self.repetir = Repetir.NO
        self.aleatorio = False

    # --- controles ---
    def play(self) -> str:
        if self.actual is None:
            return self.siguiente()
        self.reproduciendo = True
        self._sonar()
        return f"Reproduciendo: {self.actual}"

    def pausa(self) -> str:
        self.reproduciendo = False
        if AUDIO:
            pygame.mixer.music.pause()
        return "En pausa"

    def siguiente(self) -> str:
        if self.repetir is Repetir.UNA and self.actual:
            return self.play()
        if self.actual:
            self.historial.append(self.actual)
        if not self.colas:
            if self.repetir is Repetir.TODAS:
                self.colas = deque(self.original)
            if self.aleatorio:
                self._barajar()
            else:
                self.actual, self.reproduciendo = None, False
            return "Fin de la lista"
        self.actual = self.colas.popleft()

```

```

        return self.play()

def anterior(self) -> str:
    if not self.historial:
        return "No hay canciones anteriores"
    if self.actual:
        self cola.appendleft(self.actual)
        self.actual = self.historial.pop()
    return self.play()

def alternar_aleatorio(self) -> str:
    self.aleatorio = not self.aleatorio
    if self.aleatorio:
        self._barajar()
    else:
        restantes = set(self cola)
        self cola = deque(c for c in self.original if c in restantes)
    return f"Aleatorio: {'sí' if self.aleatorio else 'no'}"

def cambiar_repetir(self) -> str:
    modos = list(Repetir)
    self.repetir = modos[(modos.index(self.repetir) + 1) % len(modos)]
    return f"Repetir: {self.repetir.value}"

def agregar(self, cancion: Cancion) -> None:
    self.original.append(cancion)
    self cola.append(cancion)

def duracion_total(self) -> str:
    total = sum(c.duracion_seg for c in self.original)
    horas, resto = divmod(total, 3600)
    minutos, segundos = divmod(resto, 60)
    return f"{horas}h {minutos}m {segundos}s"

def buscar(self, texto: str) -> list[Cancion]:
    texto = texto.lower()
    return [c for c in self.original if texto in c.titulo.lower() or texto in
c.artista.lower()]

# --- internos ---
def _barajar(self) -> None:
    lista = list(self cola)
    random.shuffle(lista)
    self cola = deque(lista)

def _sonar(self) -> None:
    if AUDIO and self.actual and self.actual.ruta:
        pygame.mixer.music.load(self.actual.ruta)
        pygame.mixer.music.play()

def demo() -> None:
    canciones = [
        Cancion("La Tierra del Olvido", "Carlos Vives", 272),
        Cancion("Colombia Tierra Querida", "Lucho Bermúdez", 185),
        Cancion("Fruta Fresca", "Carlos Vives", 257),
        Cancion("La Pollera Colorá", "Wilson Choperena", 198),
    ]
    rep = Reproductor(canciones)
    comandos = {
        "p": rep.play, "s": rep.siguiete, "a": rep.anterior, "z": rep.pausa,
        "r": rep.alternar_aleatorio, "t": rep.cambiar_repetir,
    }

```

```

}
print(f"Lista de {len(canciones)} canciones · duración {rep.duracion_total()}")
print("[p]lay [s]iguiente [a]nterior pau[z]a aleato[r]io repe[t]ir [b]uscar [q]salir")
while True:
    comando = input("> ").strip().lower()
    if comando == "q":
        break
    if comando == "b":
        for c in rep.buscar(input("Buscar: ")):
            print(" ", c)
        continue
    accion = comandos.get(comando)
    print(accion() if accion else "Comando no reconocido")

if __name__ == "__main__":
    demo()

```

Para que suene de verdad

1. `pip install pygame`
2. Crea objetos `Cancion` con la ruta a archivos de audio propios (`.mp3` u `.ogg`):


```
Cancion("Mi canción", "Yo", 180, ruta="musica/cancion.mp3").
```
3. El método `_sonar()` detecta que pygame está disponible y reproduce el archivo.

Retos de ampliación

1. Agrega un "me gusta" a las canciones y un modo que solo reproduzca las favoritas.
2. Guarda las listas de reproducción en JSON.
3. Crea una interfaz gráfica con `tkinter` (botones de play, pausa, siguiente).
4. Aplica el patrón **State** (Módulo 20): una clase por estado (`Reproduciendo` , `Pausado` , `Detenido`).

Módulo 18: Proyecto — Juegos sencillos

Los juegos son perfectos para practicar lógica: tienen reglas claras, estados que cambian y retroalimentación inmediata. Todos siguen el **ciclo de juego** (*game loop*):

1. **Leer entradas** (teclado, ratón).
2. **Actualizar el estado** (mover, detectar colisiones, sumar puntos).
3. **Dibujar** el estado en pantalla.
4. Repetir hasta que el juego termine.

18.1 Adivina el número (consola)

Conceptos: `random`, `for` con límite de intentos, validación con `isdigit()`.

 **Archivo:** `proyectos/03_juegos/adivina_numero.py`

```
"""Adivina el número: ciclos, condicionales, random y validación de entradas."""
import random

def jugar(minimo: int = 1, maximo: int = 100, intentos_max: int = 7) -> bool:
    secreto = random.randint(minimo, maximo)
    print(f"Pensé un número entre {minimo} y {maximo}. Tienes {intentos_max} intentos.")
    for intento in range(1, intentos_max + 1):
        texto = input(f"Intento {intento}: ")
        if not texto.isdigit():
            print(" Solo números enteros positivos.")
            continue
        numero = int(texto)
        if numero == secreto:
            print(f" ¡Correcto! Lo lograste en {intento} intento(s).")
            return True
        print(" Más alto" if numero < secreto else " Más bajo")
    print(f"Se acabaron los intentos. El número era {secreto}.")
    return False

if __name__ == "__main__":
    jugar()
```

Pregunta para pensar: ¿cuál es la mejor estrategia para adivinar un número entre 1 y 100 en máximo 7 intentos? (Pista: es la búsqueda binaria del Módulo 11; $2^7 = 128 \geq 100$.)

18.2 Rompecabezas deslizante (juego del 8 / del 15)

Conceptos: matrices (listas de listas), diccionario de direcciones, intercambio de valores, funciones pequeñas y reutilizables. El tablero se mezcla haciendo movimientos válidos desde la posición resuelta: así se **garantiza que siempre tenga solución** (la mitad de las

mezclas totalmente aleatorias no la tienen).

 **Archivo:** proyectos/03_juegos/rompecabezas.py

```
"""
Rompecabezas deslizante (juego del 15) en consola.
Conceptos: listas de listas (matrices), tuplas, funciones puras, random, ciclos.
Se mezcla haciendo movimientos válidos desde el tablero resuelto,
así el rompecabezas SIEMPRE tiene solución.

Ejecutar: python rompecabezas.py (tamaño 3 para 8-puzzle, 4 para 15-puzzle)
"""

import random

VACIO = 0
MOVIMIENTOS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)} # mueve el hueco

def crear_tablero(n: int) -> list[list[int]]:
    numeros = list(range(1, n * n)) + [VACIO]
    return [numeros[filas * n:(filas + 1) * n] for filas in range(n)]

def buscar_vacio(tablero: list[list[int]]) -> tuple[int, int]:
    for f, fila in enumerate(tablero):
        for c, valor in enumerate(fila):
            if valor == VACIO:
                return f, c
    raise ValueError("El tablero no tiene casilla vacía")

def mover(tablero: list[list[int]], tecla: str) -> bool:
    """Intercambia el hueco con la ficha vecina. Devuelve True si el movimiento fue
    válido."""
    if tecla not in MOVIMIENTOS:
        return False
    df, dc = MOVIMIENTOS[tecla]
    f, c = buscar_vacio(tablero)
    nf, nc = f + df, c + dc
    n = len(tablero)
    if not (0 <= nf < n and 0 <= nc < n):
        return False
    tablero[f][c], tablero[nf][nc] = tablero[nf][nc], tablero[f][c]
    return True

def mezclar(tablero: list[list[int]], pasos: int = 200) -> None:
    teclas = list(MOVIMIENTOS)
    for _ in range(pasos):
        mover(tablero, random.choice(teclas))

def esta_resuelto(tablero: list[list[int]]) -> bool:
    return tablero == crear_tablero(len(tablero))

def dibujar(tablero: list[list[int]]) -> str:
    ancho = len(str(len(tablero) * 2))
    lineas = []
    for fila in tablero:
        lineas.append(" ".join(str(v).rjust(ancho) if v else " " * ancho for v in fila))
```

```

return "\n".join(lineas)

def jugar(n: int = 3) -> None:
    tablero = crear_tablero(n)
    mezclar(tablero)
    movimientos = 0
    while not esta_resuelto(tablero):
        print("\n" + dibujar(tablero))
        tecla = input("Mover hueco [w/a/s/d] o q para salir: ").strip().lower()
        if tecla == "q":
            return
        if mover(tablero, tecla):
            movimientos += 1
        else:
            print("Movimiento no válido")
    print("\n" + dibujar(tablero))
    print(f";Resuelto en {movimientos} movimientos!")

if __name__ == "__main__":
    jugar(3)

```

18.3 Snake (gráfico con pygame)

Instalar: `pip install pygame`. **Conceptos:** coordenadas en una cuadrícula, la serpiente como lista de tuplas (se inserta la cabeza y se elimina la cola), colisiones, eventos de teclado, velocidad variable.

 **Archivo:** `proyectos/03_juegos/snake.py`

```

"""
Snake con pygame.
Instalar: pip install pygame
Ejecutar: python snake.py
Controles: flechas del teclado. Esc para salir.
Conceptos: ciclo de juego (game loop), listas como cola, tuplas, colisiones, eventos.
"""

import random
import sys

import pygame

CELDA = 20
COLUMNAS, FILAS = 30, 20
ANCHO, ALTO = COLUMNAS * CELDA, FILAS * CELDA
FPS = 10
NEGRO, VERDE, ROJO, BLANCO = (15, 15, 15), (40, 200, 90), (220, 60, 60), (240, 240, 240)

ARRIBA, ABAJO, IZQUIERDA, DERECHA = (0, -1), (0, 1), (-1, 0), (1, 0)
TECLAS = {pygame.K_UP: ARRIBA, pygame.K_DOWN: ABAJO, pygame.K_LEFT: IZQUIERDA,
pygame.K_RIGHT: DERECHA}

def nueva_comida(serpiente: list[tuple[int, int]]) -> tuple[int, int]:
    libres = [(x, y) for x in range(COLUMNAS) for y in range(FILAS) if (x, y) not in
serpiente]
    return random.choice(libres)

```

```

def main() -> None:
    pygame.init()
    pantalla = pygame.display.set_mode((ANCHO, ALTO))
    pygame.display.set_caption("Snake - Python desde Cero")
    reloj = pygame.time.Clock()
    fuente = pygame.font.SysFont(None, 28)

    serpiente = [(COLUMNAS // 2, FILAS // 2)] # la cabeza es serpiente[0]
    direccion = DERECHA
    comida = nueva_comida(serpiente)
    puntos = 0
    vivo = True

    while True:
        # 1. Eventos
        for evento in pygame.event.get():
            if evento.type == pygame.QUIT or (evento.type == pygame.KEYDOWN and evento.key
== pygame.K_ESCAPE):
                pygame.quit()
                sys.exit()
            if evento.type == pygame.KEYDOWN and evento.key in TECLAS:
                nueva = TECLAS[evento.key]
                # No se permite girar 180 grados sobre sí misma
                if (nueva[0] + direccion[0], nueva[1] + direccion[1]) != (0, 0):
                    direccion = nueva
            if evento.type == pygame.KEYDOWN and evento.key == pygame.K_SPACE and not vivo:
                serpiente, direccion, puntos, vivo = [(COLUMNAS // 2, FILAS // 2)],
DERECHA, 0, True
                comida = nueva_comida(serpiente)

        # 2. Lógica
        if vivo:
            cabeza = (serpiente[0][0] + direccion[0], serpiente[0][1] + direccion[1])
            choca_pared = not (0 <= cabeza[0] < COLUMNAS and 0 <= cabeza[1] < FILAS)
            if choca_pared or cabeza in serpiente:
                vivo = False
            else:
                serpiente.insert(0, cabeza)
                if cabeza == comida:
                    puntos += 1
                    comida = nueva_comida(serpiente)
                else:
                    serpiente.pop() # si no come, la cola avanza

        # 3. Dibujo
        pantalla.fill(NEGRO)
        pygame.draw.rect(pantalla, ROJO, (comida[0] * CELDA, comida[1] * CELDA, CELDA,
CELDA))
        for x, y in serpiente:
            pygame.draw.rect(pantalla, VERDE, (x * CELDA, y * CELDA, CELDA - 1, CELDA - 1))
        texto = f"Puntos: {puntos}" if vivo else f"Fin del juego ({puntos}). Espacio para
reiniciar"
        pantalla.blit(fuente.render(texto, True, BLANCO), (10, 10))
        pygame.display.flip()
        reloj.tick(FPS + puntos // 5) # la velocidad aumenta cada 5 puntos

if __name__ == "__main__":
    main()

```

Retos de ampliación

1. **Snake:** guarda el récord en un archivo de texto y muéstralo en pantalla.
2. **Snake:** agrega obstáculos o una comida especial que valga 3 puntos y desaparezca a los 5 segundos.
3. **Rompecabezas:** cuenta el tiempo con `time.perf_counter()` y guarda un ranking en JSON.
4. **Nuevo juego:** Triqui (tres en línea) para dos jugadores en consola, con detección de ganador en filas, columnas y diagonales.

Módulo 19: Principios KISS, DRY, YAGNI y SOLID

Los principios de diseño son reglas de sentido común que hacen que el código sea fácil de leer, probar y cambiar. No son leyes: son guías que un buen programador aplica con criterio.

Principios generales

Principio	Significado	En la práctica
KISS (<i>Keep It Simple, Stupid</i>)	Mantenlo simple	Si una función de 5 líneas resuelve el problema, no escribas una de 50
DRY (<i>Don't Repeat Yourself</i>)	No te repitas	Si copias y pegas el mismo código, conviértelo en una función
YAGNI (<i>You Aren't Gonna Need It</i>)	No lo vas a necesitar	No programes funciones "por si acaso" que nadie pidió

SOLID

S — Responsabilidad única (*Single Responsibility*)

Cada clase o función debe tener **una sola razón para cambiar**.

```
# Mal: una clase que calcula, guarda en archivo y envía correos
class ReporteMalo:
    def calcular(self): ...
    def guardar_pdf(self): ...
    def enviar_correo(self): ...

# Bien: cada responsabilidad en su propia clase
class CalculadoraVentas:
    def total(self, ventas: list[float]) -> float:
        return sum(ventas)

class ExportadorCSV:
    def exportar(self, datos: dict, ruta: str) -> None: ...

class NotificadorCorreo:
    def enviar(self, destino: str, mensaje: str) -> None: ...
```

O — Abierto/cerrado (*Open/Closed*)

El código debe estar **abierto a la extensión y cerrado a la modificación**: agregar un caso nuevo no debería obligar a editar lo que ya funciona.

```
# Mal: cada método de pago nuevo obliga a modificar esta función
def pagar_mal(metodo: str, valor: float) -> str:
```

```

if metodo == "tarjeta":
    return f"Tarjeta: {valor}"
elif metodo == "nequi":
    return f"Nequi: {valor}"

# Bien: se agregan clases nuevas sin tocar las existentes
class MetodoPago:
    def pagar(self, valor: float) -> str:
        raise NotImplementedError

class Tarjeta(MetodoPago):
    def pagar(self, valor: float) -> str:
        return f"Tarjeta: {valor}"

class BilleteraDigital(MetodoPago):
    def pagar(self, valor: float) -> str:
        return f"Billetera digital: {valor}"

def procesar(metodo: MetodoPago, valor: float) -> str:
    return metodo.pagar(valor)

```

L — Sustitución de Liskov

Una clase hija debe poder **reemplazar a su padre** sin romper el programa. Si `Pinguino` hereda de `Ave` y el método `volar()` lanza un error, se viola el principio: el diseño debería separar `Ave` de `AveVoladora`.

I — Segregación de interfaces

Es mejor tener **varias interfaces pequeñas** que una grande que obligue a implementar métodos que no se usan. Una impresora sencilla no debería estar obligada a implementar `escanear()` y `enviar_fax()`.

D — Inversión de dependencias

Las clases de alto nivel deben depender de **abstracciones**, no de detalles concretos. En Python se logra recibiendo las dependencias en el constructor (**inyección de dependencias**):

```

class RepositorioMemoria:
    def __init__(self):
        self.datos = []
    def guardar(self, item):
        self.datos.append(item)

class ServicioUsuarios:
    def __init__(self, repositorio):          # recibe la dependencia, no la crea
        self.repositorio = repositorio
    def registrar(self, nombre: str):
        self.repositorio.guardar(nombre)

servicio = ServicioUsuarios(RepositorioMemoria())  # en producción: un repositorio con
base de datos
servicio.registrar("Ana")

```

Así se puede cambiar la base de datos, o usar una falsa en las pruebas, sin tocar `ServicioUsuarios`. La API del Módulo 23 usa exactamente esta idea con `Depends` de FastAPI.

Preguntas de repaso

- **¿Qué principio se viola si una clase `Factura` calcula impuestos, genera el PDF y envía el correo?** Responsabilidad única (S).
- **¿Qué es la inyección de dependencias?** Entregar a un objeto las piezas que necesita (por parámetro) en lugar de que él mismo las cree.

Módulo 20: Patrones de diseño

Un **patrón de diseño** es una solución probada y reutilizable a un problema recurrente en el diseño de software. No es código para copiar, sino una **plantilla de solución** con nombre propio, lo que permite que los equipos se comuniquen rápido ("aquí usemos un Observer"). Se popularizaron con el libro de la "Banda de los Cuatro" (*Gang of Four*, 1994), que los clasifica en tres grupos:

Tipo	Propósito	Patrones de este módulo
Creacionales	Cómo se crean los objetos	Singleton, Factory
Estructurales	Cómo se combinan clases y objetos	Adapter, Decorator
De comportamiento	Cómo se comunican y reparten responsabilidades	Strategy, Observer

Todos los ejemplos están en `proyectos/04_patrones_diseno/` y se ejecutan con `python nombre_archivo.py`.

Strategy (comportamiento)

Problema: hay varias formas de hacer lo mismo (calcular una tarifa, un descuento, una ruta) y se quiere cambiar entre ellas sin llenar el código de `if`. **Solución:** encapsular cada algoritmo en su propia clase con la misma interfaz.

 **Archivo:** `proyectos/04_patrones_diseno/strategy_transporte.py`

```

"""Patrón Strategy: cambiar el algoritmo (tarifa) sin modificar a quien lo usa."""
from typing import Protocol

class EstrategiaTarifa(Protocol):
    def calcular(self, km: float) -> float: ...

class Metro:
    def calcular(self, km: float) -> float:
        return 3_950 # tarifa plana de ejemplo

class Taxi:
    def calcular(self, km: float) -> float:
        return 5_800 + km * 1_300 # banderazo + valor por km (ejemplo)

class Bicicleta:
    def calcular(self, km: float) -> float:
        return 0

class PlanificadorViaje:
    def __init__(self, estrategia: EstrategiaTarifa) -> None:
        self.estrategia = estrategia

```

```

def costo(self, km: float) -> float:
    return self.estrategia.calcular(km)

if __name__ == "__main__":
    for medio in (Metro(), Taxi(), Bicicleta()):
        viaje = PlanificadorViaje(medio)
        print(f"{type(medio).__name__:<10} 8 km -> ${viaje.costo(8):,.0f}")

```

Observer (comportamiento)

Problema: cuando un objeto cambia, varios otros deben enterarse (notificaciones, inventario, correo) sin que el objeto conozca los detalles de cada uno. **Solución:** los interesados se "suscriben" y el objeto les avisa. Es la base de los eventos en interfaces gráficas y de los sistemas de mensajería.

 **Archivo:** proyectos/04_patrones_diseno/observer_pedidos.py

```

"""Patrón Observer: varios objetos reaccionan cuando otro cambia de estado."""
from typing import Callable

Suscriptor = Callable[[str, str], None]

class Pedido:
    def __init__(self, codigo: str) -> None:
        self.codigo = codigo
        self.estado = "creado"
        self._suscriptores: list[Suscriptor] = []

    def suscribir(self, funcion: Suscriptor) -> None:
        self._suscriptores.append(funcion)

    def cambiar_estado(self, nuevo: str) -> None:
        self.estado = nuevo
        for notificar in self._suscriptores:
            notificar(self.codigo, nuevo)

    def enviar_sms(codigo: str, estado: str) -> None:
        print(f"[SMS] Pedido {codigo}: {estado}")

    def enviar_correo(codigo: str, estado: str) -> None:
        print(f"[Correo] Pedido {codigo} cambió a '{estado}'")

    def actualizar_inventario(codigo: str, estado: str) -> None:
        if estado == "pagado":
            print(f"[Bodega] Separar productos del pedido {codigo}")

if __name__ == "__main__":
    pedido = Pedido("A-102")
    for s in (enviar_sms, enviar_correo, actualizar_inventario):
        pedido.suscribir(s)

```

```
pedido.cambiar_estado("pagado")
pedido.cambiar_estado("enviado")
```

Factory (creacional)

Problema: el código cliente no debería saber qué clase concreta instanciar. **Solución:** una función o clase "fábrica" decide qué objeto crear a partir de un parámetro.

 **Archivo:** proyectos/04_patrones_diseno/factory_notificaciones.py

```
"""Patrón Factory: centraliza la creación de objetos según un parámetro."""
from abc import ABC, abstractmethod

class Notificador(ABC):
    @abstractmethod
    def enviar(self, destino: str, mensaje: str) -> str: ...

class Email(Notificador):
    def enviar(self, destino: str, mensaje: str) -> str:
        return f"Email a {destino}: {mensaje}"

class SMS(Notificador):
    def enviar(self, destino: str, mensaje: str) -> str:
        return f"SMS a {destino}: {mensaje[:160]}"

class WhatsApp(Notificador):
    def enviar(self, destino: str, mensaje: str) -> str:
        return f"WhatsApp a {destino}: {mensaje}"

_REGISTRO: dict[str, type[Notificador]] = {"email": Email, "sms": SMS, "whatsapp":
WhatsApp}

def crear_notificador(tipo: str) -> Notificador:
    try:
        return _REGISTRO[tipo.lower]()
    except KeyError:
        raise ValueError(f"Tipo de notificador desconocido: {tipo}") from None

if __name__ == "__main__":
    for canal in ("email", "sms", "whatsapp"):
        print(crear_notificador(canal).enviar("estudiante@ceipa.edu.co", "Tu nota fue
publicada"))
```

Singleton (creacional)

Problema: debe existir **una sola instancia** de algo (configuración, conexión a base de datos, registro de logs). **Solución:** controlar la creación en `__new__`. En Python, un módulo ya se comporta como singleton, por eso este patrón se usa menos que en Java.

 **Archivo:** proyectos/04_patrones_diseno/singleton_configuracion.py

```
"""Patrón Singleton: una sola instancia compartida (útil para configuración o
conexiones)."""

class Configuracion:
    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia.valores = {"moneda": "COP", "iva": 0.19}
        return cls._instancia

if __name__ == "__main__":
    a = Configuracion()
    b = Configuracion()
    a.valores["iva"] = 0.05
    print(a is b, b.valores["iva"])  # True 0.05
    # En Python, lo más simple suele ser un MÓDULO: los módulos ya se cargan una sola vez.
```

Decorator (estructural)

Problema: agregar comportamiento (medir tiempo, registrar logs, reintentar, validar permisos) sin modificar la función original. **Solución:** envolverla. Python tiene sintaxis propia para esto: `@decorador`. FastAPI, Flask y pytest usan decoradores en todas partes.

 **Archivo:** proyectos/04_patrones_diseno/decorator_medicion.py

```
"""Patrón Decorator (decoradores de Python): agrega comportamiento sin tocar la función."""
import functools
import time

def medir_tiempo(funcion):
    @functools.wraps(funcion)
    def envoltura(*args, **kwargs):
        inicio = time.perf_counter()
        resultado = funcion(*args, **kwargs)
        print(f"{funcion.__name__} tardó {time.perf_counter() - inicio:.4f} s")
        return resultado
    return envoltura

def reintentar(veces: int = 3):
    """Decorator con parámetros: reintenta una operación que puede fallar (p. ej. una API)."""
    def decorador(funcion):
        @functools.wraps(funcion)
        def envoltura(*args, **kwargs):
            for intento in range(1, veces + 1):
                try:
                    return funcion(*args, **kwargs)
                except ConnectionError as error:
                    print(f"Intento {intento} falló: {error}")
```

```

        raise ConnectionError(f"Falló después de {veces} intentos")
    return envoltura
return decorador

@medir_tiempo
def suma_hasta(n: int) -> int:
    return sum(range(n + 1))

if __name__ == "__main__":
    print(suma_hasta(1_000_000))

```

Adapter (estructural)

Problema: una librería externa tiene una interfaz distinta a la que espera tu sistema.

Solución: una clase intermedia que "traduce" (como un adaptador de enchufe).

 **Archivo:** proyectos/04_patrones_diseno/adapter_pagos.py

```

"""Patrón Adapter: hace compatible una clase externa con la interfaz que espera tu sistema."""

class PasarelaExterna:
    """Simula una librería de terceros con una firma distinta (valor en centavos)."""
    def make_payment(self, amount_cents: int, currency: str) -> dict:
        return {"ok": True, "charged": amount_cents, "currency": currency}

class ProcesadorPagos:
    """Interfaz que usa nuestra tienda: pagar(valor_en_pesos) -> bool."""
    def pagar(self, valor: float) -> bool:
        raise NotImplementedError

class AdaptadorPasarela(ProcesadorPagos):
    def __init__(self, externa: PasarelaExterna) -> None:
        self.externa = externa

    def pagar(self, valor: float) -> bool:
        respuesta = self.externa.make_payment(round(valor * 100), "COP")
        return respuesta["ok"]

if __name__ == "__main__":
    procesador: ProcesadorPagos = AdaptadorPasarela(PasarelaExterna())
    print("Pago aprobado:", procesador.pagar(45_900))

```

Otros patrones que vale la pena conocer

Patrón	Idea en una línea	Ejemplo cotidiano
Builder	Construir objetos complejos paso a paso	Armar un pedido de hamburguesa con ingredientes opcionales

Patrón	Idea en una línea	Ejemplo cotidiano
Facade	Una interfaz simple frente a un sistema complejo	Un botón "pagar" que coordina inventario, cobro y envío
State	El comportamiento cambia según el estado interno	El reproductor: el botón play hace algo distinto si está pausado
Repository	Aislar el acceso a datos detrás de una clase	<code>RepositorioLibros</code> del Módulo 23
Dependency Injection	Entregar las dependencias desde afuera	<code>Depends()</code> de FastAPI

Reto 20.1

Implementa un **convertor de moneda** con el patrón Strategy: estrategias `PesosADolares`, `PesosAEuros` y `DolaresAPesos` con tasas fijas de ejemplo. El programa debe permitir cambiar de estrategia en tiempo de ejecución.

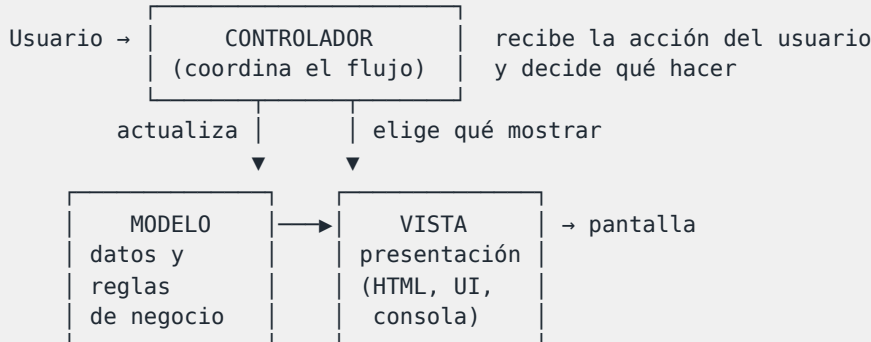
Preguntas de repaso

- **¿Qué es un patrón de diseño?** Una solución general, reutilizable y con nombre para un problema de diseño que se repite.
- **¿En qué se diferencian Strategy y Factory?** Factory decide **qué objeto crear**; Strategy permite **intercambiar el algoritmo** que usa un objeto.

Módulo 21: Patrones arquitectónicos

Si los patrones de diseño organizan clases, los **patrones arquitectónicos** organizan el **sistema completo**: cómo se dividen las grandes piezas y cómo se comunican.

MVC: Modelo - Vista - Controlador



Capa	Responsabilidad	En Django / Flask
Modelo	Datos, validaciones y reglas de negocio	Modelos del ORM
Vista	Mostrar la información	Plantillas HTML (Jinja2 / Django templates)
Controlador	Recibir peticiones, llamar al modelo, elegir la vista	Funciones de ruta (Flask) o <i>views</i> (Django)

Ventaja: se puede cambiar la interfaz (de consola a web) sin tocar las reglas de negocio. Django llama a su variante **MTV** (Modelo – Template – Vista), pero la idea es la misma.

 **Archivo:** proyectos/05_arquitectura_mvc/mvc_tareas.py

```

"""
Patrón arquitectónico MVC en consola: lista de tareas.
Modelo = datos y reglas | Vista = mostrar | Controlador = coordina entradas del usuario.
"""

from dataclasses import dataclass

# ----- MODELO -----

@dataclass
class Tarea:
    id: int
    titulo: str
    hecha: bool = False

class ModeloTareas:
    def __init__(self) -> None:
        self._tareas: dict[int, Tarea] = {}
        self._siguiente_id = 1

```

```

def agregar(self, titulo: str) -> Tarea:
    if not titulo.strip():
        raise ValueError("El título no puede estar vacío")
    tarea = Tarea(self._siguiente_id, titulo.strip())
    self._tareas[tarea.id] = tarea
    self._siguiente_id += 1
    return tarea

def completar(self, id_tarea: int) -> Tarea:
    if id_tarea not in self._tareas:
        raise KeyError(f"No existe la tarea {id_tarea}")
    self._tareas[id_tarea].hecha = True
    return self._tareas[id_tarea]

def listar(self) -> list[Tarea]:
    return list(self._tareas.values())

# ----- VISTA -----
class VistaConsola:
    def mostrar_tareas(self, tareas: list[Tarea]) -> None:
        if not tareas:
            print(" (sin tareas)")
        for t in tareas:
            print(f" [{'x' if t.hecha else ' '}] {t.id}. {t.titulo}")

    def mensaje(self, texto: str) -> None:
        print(f" > {texto}")

    def pedir(self, texto: str) -> str:
        return input(texto)

# ----- CONTROLADOR -----
class ControladorTareas:
    def __init__(self, modelo: ModeloTareas, vista: VistaConsola) -> None:
        self.modelo, self.vista = modelo, vista

    def ejecutar(self) -> None:
        while True:
            opcion = self.vista.pedir("\n[1] Agregar [2] Completar [3] Ver [0] Salir: ")
            try:
                if opcion == "1":
                    t = self.modelo.agregar(self.vista.pedir("Título: "))
                    self.vista.mensaje(f"Tarea {t.id} creada")
                elif opcion == "2":
                    t = self.modelo.completar(int(self.vista.pedir("Id: ")))
                    self.vista.mensaje(f"'{t.titulo}' completada")
                elif opcion == "3":
                    self.vista.mostrar_tareas(self.modelo.listar())
                elif opcion == "0":
                    break
            except (ValueError, KeyError) as error:
                self.vista.mensaje(f"Error: {error}")

if __name__ == "__main__":
    ControladorTareas(ModeloTareas(), VistaConsola()).ejecutar()

```

Arquitectura en capas

Presentación (API / controladores)	→ app/main.py	HTTP, JSON, códigos de estado
▼		
Servicio (lógica de negocio)	→ app/servicio.py	reglas: "no prestar dos veces"
▼		
Repositorio (acceso a datos)	→ app/repositorio.py	SQL, base de datos

Cada capa solo habla con la de abajo. Es la estructura del backend del Módulo 23 y la que usan la mayoría de APIs profesionales.

Cliente - servidor

El **cliente** (navegador, app móvil, script) envía **peticiones**; el **servidor** procesa y responde. En la web la comunicación usa HTTP. Los Módulos 22 a 24 construyen las dos partes.

Monolito frente a microservicios

Aspecto	Monolito	Microservicios
Estructura	Una sola aplicación con todos los módulos	Varios servicios pequeños e independientes
Despliegue	Todo junto	Cada servicio por separado
Base de datos	Normalmente una compartida	Cada servicio es dueño de sus datos
Escalabilidad	Se escala la aplicación completa	Se escala solo el servicio con más carga
Complejidad	Baja al inicio	Alta: red, fallos parciales, monitoreo
Cuándo usar	Proyectos pequeños y medianos, equipos pequeños	Sistemas grandes con varios equipos

Consejo profesional: casi todos los proyectos deberían empezar como un monolito bien organizado en capas. Los microservicios resuelven problemas de organizaciones grandes (Módulo 25).

Otras arquitecturas que vale la pena conocer

- **Hexagonal (puertos y adaptadores):** el núcleo de negocio no depende de frameworks ni bases de datos; todo lo externo se conecta mediante adaptadores.
- **Orientada a eventos:** los componentes se comunican publicando y escuchando eventos (Observer a gran escala, con herramientas como RabbitMQ o Kafka).
- **Serverless:** funciones que se ejecutan en la nube solo cuando se necesitan (AWS Lambda, Google Cloud Functions).

Preguntas de repaso

- **Explica MVC con un ejemplo.** En una tienda en línea: el Modelo tiene los productos y la regla de stock; la Vista es la página del catálogo; el Controlador recibe el clic en "comprar", pide al modelo descontar stock y muestra la vista de confirmación.
- **¿Cuál es la principal desventaja de los microservicios?** La complejidad operativa: comunicación por red, fallos parciales, trazabilidad y despliegues múltiples.

Módulo 22: Consumo de APIs REST

Una **API** (*Application Programming Interface*) es un contrato que permite que dos programas se comuniquen. Una **API REST** usa HTTP: el cliente pide un **recurso** (identificado por una URL) con un **verbo**, y el servidor responde con un **código de estado** y, normalmente, datos en **JSON**.

Verbos HTTP

Verbo	Acción (CRUD)	Ejemplo
GET	Leer	GET /libros · GET /libros/7
POST	Crear	POST /libros con el libro en el cuerpo
PUT	Reemplazar	PUT /libros/7
PATCH	Modificar parcialmente	PATCH /libros/7 con solo el campo que cambia
DELETE	Eliminar	DELETE /libros/7

Códigos de estado

Código	Significado	Cuándo
200 OK	Todo bien	Lectura o actualización exitosa
201 Created	Recurso creado	Después de un POST
204 No Content	Éxito sin cuerpo	Después de un DELETE
400 Bad Request	Petición mal formada	Datos incorrectos
401 / 403	No autenticado / sin permiso	Falta token o no tiene rol
404 Not Found	No existe	Id inexistente
409 Conflict	Conflicto con el estado actual	Prestar un libro ya prestado
422 Unprocessable Content	Validación fallida	FastAPI lo usa cuando los datos no cumplen el esquema
500 Internal Server Error	Error del servidor	Excepción no controlada

Regla rápida: **2xx** éxito, **4xx** error del cliente, **5xx** error del servidor.

Anatomía de una petición

```
GET https://api.open-meteo.com/v1/forecast?
latitude=6.24&longitude=-75.58&current=temperature_2m
|           |           |           |
|           |           |           | parámetros de consulta (query)
```

└ verbo

Encabezados (headers): Accept: application/json, Authorization: Bearer <token>

Cuerpo (body): solo en POST / PUT / PATCH, normalmente JSON

La librería `requests`

Instalar con `pip install requests`. Es la forma más popular de consumir APIs en Python.

```
import requests

r = requests.get("https://jsonplaceholder.typicode.com/users/1", timeout=10)
print(r.status_code)           # 200
usuario = r.json()             # el JSON se convierte en diccionario
print(usuario["name"], usuario["address"]["city"])
```

Buenas prácticas: usa siempre `timeout`; revisa el código de estado (`r.raise_for_status()` lanza una excepción si es 4xx o 5xx); captura `requests.RequestException`; nunca escribas claves de API en el código (usa variables de entorno con `os.getenv("API_KEY")`).

Caso real 1: clima actual de ciudades colombianas

Open-Meteo es una API pública gratuita que **no requiere registro ni clave**, ideal para clase.

 **Archivo:** `proyectos/06_consumo_api/clima_medellin.py`

```
"""
Consumo de una API REST pública (Open-Meteo, no requiere clave).
Instalar: pip install requests
Ejecutar: python clima_medellin.py
"""
import requests

URL = "https://api.open-meteo.com/v1/forecast"
CIUDADES = {
    "Medellín": (6.2442, -75.5812),
    "Bogotá": (4.7110, -74.0721),
    "Cartagena": (10.3910, -75.4794),
}

def clima_actual(latitud: float, longitud: float) -> dict:
    parametros = {
        "latitude": latitud,
        "longitude": longitud,
        "current": "temperature_2m,relative_humidity_2m,wind_speed_10m",
        "timezone": "America/Bogota",
    }
    respuesta = requests.get(URL, params=parametros, timeout=10)
    respuesta.raise_for_status()           # lanza excepción si el código HTTP es 4xx o 5xx
    return respuesta.json()["current"]     # JSON -> diccionario de Python

if __name__ == "__main__":
```

```

for ciudad, (lat, lon) in CIUDADES.items():
    try:
        datos = clima_actual(lat, lon)
        print(f"{ciudad:<10} {datos['temperature_2m']:>5} °C "
              f"humedad {datos['relative_humidity_2m']} % viento "
              f"{datos['wind_speed_10m']} km/h")
    except requests.RequestException as error:
        print(f"{ciudad}: no se pudo consultar ({error})")

```

Caso real 2: CRUD completo contra una API de práctica

JSONPlaceholder simula una API real: acepta POST, PUT y DELETE y responde como si guardara, sin guardar nada. Perfecta para practicar los cuatro verbos.

 **Archivo:** proyectos/06_consumo_api/jsonplaceholder_crud.py

```

"""
CRUD contra una API REST de práctica (JSONPlaceholder simula las escrituras).
Muestra los cuatro verbos HTTP principales y cómo leer códigos de estado.
"""
import requests

BASE = "https://jsonplaceholder.typicode.com"

def main() -> None:
    # GET: leer una colección con filtro por query string (?userId=1)
    r = requests.get(f"{BASE}/todos", params={"userId": 1}, timeout=10)
    tareas = r.json()
    pendientes = [t["title"] for t in tareas if not t["completed"]]
    print(f"GET {r.status_code}: el usuario 1 tiene {len(pendientes)} tareas pendientes")

    # POST: crear (se envía un diccionario que requests convierte a JSON)
    r = requests.post(f"{BASE}/posts", json={"title": "Hola", "body": "Desde Python",
"userId": 1}, timeout=10)
    print(f"POST {r.status_code}: creado con id {r.json()['id']}") # 201 Created

    # PUT: reemplazar un recurso completo
    r = requests.put(f"{BASE}/posts/1", json={"id": 1, "title": "Editado", "body": "...",
"userId": 1}, timeout=10)
    print(f"PUT {r.status_code}: título ahora es '{r.json()['title']}'") # 200 OK

    # DELETE: eliminar
    r = requests.delete(f"{BASE}/posts/1", timeout=10)
    print(f"DEL {r.status_code}") # 200 OK

if __name__ == "__main__":
    main()

```

APIs públicas para proyectos de clase

API	Datos	¿Requiere clave?
Open-Meteo	Clima y pronóstico	No

API	Datos	¿Requiere clave?
JSONPlaceholder	Datos falsos para practicar CRUD	No
PokéAPI	Información de Pokémon	No
REST Countries	Países, capitales, monedas	No
Datos Abiertos Colombia (datos.gov.co)	Datos oficiales del Estado colombiano	No, para consultas básicas

El repositorio **public-apis/public-apis** de GitHub mantiene una lista enorme de APIs gratuitas clasificadas por tema.

Reto 22.1

Consulta REST Countries o PokéAPI, pide un nombre al usuario y muestra tres datos del resultado. Maneja el caso en que el nombre no existe (código 404).

Preguntas de repaso

- **¿Qué devuelve `r.json()` ?** El cuerpo de la respuesta convertido en diccionario o lista de Python.
- **¿Qué código se espera al crear un recurso con POST?** 201 Created.
- **¿Por qué usar `timeout` ?** Para que el programa no se quede esperando indefinidamente si el servidor no responde.

Módulo 23: Backend — API REST con FastAPI y SQL

Ahora construimos el otro lado: **el servidor**. Usaremos **FastAPI**, un framework moderno y muy popular que valida los datos automáticamente con **Pydantic** y genera documentación interactiva (Swagger) sin escribir una línea extra. Los datos se guardan con **SQLite**, una base de datos SQL que viene incluida en Python (`sqlite3`).

Framework	Estilo	Cuándo usarlo
FastAPI	APIs modernas, validación automática, asíncrono	APIs REST y microservicios
Django	"Baterías incluidas": ORM, panel de administración, autenticación	Aplicaciones web completas
Flask	Minimalista y flexible	Proyectos pequeños o para aprender HTTP

Estructura del proyecto (arquitectura en capas)

```

proyectos/07_backend_fastapi/
├── app/
│   ├── __init__.py
│   ├── esquemas.py          # DTO: qué datos entran y salen (Pydantic)
│   ├── repositorio.py       # Acceso a datos (SQL)
│   ├── servicio.py          # Reglas de negocio
│   └── main.py               # Endpoints HTTP (controlador)
├── tests/
│   └── test_api.py          # Pruebas automáticas
└── requirements.txt
  
```

SQL esencial para el backend

```

CREATE TABLE libros (id INTEGER PRIMARY KEY AUTOINCREMENT, titulo TEXT NOT NULL, anio
INTEGER);
INSERT INTO libros (titulo, anio) VALUES ('María', 1867);           -- crear
SELECT * FROM libros WHERE titulo LIKE 'M%';                       -- leer (empieza por M)
SELECT COUNT(*) FROM libros;                                       -- contar registros
UPDATE libros SET anio = 1868 WHERE id = 1;                        -- actualizar
DELETE FROM libros WHERE id = 1;                                   -- eliminar
  
```

Seguridad: nunca construyas SQL concatenando texto del usuario (`f"... WHERE id = {id}"`): permite **inyección SQL**. Usa siempre parámetros `?`, como en el repositorio de abajo.

Esquemas (Pydantic)

 **Archivo:** proyectos/07_backend_fastapi/app/esquemas.py

```
"""Esquemas (DTO) validados con Pydantic: definen qué entra y qué sale de la API."""
from pydantic import BaseModel, Field

class LibroEntrada(BaseModel):
    titulo: str = Field(min_length=1, max_length=120, examples=["Cien años de soledad"])
    autor: str = Field(min_length=1, max_length=80, examples=["Gabriel García Márquez"])
    anio: int = Field(ge=1450, le=2100, examples=[1967])

class Libro(LibroEntrada):
    id: int
    prestado: bool = False
```

Repositorio (SQL con sqlite3)

 **Archivo:** proyectos/07_backend_fastapi/app/repositorio.py

```
"""Capa de acceso a datos: SQL puro con sqlite3 (viene incluido en Python)."""
import sqlite3

class RepositorioLibros:
    def __init__(self, ruta: str = "biblioteca.db") -> None:
        self.con = sqlite3.connect(ruta, check_same_thread=False)
        self.con.row_factory = sqlite3.Row # filas accesibles como diccionario
        self.con.execute(
            """CREATE TABLE IF NOT EXISTS libros (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                titulo        TEXT    NOT NULL,
                autor         TEXT    NOT NULL,
                anio          INTEGER NOT NULL,
                prestado       INTEGER NOT NULL DEFAULT 0)"""
        )
        self.con.commit()

    @staticmethod
    def _a_dict(fila: sqlite3.Row) -> dict:
        datos = dict(fila)
        datos["prestado"] = bool(datos["prestado"])
        return datos

    def listar(self, titulo: str | None = None) -> list[dict]:
        if titulo:
            # Consulta parametrizada con ? -> evita inyección SQL
            filas = self.con.execute("SELECT * FROM libros WHERE titulo LIKE ?", (f"%{titulo}%",))
        else:
            filas = self.con.execute("SELECT * FROM libros ORDER BY id")
        return [self._a_dict(f) for f in filas]

    def obtener(self, id_libro: int) -> dict | None:
        fila = self.con.execute("SELECT * FROM libros WHERE id = ?",
```

```

(id_libro,)).fetchone()
    return self._a_dict(fila) if fila else None

def crear(self, datos: dict) -> dict:
    cursor = self.con.execute(
        "INSERT INTO libros (titulo, autor, anio) VALUES (?, ?, ?)",
        (datos["titulo"], datos["autor"], datos["anio"]),
    )
    self.con.commit()
    return self.obtener(cursor.lastrowid)

def actualizar(self, id_libro: int, datos: dict) -> dict | None:
    self.con.execute(
        "UPDATE libros SET titulo = ?, autor = ?, anio = ? WHERE id = ?",
        (datos["titulo"], datos["autor"], datos["anio"], id_libro),
    )
    self.con.commit()
    return self.obtener(id_libro)

def marcar_prestado(self, id_libro: int, prestado: bool) -> dict | None:
    self.con.execute("UPDATE libros SET prestado = ? WHERE id = ?", (int(prestado),
id_libro))
    self.con.commit()
    return self.obtener(id_libro)

def eliminar(self, id_libro: int) -> bool:
    cursor = self.con.execute("DELETE FROM libros WHERE id = ?", (id_libro,))
    self.con.commit()
    return cursor.rowcount > 0

```

Servicio (reglas de negocio)

 **Archivo:** proyectos/07_backend_fastapi/app/servicio.py

```

"""Capa de servicio: reglas de negocio. No sabe nada de HTTP ni de SQL."""
from .repositorio import RepositorioLibros

class LibroNoEncontrado(Exception):
    pass

class LibroYaPrestado(Exception):
    pass

class ServicioLibros:
    def __init__(self, repositorio: RepositorioLibros) -> None:
        self.repo = repositorio

    def listar(self, titulo: str | None = None) -> list[dict]:
        return self.repo.listar(titulo)

    def obtener(self, id_libro: int) -> dict:
        libro = self.repo.obtener(id_libro)
        if libro is None:
            raise LibroNoEncontrado(f"No existe el libro {id_libro}")
        return libro

```

```

def crear(self, datos: dict) -> dict:
    return self.repo.crear(datos)

def actualizar(self, id_libro: int, datos: dict) -> dict:
    self.obtener(id_libro)
    return self.repo.actualizar(id_libro, datos)

def eliminar(self, id_libro: int) -> None:
    if not self.repo.eliminar(id_libro):
        raise LibroNoEncontrado(f"No existe el libro {id_libro}")

def prestar(self, id_libro: int) -> dict:
    libro = self.obtener(id_libro)
    if libro["prestado"]:
        raise LibroYaPrestado(f"'{libro['titulo']}' ya está prestado")
    return self.repo.marcar_prestado(id_libro, True)

def devolver(self, id_libro: int) -> dict:
    self.obtener(id_libro)
    return self.repo.marcar_prestado(id_libro, False)

```

Controlador (endpoints)

 **Archivo:** proyectos/07_backend_fastapi/app/main.py

```

"""
API REST de biblioteca con FastAPI (capa de presentación / controlador).
Instalar: pip install -r requirements.txt
Ejecutar: uvicorn app.main:app --reload      (desde la carpeta 07_backend_fastapi)
Documentación interactiva (Swagger): http://127.0.0.1:8000/docs
"""

from fastapi import Depends, FastAPI, Request, status
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from .esquemas import Libro, LibroEntrada
from .repositorio import RepositorioLibros
from .servicio import LibroNoEncontrado, LibroYaPrestado, ServicioLibros

app = FastAPI(title="API Biblioteca", version="1.0.0",
              description="Proyecto de la guía Python desde Cero")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=
["*"])

_servicio = ServicioLibros(RepositorioLibros("biblioteca.db"))

def obtener_servicio() -> ServicioLibros:
    """Inyección de dependencias: las pruebas pueden reemplazar este servicio."""
    return _servicio

# Manejo centralizado de errores de negocio -> códigos HTTP
@app.exception_handler(LibroNoEncontrado)
def manejar_no_encontrado(_: Request, error: LibroNoEncontrado) -> JSONResponse:
    return JSONResponse(status_code=404, content={"detalle": str(error)})

@app.exception_handler(LibroYaPrestado)

```

```

def manejar_ya_prestado(_: Request, error: LibroYaPrestado) -> JSONResponse:
    return JSONResponse(status_code=409, content={"detalle": str(error)})

@app.get("/libros", response_model=list[Libro])
def listar(titulo: str | None = None, servicio: ServicioLibros = Depends(obtener_servicio)):
    return servicio.listar(titulo)

@app.get("/libros/{id_libro}", response_model=Libro)
def obtener(id_libro: int, servicio: ServicioLibros = Depends(obtener_servicio)):
    return servicio.obtener(id_libro)

@app.post("/libros", response_model=Libro, status_code=status.HTTP_201_CREATED)
def crear(entrada: LibroEntrada, servicio: ServicioLibros = Depends(obtener_servicio)):
    return servicio.crear(entrada.model_dump())

@app.put("/libros/{id_libro}", response_model=Libro)
def actualizar(id_libro: int, entrada: LibroEntrada, servicio: ServicioLibros = Depends(obtener_servicio)):
    return servicio.actualizar(id_libro, entrada.model_dump())

@app.delete("/libros/{id_libro}", status_code=status.HTTP_204_NO_CONTENT)
def eliminar(id_libro: int, servicio: ServicioLibros = Depends(obtener_servicio)):
    servicio.eliminar(id_libro)

@app.post("/libros/{id_libro}/prestamo", response_model=Libro)
def prestar(id_libro: int, servicio: ServicioLibros = Depends(obtener_servicio)):
    return servicio.prestar(id_libro)

@app.delete("/libros/{id_libro}/prestamo", response_model=Libro)
def devolver(id_libro: int, servicio: ServicioLibros = Depends(obtener_servicio)):
    return servicio.devolver(id_libro)

```

Cómo ejecutarlo

```

cd proyectos/07_backend_fastapi
python -m venv .venv
source .venv/bin/activate           # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload       # también: fastapi dev app/main.py

```

Abre **<http://127.0.0.1:8000/docs>**: aparece Swagger UI, donde puedes probar cada endpoint desde el navegador con el botón *Try it out*.

Endpoint	Resultado esperado
POST /libros con {"titulo": "María", "autor": "Jorge Isaacs", "anio": 1867}	201 y el libro con su <code>id</code>
POST /libros con {"titulo": "", "autor": "X", "anio": 3000}	422 (validación de Pydantic)

Endpoint	Resultado esperado
GET /libros?titulo=mar	Lista filtrada
POST /libros/1/prestamo dos veces	200 la primera vez, 409 la segunda
GET /libros/99	404
DELETE /libros/1	204

Qué observar en este código

- **Inyección de dependencias:** `Depends(obtener_servicio)` permite reemplazar el servicio en las pruebas por uno con base de datos en memoria.
- **Manejo centralizado de errores:** las excepciones de negocio (`LibroNoEncontrado` , `LibroYaPrestado`) se traducen a 404 y 409 en un solo lugar.
- **Validación declarativa:** `Field(ge=1450, le=2100)` evita escribir `if` para cada dato.
- **CORS:** el *middleware* permite que un frontend en otro puerto consuma la API.

Reto 23.1

Agrega la entidad `Autor` con su propio CRUD y relaciona cada libro con un `autor_id` (clave foránea). Agrega el endpoint `GET /autores/{id}/libros`.

Módulo 24: Frontend con Python

El **frontend** es la parte que ve el usuario. En la web, el lenguaje nativo del navegador es JavaScript (con HTML y CSS), pero hoy existen herramientas que permiten construir interfaces **escribiendo solo Python**.

Herramienta	Tipo	Ideal para
Streamlit	Aplicación web desde un script	Dashboards, prototipos, ciencia de datos
NiceGUI / Reflex / Flet	Aplicaciones web (y de escritorio en Flet) con componentes	Aplicaciones más completas sin escribir JavaScript
PyScript	Python ejecutándose dentro del navegador	Páginas HTML con lógica en Python
Jinja2 (Flask / Django)	Plantillas HTML generadas en el servidor	Sitios web tradicionales
Tkinter	Interfaz de escritorio (incluida en Python)	Herramientas locales sencillas

Proyecto: interfaz web para la API de biblioteca (Streamlit)

Este frontend consume la API del Módulo 23: lista, busca, registra, presta y devuelve libros. Es una arquitectura **cliente-servidor** completa.

 **Archivo:** proyectos/08_frontend_streamlit/app.py

```
"""
Frontend web 100 % Python con Streamlit que consume la API del proyecto 07.
Instalar: pip install streamlit requests
Ejecutar: 1) En otra terminal, levanta la API: uvicorn app.main:app (carpeta
07_backend_fastapi)
          2) streamlit run app.py
"""
import requests
import streamlit as st

API = "http://127.0.0.1:8000"

st.set_page_config(page_title="Biblioteca", page_icon="📖")
st.title("📖 Biblioteca")

# ----- Formulario para crear (POST) -----
with st.form("nuevo_libro", clear_on_submit=True):
    st.subheader("Registrar libro")
    titulo = st.text_input("Título")
    autor = st.text_input("Autor")
    anio = st.number_input("Año", min_value=1450, max_value=2100, value=2000, step=1)
    if st.form_submit_button("Guardar"):
        r = requests.post(f"{API}/libros", json={"titulo": titulo, "autor": autor, "anio":
int(anio)}, timeout=5)
```

```

    if r.status_code == 201:
        st.success(f"Libro creado con id {r.json()['id']}")
    else:
        st.error(f"Error {r.status_code}: revisa los datos")

# ----- Listado con búsqueda (GET) -----
busqueda = st.text_input("Buscar por título")
try:
    libros = requests.get(f"{API}/libros", params={"titulo": busqueda or None},
        timeout=5).json()
except requests.ConnectionError:
    st.warning("La API no está disponible. ¿Ejecutaste uvicorn?")
    st.stop()

st.metric("Libros registrados", len(libros))
for libro in libros:
    col1, col2 = st.columns([4, 1])
    estado = "🔴 Prestado" if libro["prestado"] else "🟢 Disponible"
    col1.write(f"**{libro['titulo']}** · {libro['autor']} ({libro['anio']}) - {estado}")
    if not libro["prestado"] and col2.button("Prestar", key=f"p{libro['id']}"):
        requests.post(f"{API}/libros/{libro['id']}/prestamo", timeout=5)
        st.rerun()
    if libro["prestado"] and col2.button("Devolver", key=f"d{libro['id']}"):
        requests.delete(f"{API}/libros/{libro['id']}/prestamo", timeout=5)
        st.rerun()

```

Cómo ejecutarlo:

```

# Terminal 1: el backend
cd proyectos/07_backend_fastapi && uvicorn app.main:app
# Terminal 2: el frontend
cd proyectos/08_frontend_streamlit
pip install streamlit requests
streamlit run app.py           # se abre en http://localhost:8501

```

Alternativa de escritorio: Tkinter

```

import tkinter as tk
from tkinter import messagebox

def convertir():
    try:
        celsius = float(entrada.get())
        resultado.config(text=f"{celsius * 9 / 5 + 32:.1f} °F")
    except ValueError:
        messagebox.showerror("Error", "Escribe un número válido")

ventana = tk.Tk()
ventana.title("Conversor de temperatura")
tk.Label(ventana, text="Grados Celsius:").pack(padx=20, pady=5)
entrada = tk.Entry(ventana)
entrada.pack(padx=20)
tk.Button(ventana, text="Convertir", command=convertir).pack(pady=10)
resultado = tk.Label(ventana, text="", font=("Arial", 16))
resultado.pack(pady=10)
ventana.mainloop()           # ciclo de eventos: la ventana espera acciones del usuario

```

Frontend clásico: HTML + JavaScript que consume tu API

Para quien quiera ver cómo lo hace la web tradicional, este archivo HTML consume el mismo backend con `fetch`:

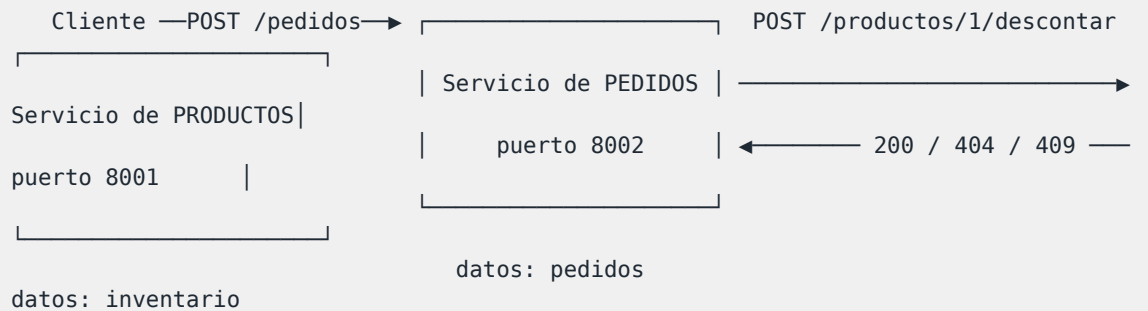
```
<!doctype html>
<html lang="es">
<body>
  <h1>Libros</h1>
  <ul id="lista"></ul>
  <script>
    fetch("http://127.0.0.1:8000/libros")
      .then(respuesta => respuesta.json())
      .then(libros => {
        document.getElementById("lista").innerHTML =
          libros.map(l => `<li>${l.titulo} (${l.anio})</li>`).join("");
      });
  </script>
</body>
</html>
```

Reto 24.1

Agrega al dashboard de Streamlit un gráfico de barras con la cantidad de libros por década de publicación (`st.bar_chart`).

Módulo 25: Microservicios

Un **microservicio** es una aplicación pequeña, independiente y con una sola responsabilidad de negocio, que se comunica con otros servicios por la red (normalmente HTTP/REST o mensajería). Cada uno tiene **su propia base de datos** y se despliega por separado.



Servicio de productos

Archivo: proyectos/09_microservicios/servicio_productos/main.py

```

"""
Microservicio de PRODUCTOS (puerto 8001). Dueño de su propio inventario.
Ejecutar: uvicorn main:app --port 8001      (desde servicio_productos)
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="Servicio de productos")

INVENTARIO: dict[int, dict] = {
    1: {"id": 1, "nombre": "Arduino UNO", "precio": 89_000, "stock": 10},
    2: {"id": 2, "nombre": "Sensor ultrasónico HC-SR04", "precio": 9_500, "stock": 25},
    3: {"id": 3, "nombre": "Servomotor SG90", "precio": 12_000, "stock": 0},
}

class Descuento(BaseModel):
    cantidad: int = Field(gt=0)

@app.get("/productos")
def listar():
    return list(INVENTARIO.values())

@app.get("/productos/{id_producto}")
def obtener(id_producto: int):
    if id_producto not in INVENTARIO:
        raise HTTPException(status_code=404, detail="Producto no existe")
    return INVENTARIO[id_producto]

@app.post("/productos/{id_producto}/descontar")
def descontar(id_producto: int, datos: Descuento):

```

```

producto = obtener(id_producto)
if producto["stock"] < datos.cantidad:
    raise HTTPException(status_code=409, detail="Stock insuficiente")
producto["stock"] -= datos.cantidad
return producto

```

Servicio de pedidos

 **Archivo:** proyectos/09_microservicios/servicio_pedidos/main.py

```

"""
Microservicio de PEDIDOS (puerto 8002). Se comunica con PRODUCTOS por HTTP.
Ejecutar: uvicorn main:app --port 8002      (desde servicio_pedidos)
"""

import os

import httpx
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

PRODUCTOS_URL = os.getenv("PRODUCTOS_URL", "http://127.0.0.1:8001")
app = FastAPI(title="Servicio de pedidos")
PEDIDOS: list[dict] = []

class PedidoEntrada(BaseModel):
    id_producto: int
    cantidad: int = Field(gt=0)

@app.post("/pedidos", status_code=201)
def crear_pedido(pedido: PedidoEntrada):
    try:
        r = httpx.post(f"{PRODUCTOS_URL}/productos/{pedido.id_producto}/descontar",
                      json={"cantidad": pedido.cantidad}, timeout=5)
    except httpx.RequestError:
        # El otro servicio está caído: respondemos 503 en lugar de fallar sin control
        raise HTTPException(status_code=503, detail="Servicio de productos no disponible")
    if r.status_code != 200:
        raise HTTPException(status_code=r.status_code, detail=r.json()["detail"])
    producto = r.json()
    nuevo = {
        "id": len(PEDIDOS) + 1,
        "producto": producto["nombre"],
        "cantidad": pedido.cantidad,
        "total": producto["precio"] * pedido.cantidad,
    }
    PEDIDOS.append(nuevo)
    return nuevo

@app.get("/pedidos")
def listar():
    return PEDIDOS

```

Ejecución

Opción 1: dos terminales.

```
cd proyectos/09_microservicios/servicio_productos && uvicorn main:app --port 8001
cd proyectos/09_microservicios/servicio_pedidos && uvicorn main:app --port 8002
```

Luego, en `http://127.0.0.1:8002/docs`, crea un pedido con `{"id_producto": 1, "cantidad": 2}` y consulta en `http://127.0.0.1:8001/productos` cómo bajó el stock. Prueba también el producto 3 (sin stock → 409) y apaga el servicio de productos para ver el 503.

Opción 2: Docker Compose (si tienes Docker instalado):

 **Archivo:** `proyectos/09_microservicios/docker-compose.yml`

```
# Levanta los dos microservicios con Docker: docker compose up
services:
  productos:
    image: python:3.14-slim
    working_dir: /app
    volumes: ["/servicio_productos:/app"]
    command: sh -c "pip install fastapi uvicorn && uvicorn main:app --host 0.0.0.0 --port 8001"
    ports: ["8001:8001"]

  pedidos:
    image: python:3.14-slim
    working_dir: /app
    volumes: ["/servicio_pedidos:/app"]
    environment:
      PRODUCTOS_URL: http://productos:8001 # nombre del servicio = nombre de host en la red de Docker
    command: sh -c "pip install fastapi uvicorn httpx && uvicorn main:app --host 0.0.0.0 --port 8002"
    ports: ["8002:8002"]
    depends_on: [productos]
```

Conceptos clave

Concepto	Qué es
API Gateway	Punto de entrada único que enruta las peticiones a cada servicio
Descubrimiento de servicios	Cómo un servicio encuentra la dirección de otro (en Docker, por el nombre del servicio)
Tolerancia a fallos	Responder de forma controlada cuando otro servicio falla (el 503 de este ejemplo, <i>reintentos</i> , <i>circuit breaker</i>)
Comunicación asíncrona	Enviar mensajes a una cola (RabbitMQ, Kafka) en lugar de esperar la respuesta
Contenedores	Empaquetar cada servicio con sus dependencias (Docker)

Reto 25.1

Crea un tercer microservicio de **notificaciones** (puerto 8003) con un endpoint `POST /notificaciones`. Haz que el servicio de pedidos lo llame después de crear un pedido, y que si notificaciones está caído el pedido se cree de todas formas.

Módulo 26: Simulación 3D

Simular es **representar un sistema real con un modelo** para estudiar su comportamiento sin construirlo. En Ingeniería (y especialmente en Robótica) se simulan trayectorias, brazos robóticos, sensores y procesos industriales antes de invertir en hardware.

Herramienta	Para qué sirve
NumPy	Cálculo numérico con vectores y matrices
Matplotlib (mplot3d)	Gráficas 2D y 3D de resultados
VPython	Animaciones 3D de física muy sencillas
Ursina	Motor de juegos 3D en Python: mundos interactivos
PyBullet	Física de cuerpos rígidos y robots (usado en investigación)
ROS 2 + Gazebo	Estándar de la industria para robótica (nivel avanzado)

Simulación 1: trayectoria de un proyectil con viento (Matplotlib)

Modelo físico: en cada pequeño intervalo de tiempo `dt`, la velocidad cambia por la aceleración (gravedad en z , viento en y) y la posición cambia por la velocidad. Esto se llama **método de Euler**, la forma más simple de resolver ecuaciones de movimiento por computador.

 **Archivo:** proyectos/10_simulacion_3d/proyectil_3d.py

```
"""
Simulación 3D de un proyectil con viento lateral (física básica, método de Euler).
Útil para Robótica e Ingeniería: modelar, simular y visualizar.
Instalar: pip install matplotlib
Ejecutar: python proyectil_3d.py
"""

import math

import matplotlib.pyplot as plt

G = 9.81 # gravedad (m/s²)

def simular(velocidad: float, angulo_elev: float, angulo_dir: float,
            viento_y: float = 0.0, dt: float = 0.01) -> tuple[list, list, list]:
    """Devuelve las listas x, y, z de la trayectoria hasta tocar el suelo."""
    elev, direc = math.radians(angulo_elev), math.radians(angulo_dir)
    vx = velocidad * math.cos(elev) * math.cos(direc)
    vy = velocidad * math.cos(elev) * math.sin(direc)
    vz = velocidad * math.sin(elev)
    x = y = z = 0.0
    xs, ys, zs = [x], [y], [z]
    while z >= 0:
        vy += viento_y * dt # el viento empuja lateralmente
        vz -= G * dt # la gravedad frena la subida
```

```

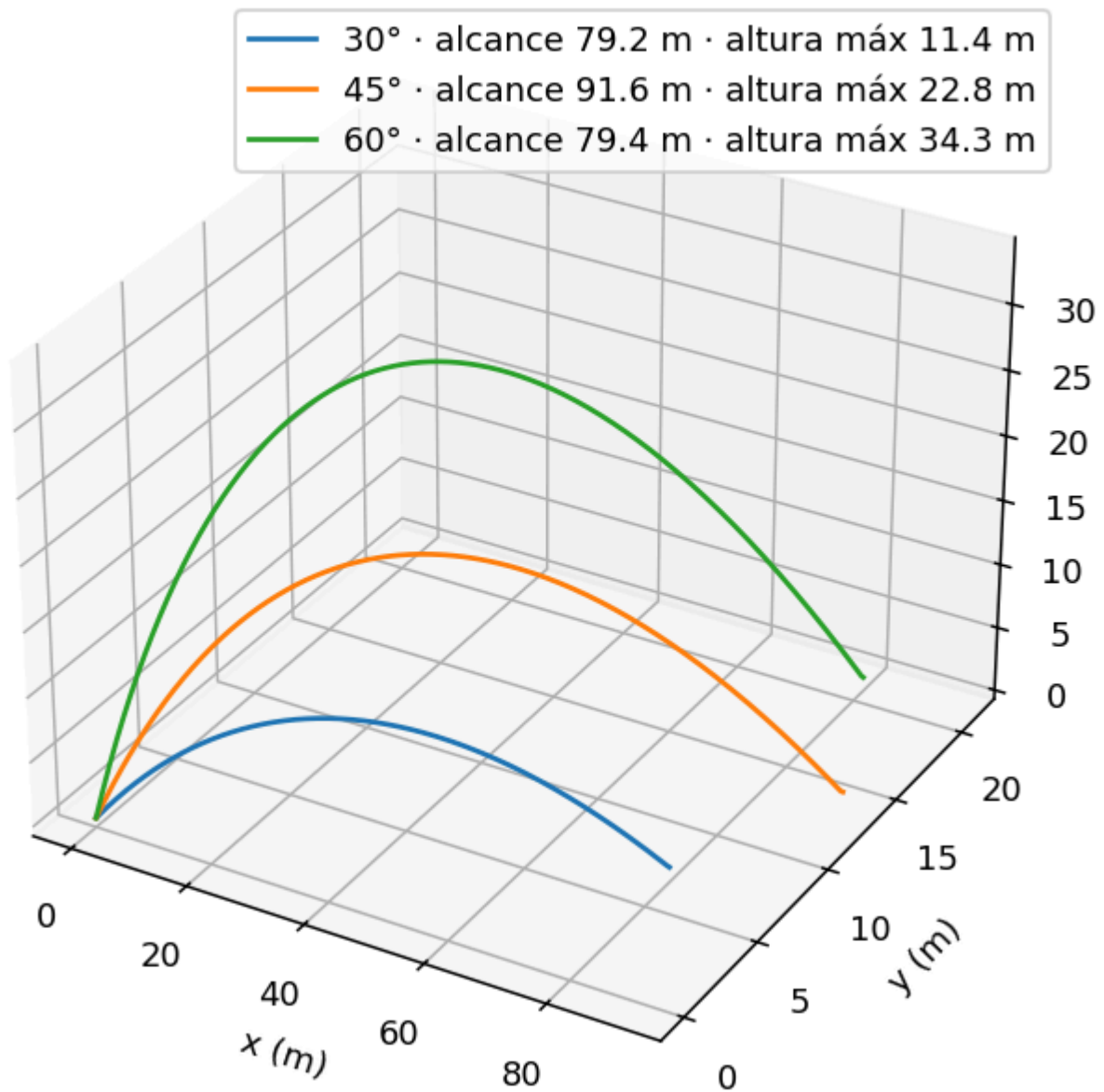
        x, y, z = x + vx * dt, y + vy * dt, z + vz * dt
        xs.append(x); ys.append(y); zs.append(max(z, 0))
    return xs, ys, zs

def main() -> None:
    figura = plt.figure(figsize=(9, 6))
    ejes = figura.add_subplot(projection="3d")
    for angulo in (30, 45, 60):
        xs, ys, zs = simular(velocidad=30, angulo_elev=angulo, angulo_dir=0, viento_y=1.5)
        ejes.plot(xs, ys, zs, label=f"{angulo}° · alcance {xs[-1]:.1f} m · altura máx
{max(zs):.1f} m")
    ejes.set_xlabel("x (m)"); ejes.set_ylabel("y (m)"); ejes.set_zlabel("altura (m)")
    ejes.set_title("Trayectorias con viento lateral de 1.5 m/s²")
    ejes.legend()
    plt.show()

if __name__ == "__main__":
    main()

```

Trayectorias con viento lateral de 1.5 m/s²



Validación del modelo: sin viento, a 30 m/s y 45°, la teoría dice que el alcance es $v^2/g \approx 91.7 \text{ m}$ y la altura máxima $v^2 \cdot \sin^2(45^\circ)/(2g) \approx 22.9 \text{ m}$. La simulación da **91.6 m** y **22.8 m**: el pequeño error viene del tamaño del paso `dt`. Reto: prueba con `dt = 0.001` y observa cómo se reduce el error.

Simulación 2: mundo 3D interactivo (Ursina)

Un cubo que se mueve con el teclado, salta con gravedad y cambia de color al chocar con obstáculos: los mismos conceptos de un videojuego o de un robot móvil que evita obstáculos.

Archivo: `proyectos/10_simulacion_3d/mundo_ursina.py`

```
"""
```

```
Mundo 3D interactivo con Ursina (motor de juegos 3D en Python).
Instalar: pip install ursina
```

Ejecutar: `python mundo_ursina.py`

Controles: `W A S D` para mover el cubo, `espacio` para saltar, `clic derecho + ratón` para girar la cámara.

"""

```
from ursina import EditorCamera, Entity, Ursina, color, held_keys, time
```

```
app = Ursina()
```

```
suelo = Entity(model="plane", scale=20, color=color.light_gray, texture="white_cube",
               texture_scale=(20, 20))
```

```
jugador = Entity(model="cube", color=color.orange, position=(0, 0.5, 0))
```

```
obstaculos = [Entity(model="cube", color=color.azure, position=(x, 0.5, 4), scale=(1, 1, 1))
```

```
               for x in range(-6, 7, 3)]
```

```
velocidad_y = 0.0
```

```
def update():                                     # Ursina llama esta función en cada cuadro
```

```
    global velocidad_y
```

```
    velocidad = 5 * time.dt
```

```
    jugador.x += (held_keys["d"] - held_keys["a"]) * velocidad
```

```
    jugador.z += (held_keys["w"] - held_keys["s"]) * velocidad
```

```
    velocidad_y -= 9.81 * time.dt                # gravedad
```

```
    jugador.y = max(0.5, jugador.y + velocidad_y * time.dt)
```

```
    if jugador.y == 0.5:
```

```
        velocidad_y = 0
```

```
    jugador.color = color.red if any(jugador.intersects(o).hit for o in obstaculos) else
    color.orange
```

```
def input(tecla):                                # eventos de teclado
```

```
    global velocidad_y
```

```
    if tecla == "space" and jugador.y == 0.5:
```

```
        velocidad_y = 6
```

```
EditorCamera()
```

```
app.run()
```

Retos de ampliación

1. Agrega resistencia del aire al proyectil (una aceleración proporcional a la velocidad y en sentido contrario).
2. Encuentra por simulación el ángulo que maximiza el alcance con viento (prueba de 1° a 89°).
3. En Ursina, haz que los obstáculos se muevan de un lado a otro y cuenta cuántas veces choca el jugador.

Módulo 27: Pruebas unitarias con pytest

Una **prueba unitaria** verifica automáticamente que una pieza pequeña del código (una función, un método) hace lo que debe. Las pruebas permiten cambiar el código con confianza: si algo se rompe, una prueba falla y lo avisa.

pytest es la herramienta más usada en Python (`pip install pytest`). Reglas: los archivos se llaman `test_*.py`, las funciones `test_*`, y se verifica con `assert`.

Código a probar

 **Archivo:** `proyectos/11_pruebas_pytest/calculadora.py`

```
"""Funciones a probar con pytest."""

def dividir(a: float, b: float) -> float:
    if b == 0:
        raise ZeroDivisionError("No se puede dividir entre cero")
    return a / b

def es_primo(n: int) -> bool:
    if n < 2:
        return False
    divisor = 2
    while divisor * divisor <= n:
        if n % divisor == 0:
            return False
        divisor += 1
    return True

def es_palindromo(texto: str) -> bool:
    limpio = "".join(c.lower() for c in texto if c.isalnum())
    return limpio == limpio[::-1]

def calcular_iva(valor: float, tasa: float = 0.19) -> float:
    return round(valor * tasa, 2)
```

Pruebas

 **Archivo:** `proyectos/11_pruebas_pytest/test_calculadora.py`

```
"""Ejecutar: pytest -v (pip install pytest)"""
import pytest

from calculadora import calcular_iva, dividir, es_palindromo, es_primo
```

```
def test_dividir():
    assert dividir(10, 4) == 2.5

def test_dividir_entre_cero_lanza_error():
    with pytest.raises(ZeroDivisionError):
        dividir(5, 0)

@pytest.mark.parametrize("numero, esperado", [(0, False), (1, False), (2, True), (9, False), (97, True)])
def test_es_primo(numero, esperado):
    assert es_primo(numero) == esperado

def test_palindromo_ignora_espacios_y_mayusculas():
    assert es_palindromo("Anita lava la tina")
    assert not es_palindromo("Python")

def test_iva_por_defecto():
    assert calcular_iva(100_000) == 19_000
```

Ejecución

```
$ pytest -v
test_calculadora.py::test_dividir PASSED
test_calculadora.py::test_dividir_entre_cero_lanza_error PASSED
test_calculadora.py::test_es_primo[0-False] PASSED
...
===== 9 passed in 0.01s =====
```

Buenas prácticas: patrón AAA

1. **Arrange** (preparar): crear los datos y objetos.
2. **Act** (actuar): ejecutar la función que se prueba.
3. **Assert** (verificar): comprobar el resultado.

Además: una prueba verifica **una sola idea**; su nombre dice qué se espera (`test_no_se_puede_prestar_dos_veces`); prueba los **casos límite** (cero, vacío, negativo, el valor exacto del límite); y las pruebas no dependen unas de otras.

Pruebas de APIs: el archivo `proyectos/07_backend_fastapi/tests/test_api.py` usa el `TestClient` de FastAPI y una base de datos en memoria para probar los códigos 201, 204, 404, 409 y 422 sin levantar el servidor.

Reto 27.1

Escribe pruebas para las funciones `cuota_credito`, `interes_compuesto` y `formato_pesos` del proyecto de finanzas. Incluye el caso de tasa 0 %.

Módulo 28: Banco de preguntas y respuestas

Preguntas frecuentes en parciales y entrevistas técnicas de Python para perfiles junior, agrupadas por tema. **Úsalas como autoevaluación:** tapa la respuesta e intenta responder primero. Todas las respuestas fueron verificadas ejecutando el código.

Sintaxis y fundamentos

- 1. ¿Qué define un bloque de código en Python?** La indentación (por convención, 4 espacios) después de una línea que termina en `:`.
- 2. ¿Qué imprime `print(type(3 / 3))`?** `<class 'float'>`. El operador `/` siempre devuelve `float`.
- 3. ¿Es válido `x++` en Python?** No: escrito solo produce `SyntaxError`. Se usa `x += 1`. (Ojo: dentro de una expresión como `x++ + y` Python lo lee como `x + (+(+y))` y no falla, pero tampoco incrementa `x`.)
- 4. ¿Qué diferencia hay entre `print` y `return`?** `print` muestra un valor en pantalla; `return` entrega el valor a quien llamó la función para que lo use. Una función puede imprimir y aun así devolver `None`.
- 5. ¿Qué es `None` y cómo se compara?** Representa la ausencia de valor. Se compara con `is`: `if x is None:`.
- 6. ¿Qué diferencia hay entre una lista y una tupla?** La lista es mutable (`[1, 2]`); la tupla es inmutable (`(1, 2)`) y puede usarse como clave de diccionario.
- 7. ¿Python pasa los argumentos por valor o por referencia?** Por "referencia a objeto": la función recibe una referencia al mismo objeto. Si el objeto es mutable (lista, diccionario) y la función lo modifica, el cambio se ve afuera; si se reasigna la variable dentro de la función, no.

Operadores aritméticos

- 8. ¿Qué imprime `print(17 // 5, 17 % 5, 2 ** 3 ** 2)`?** `3 2 512`. La potencia se asocia de derecha a izquierda: `2 ** 9`.
- 9. ¿Qué devuelve `-7 // 2`?** `-4`: la división entera redondea hacia abajo (hacia menos infinito).
- 10. ¿Qué devuelve `-7 % 3`?** `2`: en Python el residuo toma el signo del divisor (en Java daría `-1`).

11. ¿Qué imprime `print(round(2.5), round(3.5))` ? `2 4` . Python redondea al par más cercano cuando el valor está exactamente en la mitad (*redondeo bancario*).

12. ¿Por qué `0.1 + 0.2 == 0.3` es `False` ? Porque los `float` se representan en binario y `0.1` no tiene representación exacta. Para comparar se usa `math.isclose(0.1 + 0.2, 0.3)` y para dinero `Decimal` .

Operadores de comparación

13. ¿Qué imprime `print(1 < 3 < 2)` ? `False` . Equivale a `1 < 3 and 3 < 2` .

14. ¿Diferencia entre `==` e `is` ? `==` compara el **contenido**; `is` compara si son **el mismo objeto**. `[1] == [1]` es `True` , `[1] is [1]` es `False` .

15. ¿Qué imprime `print("10" > "9")` ? `False` : los textos se comparan carácter por carácter y `"1"` es menor que `"9"` .

16. ¿Qué imprime `print(True == 1, True + True)` ? `True 2` : `bool` es una subclase de `int` (`True` vale 1).

Operadores lógicos

17. ¿Qué imprime `print(0 or [] or "hola")` ? `hola` : `or` devuelve el primer valor verdadero.

18. ¿Qué imprime `print(5 and 0 and 3)` ? `0` : `and` devuelve el primer valor falso.

19. ¿Qué es el cortocircuito? Que `and` no evalúa el segundo operando si el primero es falso, y `or` no lo evalúa si el primero es verdadero. Permite escribir `if lista and lista[0] > 0:` sin error cuando la lista está vacía.

20. ¿Qué imprime `print(not 0, not "texto")` ? `True False` .

Textos

21. ¿Qué imprime `"Python"[1:4]` ? `yth` : desde el índice 1 hasta el 3 (el 4 no se incluye).

22. ¿Cómo se invierte un texto? Con `texto[::-1]` .

23. ¿Qué imprime `print("a,b,,c".split(","))` ? `['a', 'b', '', 'c']` .

24. ¿Qué es una f-string? Un texto con prefijo `f` donde las expresiones entre llaves se evalúan: `f"Total: {precio * 2:,.2f}"` .

Listas, tuplas y conjuntos

25. ¿Qué imprime este código?

```
a = [1, 2, 3]
b = a
b.append(4)
print(len(a))
```

4: `b` no es una copia, es otro nombre para la misma lista.

26. ¿Qué diferencia hay entre `append` y `extend`? `append([4, 5])` agrega la lista como **un** elemento; `extend([4, 5])` agrega **cada** elemento.

27. ¿Qué imprime `print([x * 2 for x in range(5) if x % 2 == 1])`? `[2, 6]`.

28. ¿Cómo se eliminan los duplicados de una lista? `list(set(lista))` (pierde el orden) o `list(dict.fromkeys(lista))` (conserva el orden).

Diccionarios

29. ¿Qué imprime este código?

```
d = {"a": 1, "b": 2}
d["c"] = d.get("a", 0) + d.get("z", 10)
print(d)
```

`{'a': 1, 'b': 2, 'c': 11}`.

30. ¿Qué imprime `print(len({"x": 1, "y": 2, "x": 3}))`? `2`: la clave `"x"` repetida se sobrescribe con el último valor.

31. ¿Cómo se recorren clave y valor al tiempo? `for clave, valor in d.items():`.

32. ¿Qué pasa con `{[1, 2]: "lista"}`? `TypeError: unhashable type: 'list'`. Las claves deben ser inmutables.

33. ¿Cómo se unen dos diccionarios? `**d1, **d2` o, desde Python 3.9, `d1 | d2`. Si hay claves repetidas, gana el de la derecha.

34. ¿Qué devuelve `d.pop("clave", None)` si la clave no existe? `None`, sin error.

Funciones

35. ¿Qué imprime este código?

```
def f(x, lista=[]):
    lista.append(x)
    return lista

f(1)
print(f(2))
```

`[1, 2]`: el valor por defecto mutable se crea una sola vez y se comparte entre llamadas. Se corrige con `lista=None`.

36. ¿Qué son `*args` y `**kwargs`? `*args` recibe argumentos posicionales variables como **tupla**; `**kwargs` recibe argumentos con nombre como **diccionario**.

37. ¿Qué es una `lambda`? Una función anónima de una sola expresión: `lambda x: x * 2`.

38. ¿Qué es la recursividad y qué necesita? Una función que se llama a sí misma; necesita un caso base que detenga las llamadas.

Programación orientada a objetos

39. ¿Qué es una clase, un objeto y una instancia? La clase es el molde; el objeto es un elemento concreto creado con ese molde; "instancia" es sinónimo de objeto, y se usa para enfatizar su clase ("`cuenta_ana` es una instancia de `CuentaBancaria`").

40. ¿Qué es `self`? La referencia al objeto actual dentro de sus métodos.

41. ¿Cuáles son los cuatro pilares de la POO? Encapsulamiento, herencia, polimorfismo y abstracción.

42. ¿Qué diferencia hay entre `__str__` y `__repr__`? `__str__` es la representación amigable para el usuario (`print`); `__repr__` es la técnica para el programador, idealmente con la forma de recrear el objeto.

43. ¿Qué hace `@property`? Permite acceder a un método como si fuera un atributo, útil para validar al asignar valores.

Excepciones y archivos

44. ¿Qué bloque de un `try` se ejecuta siempre? `finally`.

45. ¿Qué excepción produce `int("12a")`? `ValueError`.

46. ¿Por qué no usar `except:` vacío? Porque atrapa todos los errores, incluso los de programación, y los oculta. Se deben capturar excepciones específicas.

Web, APIs y herramientas

47. ¿Qué es una API REST? Una interfaz que expone recursos mediante URLs y verbos HTTP (`GET`, `POST`, `PUT`, `DELETE`), normalmente con datos en JSON.

48. ¿Qué diferencia hay entre FastAPI, Flask y Django? FastAPI: APIs modernas con validación automática y documentación Swagger. Flask: microframework minimalista. Django: framework completo con ORM, panel de administración y autenticación.

49. ¿Qué es un entorno virtual? Una instalación aislada de Python para un proyecto, con sus propias librerías.

50. ¿Qué es la inyección SQL y cómo se evita? Un ataque que introduce código SQL a través de los datos del usuario. Se evita con consultas parametrizadas (`cursor.execute("... WHERE id = ?", (id,))`) o con un ORM.

Módulo 29: Python Developer Test

Evaluación práctica con el mismo formato de las pruebas técnicas que aplican las empresas para cargos de desarrollador junior: **lectura de código, SQL, identificación de tecnologías y conceptos**. Resuélvela **sin ejecutar el código** y luego verifica tus respuestas ejecutándolo.

Datos del estudiante: Nombre: ____ **Fecha:** ____ Puntaje: ____ / 100

Parte A. Selección múltiple: ¿qué hace el código? (40 puntos)

A1. ¿Qué imprime el siguiente código?

```
x = 1
while x <= 10:
    x += 1
    print(x)
```

a) Los valores del 2 al 10 b) Los valores del 1 al 10 c) Los valores del 2 al 11 d) Los valores del 1 al 9

A2. ¿Qué se imprime por pantalla?

```
x = 10
y = 3
print(x % y)
```

a) 1 b) 3 c) 2 d) 0

A3. ¿Qué hace la siguiente función?

```
def misterio(x: float, y: int) -> float:
    return (x * y) / 100
```

a) Calcula la décima parte de x sobre y b) Calcula el y por ciento del valor de x c) Calcula los 2 decimales de x d) Ninguna es correcta

A4. ¿Cuál es el valor de x y de y al final del programa?

```
for x in range(10):
    print(x)

y = 0
while y < 10:
    print(y)
    y += 1
```

a) x=9, y=9 b) x=10, y=10 c) x=10, y=9 d) x=9, y=10

A5. ¿Qué ocurre con la siguiente línea en Python?

```
z = x++ + y
```

a) Produce `SyntaxError` : Python no tiene el operador `++` b) Suma `x + y` en `z` y luego incrementa `x` c) Incrementa `x` y lo suma a `y` d) Asigna a `z` el valor de `x + y` sin incrementar

A6. ¿Qué valor se muestra en pantalla?

```
x = 5
y = 5
y //= x + 1
print(y)
```

a) 6 b) 5 c) 1 d) 0

A7. ¿Qué se imprime por pantalla?

```
total = 10
contador = 4
contador -= 1
total -= contador
print(total)
```

a) 10 b) 7 c) 6 d) 4

A8. ¿Los operadores `and` y `&` hacen lo mismo?

a) Sí, son sinónimos b) No: `and` es lógico y hace cortocircuito; `&` opera bit a bit (o intersección de conjuntos) y evalúa ambos lados c) Sí, pero `&` hace cortocircuito d) Ninguna es correcta

A9. ¿Qué hace la siguiente función?

```
def misterio(unos: int, dos: int) -> int:
    calculo = 1
    for _ in range(dos):
        calculo *= unos
    return calculo
```

a) Multiplica `unos` por `dos` b) Calcula el factorial de `unos` c) Calcula `unos` elevado a la `dos` d) Ninguna es correcta

A10. ¿Qué resultado se muestra en pantalla?

```
x = 0
y = 0
while x < 5:
    y += x
    x += 1
print(y)
```

a) 10 b) 5 c) 0 d) El programa produce un error

A11. ¿Qué se imprime?

```

inventario = {"lapiz": 10, "borrador": 4}
inventario["lapiz"] -= 3
inventario["regla"] = inventario.get("regla", 0) + 2
print(len(inventario), inventario["lapiz"])

```

a) 2 7 b) 3 7 c) 3 10 d) `KeyError`

A12. ¿Qué imprime el código?

```

a = [1, 2, 3]
b = a
b.append(4)
c = a.copy()
c.append(5)
print(a)

```

a) [1, 2, 3] b) [1, 2, 3, 4] c) [1, 2, 3, 4, 5] d) [1, 2, 3, 5]

A13. ¿Qué imprime `print(bool("False"), bool(0), bool([]))` ?

a) True False False b) False False False c) True True False d) False False True

A14. ¿Qué imprime el código?

```

edad = 20
tiene_permiso = False
print(edad >= 18 and not tiene_permiso or edad > 30)

```

a) False b) True c) None d) Error de sintaxis

A15. ¿Qué imprime `print(7 / 2, 7 // 2, -7 // 2)` ?

a) 3.5 3 -3 b) 3 3 -3 c) 3.5 3 -4 d) 3.5 4 -4

A16. ¿Qué imprime el código?

```

def saludo(nombre="Invitado"):
    print(f"Hola {nombre}")

resultado = saludo()
print(resultado)

```

a) Hola Invitado / Hola Invitado b) None c) Error: falta el argumento d) Hola Invitado / None

Parte B. Bases de datos SQL (15 puntos)

B1. Considere la tabla `ESTUDIANTES(codigo, nombre, apellido, correo, telefono, fecha_nacimiento, programa, promedio)`. ¿Cuál consulta muestra todos los estudiantes del programa `'SIS01'` ?

a) `SELECT codigo, nombre, apellido FROM estudiantes WHERE programa = 'SIS01';` b) `SELECT codigo, nombre, apellido FROM estudiantes WHERE programa IS 'SIS01';` c) `SELECT codigo, nombre, apellido WHERE programa = 'SIS01' FROM estudiantes;` d) `SELECT codigo, nombre, apellido FROM estudiantes;`

B2. ¿Cómo se seleccionan todos los registros de la tabla `Personas` cuyo `nombre` comience por "a"?

a) `SELECT * FROM Personas WHERE nombre = 'a'` b) `SELECT * FROM Personas WHERE nombre LIKE '%a'` c) `SELECT * FROM Personas WHERE nombre LIKE 'a%'` d) `SELECT * FROM Personas WHERE nombre = '%a%'`

B3. ¿Cómo se inserta un nuevo registro en la tabla `Personas` ?

a) `INSERT ('Juana', 'Gómez') INTO Personas` b) `INSERT INTO Personas VALUES ('Juana', 'Gómez')` c) `INSERT VALUES ('Juana', 'Gómez') INTO Personas` d) `INSERT INTO Personas ('Juana', 'Gómez')`

B4. ¿Cuál sentencia obtiene el número total de órdenes de la tabla `Ventas` ?

NumeroOrden	Fecha	IdCliente
1	12/12/2025	13
2	13/12/2025	17

a) `SELECT COUNT(*) FROM Ventas` b) `SELECT SUM(NumeroOrden) FROM Ventas` c) `SELECT AVG(NumeroOrden) FROM Ventas` d) `SELECT MAX(NumeroOrden) FROM Ventas`

B5. En Python, ¿cuál es la forma **segura** de ejecutar la consulta con un dato escrito por el usuario?

a) `cursor.execute(f"SELECT * FROM usuarios WHERE correo = '{correo}')` b) `cursor.execute("SELECT * FROM usuarios WHERE correo = ?", (correo,))` c) `cursor.execute("SELECT * FROM usuarios WHERE correo = " + correo)` d) Las tres son igual de seguras

Parte C. Preguntas abiertas: describa qué hace el código e identifique las tecnologías, librerías o frameworks utilizados (25 puntos)

C1.

```
def consultar_historial(numero_identificacion: str, id_consulta: int) -> str | None:
    sql = """SELECT d.respuesta
              FROM historial h
              INNER JOIN detalle_consulta d ON h.id_historial = d.id_historial
              WHERE h.numero_identificacion = %s AND d.id_detalle = %s"""

    try:
        with psycopgp.connect(DB_URL) as conexion:
            with conexion.cursor() as cursor:
```

```

        cursor.execute(sql, (numero_identificacion, id_consulta))
        fila = cursor.fetchone()
        return fila[0] if fila else None
    except psycopg.Error as error:
        logger.error("No se pudo ejecutar la consulta: %s", error)
        return None

```

Respuesta: _____

C2.

```

from sqlalchemy import String
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column

class Base(DeclarativeBase):
    pass

class CentralConsulta(Base):
    __tablename__ = "central_consulta"
    id: Mapped[int] = mapped_column(primary_key=True)
    documento: Mapped[str] = mapped_column(String(20))

def insertar_consulta(sesion: Session, consulta: CentralConsulta) -> None:
    sesion.add(consulta)
    sesion.commit()

```

Respuesta: _____

C3.

```

{% extends "base.html" %}
{% block encabezado %}
    {% include "parciales/encabezado_credito.html" with context %}
{% endblock %}
{% block contenido %}
    <h2>Centrales de riesgo de {{ credito.cliente.nombre }}</h2>
    {% for reporte in credito.reportes %}
        <p>{{ reporte.entidad }}: {{ reporte.estado }}</p>
    {% endfor %}
{% endblock %}

```

Respuesta: _____

C4.

```

from django.views.generic import ListView
from .models import InformacionHome

class HomeView(ListView):
    model = InformacionHome
    template_name = "home.html"
    context_object_name = "lista_informacion"

    def get_queryset(self):
        return InformacionHome.objects.filter(activo=True).order_by("-fecha")

```

Respuesta: _____

C5.

```

from datetime import datetime
from fastapi import APIRouter, Depends, status
from pydantic import BaseModel

router = APIRouter(prefix="/centrales", tags=["Centrales"])

class Peticion(BaseModel):
    id_credito: int

class Respuesta(BaseModel):
    status_code: int
    fecha_transaccion: datetime
    mensaje: str

@router.post("/obtener-info", response_model=Respuesta, status_code=status.HTTP_200_OK)
def obtener_info_centrales(peticion: Peticion, generador: GeneradorPDF =
Depends(obtener_generador)):
    pdf_base64 = generador.obtener_pdf(peticion.id_credito)
    return Respuesta(status_code=200, fecha_transaccion=datetime.now(), mensaje=pdf_base64)

```

Respuesta: _____

Parte D. Preguntas conceptuales (20 puntos)

- D1.** Defina qué es una clase, una instancia y un objeto. ¿Qué elementos definen a un objeto?
- D2.** ¿Qué es un patrón de diseño? Mencione uno y explique para qué sirve.
- D3.** Explique mediante un diagrama el patrón MVC.
- D4.** ¿Qué es un hilo (*thread*)? ¿Para qué se usa y con qué instrucción se inicia en Python?
- D5.** ¿Qué características tiene y cuál es la diferencia entre una base de datos relacional y una no relacional?
- D6.** ¿Qué herramientas conoce que permitan el manejo centralizado de librerías (dependencias) y la construcción de proyectos en Python?
- D7.** ¿Qué diferencia hay entre una lista, una tupla, un conjunto y un diccionario? Dé un ejemplo de uso real de cada uno.
- D8.** ¿Qué es una API REST y qué significan los códigos 200, 201, 404 y 500?

Módulo 30: Soluciones de los retos y proyecto integrador

Compara tu solución con estas **después** de intentarlo. No hay una única respuesta correcta: si tu programa funciona, es legible y maneja los casos límite, está bien. Todos los archivos están en `retos/soluciones/`.

Reto 3.1: conversor de temperatura

 **Archivo:** `retos/soluciones/reto_03_1_temperatura.py`

```
"""Reto 3.1: conversor de temperatura."""
celsius = float(input("Temperatura en °C: "))
fahrenheit = celsius * 9 / 5 + 32
kelvin = celsius + 273.15
print(f"{celsius} °C = {fahrenheit:.1f} °F = {kelvin:.2f} K")
```

Reto 4.1: segundos a HH:MM:SS

 **Archivo:** `retos/soluciones/reto_04_1_segundos.py`

```
"""Reto 4.1: convertir segundos a horas, minutos y segundos con // y %."""
total = int(input("Segundos: ")) # ejemplo: 3725
horas = total // 3600
minutos = (total % 3600) // 60
segundos = total % 60
print(f"{horas:02d}:{minutos:02d}:{segundos:02d}") # 01:02:05
```

Reto 5.1: análisis de un nombre

 **Archivo:** `retos/soluciones/reto_05_1_nombre.py`

```
"""Reto 5.1: análisis de un nombre completo."""
nombre = input("Nombre completo: ").strip() # ejemplo: Ana María Pérez
partes = nombre.split()
print("Letras (sin espacios):", len(nombre.replace(" ", "")))
print("En mayúsculas:", nombre.upper())
print("Iniciales:", "".join(p[0].upper() for p in partes))
print("Invertido:", nombre[::-1])
print("Vocales:", sum(1 for letra in nombre.lower() if letra in "aeiouáéíóú"))
```

Retos 6.1, 6.2 y 6.3: mayor de tres, año bisiesto e IMC

 **Archivo:** `retos/soluciones/reto_06_retos_condicionales.py`

```

"""Retos 6.1, 6.2 y 6.3: mayor de tres, año bisiesto e IMC."""

def mayor_de_tres(a: float, b: float, c: float) -> float:
    if a >= b and a >= c:
        return a
    elif b >= c:
        return b
    return c

def es_bisiesto(anio: int) -> bool:
    # Divisible entre 4, excepto los que son divisibles entre 100 pero no entre 400
    return anio % 4 == 0 and (anio % 100 != 0 or anio % 400 == 0)

def clasificar_imc(peso_kg: float, estatura_m: float) -> tuple[float, str]:
    imc = peso_kg / estatura_m ** 2
    if imc < 18.5:
        categoria = "Bajo peso"
    elif imc < 25:
        categoria = "Normal"
    elif imc < 30:
        categoria = "Sobrepeso"
    else:
        categoria = "Obesidad"
    return round(imc, 1), categoria

if __name__ == "__main__":
    print(mayor_de_tres(4, 9, 2)) # 9
    print([a for a in (1900, 2000, 2024, 2026) if es_bisiesto(a)]) # [2000, 2024]
    print(clasificar_imc(70, 1.75)) # (22.9, 'Normal')

```

Retos 7.1, 7.2 y 7.3: tabla, promedio hasta cero y FizzBuzz

 **Archivo:** retos/soluciones/reto_07_retos_ciclos.py

```

"""Retos 7.1, 7.2 y 7.3: tabla de multiplicar, promedio hasta 0 y FizzBuzz."""

def tabla(n: int) -> None:
    for i in range(1, 11):
        print(f"{n} x {i:>2} = {n * i:>3}")

def promedio_hasta_cero() -> float:
    suma, cantidad = 0.0, 0
    while True:
        numero = float(input("Número (0 para terminar): "))
        if numero == 0:
            break
        suma += numero
        cantidad += 1
    return suma / cantidad if cantidad else 0.0

def fizzbuzz(n: int) -> list[str]:

```

```

resultado = []
for i in range(1, n + 1):
    if i % 15 == 0:
        resultado.append("FizzBuzz")
    elif i % 3 == 0:
        resultado.append("Fizz")
    elif i % 5 == 0:
        resultado.append("Buzz")
    else:
        resultado.append(str(i))
return resultado

if __name__ == "__main__":
    tabla(7)
    print(", ".join(fizzbuzz(15)))

```

Retos 8.1 y 8.2: temperaturas y duplicados

 **Archivo:** retos/soluciones/reto_08_retos_listas.py

```

"""Retos 8.1 y 8.2: temperaturas de la semana y quitar duplicados conservando el orden."""
temperaturas = [24.5, 26.1, 23.8, 27.3, 25.0, 22.9, 26.8]
dias = ["lun", "mar", "mié", "jue", "vie", "sáb", "dom"]

promedio = sum(temperaturas) / len(temperaturas)
print(f"Máxima: {max(temperaturas)}  Mínima: {min(temperaturas)}  Promedio: {promedio:.2f}")
print("Días sobre el promedio:", [d for d, t in zip(dias, temperaturas) if t > promedio])

def sin_duplicados(elementos: list) -> list:
    vistos = set()
    resultado = []
    for e in elementos:
        if e not in vistos:
            vistos.add(e)
            resultado.append(e)
    return resultado

print(sin_duplicados([3, 1, 3, 2, 1, 5]))    # [3, 1, 2, 5]
# Alternativa en una línea (los dict conservan el orden de inserción):
list(dict.fromkeys(lista))

```

Retos 9.1, 9.2 y 9.3: diccionarios

 **Archivo:** retos/soluciones/reto_09_retos_diccionarios.py

```

"""Retos 9.1, 9.2 y 9.3: contar palabras, agenda de contactos e invertir un diccionario."""
from collections import Counter

texto = "el que lee mucho y anda mucho ve mucho y sabe mucho"

# 9.1 A mano con un diccionario
conteo: dict[str, int] = {}

```

```

for palabra in texto.split():
    conteo[palabra] = conteo.get(palabra, 0) + 1
print(conteo["mucho"]) # 4
# Con la librería estándar
print(Counter(texto.split()).most_common(2)) # [('mucho', 4), ('y', 2)]

# 9.2 Agenda: diccionario de diccionarios
agenda: dict[str, dict] = {}

def agregar(nombre: str, telefono: str, correo: str) -> None:
    agenda[nombre.lower()] = {"telefono": telefono, "correo": correo}

def buscar(nombre: str) -> dict | None:
    return agenda.get(nombre.lower())

def eliminar(nombre: str) -> bool:
    return agenda.pop(nombre.lower(), None) is not None

agregar("Laura", "3001234567", "laura@correo.com")
agregar("Pedro", "3109876543", "pedro@correo.com")
print(buscar("LAURA")) # {'telefono': '3001234567', 'correo': 'laura@correo.com'}
print(eliminar("pedro"), list(agenda)) # True ['laura']

# 9.3 Invertir claves y valores (con comprensión de diccionario)
capitales = {"Antioquia": "Medellín", "Valle": "Cali", "Atlántico": "Barranquilla"}
print({ciudad: depto for depto, ciudad in capitales.items()})["Cali"] # Valle

```

Retos 10.1, 10.2 y 10.3: primos, recursividad y contraseña

 **Archivo:** retos/soluciones/reto_10_retos_funciones.py

```

"""Retos 10.1, 10.2 y 10.3: primos, suma de dígitos recursiva y validador de contraseña."""
import re

def es_primo(n: int) -> bool:
    if n < 2:
        return False
    for divisor in range(2, int(n ** 0.5) + 1):
        if n % divisor == 0:
            return False
    return True

def primos_hasta(n: int) -> list[int]:
    return [x for x in range(2, n + 1) if es_primo(x)]

def suma_digitos(n: int) -> int:
    """Caso base: un solo dígito. Caso recursivo: último dígito + suma del resto."""
    if n < 10:
        return n
    return n % 10 + suma_digitos(n // 10)

```

```
def validar_contrasena(clave: str) -> list[str]:
    """Devuelve la lista de reglas que NO se cumplen (vacía = contraseña válida)."""
    errores = []
    if len(clave) < 8:
        errores.append("mínimo 8 caracteres")
    if not any(c.isupper() for c in clave):
        errores.append("al menos una mayúscula")
    if not any(c.isdigit() for c in clave):
        errores.append("al menos un número")
    if not re.search(r"^[A-Za-z0-9]", clave):
        errores.append("al menos un símbolo")
    return errores

if __name__ == "__main__":
    print(primos_hasta(30))          # [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    print(suma_digitos(2026))       # 10
    print(validar_contrasena("hola")) # ['mínimo 8 caracteres', 'al menos una mayúscula',
    ...]
    print(validar_contrasena("Ceipa#2026")) # []
```

Retos 12.1 y 13.1: Producto y vehículos

 **Archivo:** retos/soluciones/reto_12_13_poo.py

```
"""Retos 12.1 y 13.1: clase Producto y jerarquía de vehículos (polimorfismo)."""
from abc import ABC, abstractmethod

class Producto:
    IVA = 0.19

    def __init__(self, nombre: str, precio: float, stock: int = 0) -> None:
        self.nombre = nombre
        self.precio = precio          # pasa por el setter y se valida
        self.stock = stock

    @property
    def precio(self) -> float:
        return self._precio

    @precio.setter
    def precio(self, valor: float) -> None:
        if valor <= 0:
            raise ValueError("El precio debe ser positivo")
        self._precio = valor

    def precio_con_iva(self) -> float:
        return round(self.precio * (1 + self.IVA), 2)

    def vender(self, cantidad: int) -> float:
        if cantidad > self.stock:
            raise ValueError(f"Solo hay {self.stock} unidades de {self.nombre}")
        self.stock -= cantidad
        return self.precio_con_iva() * cantidad

    def __str__(self) -> str:
        return f"{self.nombre} (${self.precio:,.0f}) - stock {self.stock}"
```

```

class Vehiculo(ABC):
    def __init__(self, placa: str) -> None:
        self.placa = placa

    @abstractmethod
    def peaje(self) -> int: ...

class Moto(Vehiculo):
    def peaje(self) -> int:
        return 0

class Carro(Vehiculo):
    def peaje(self) -> int:
        return 13_000

class Camion(Vehiculo):
    def __init__(self, placa: str, ejes: int) -> None:
        super().__init__(placa)
        self.ejes = ejes

    def peaje(self) -> int:
        return 20_000 + 8_000 * (self.ejes - 2)

if __name__ == "__main__":
    teclado = Producto("Teclado", 80_000, 5)
    print(teclado, "->", teclado.precio_con_iva())      # 95200.0
    print(teclado.vender(2), teclado.stock)            # 190400.0 3
    fila = [Moto("ABC12D"), Carro("XYZ123"), Camion("TRK456", ejes=4)]
    print(sum(v.peaje() for v in fila))                # 49000 (el mismo mensaje,
distintas respuestas)

```

Reto 14.1: notas desde un CSV

 **Archivo:** retos/soluciones/reto_14_1_notas_csv.py

```

"""Reto 14.1: leer un CSV de notas, calcular promedios y manejar errores."""
import csv
from pathlib import Path

ARCHIVO = Path("notas.csv")

# Crea un archivo de ejemplo si no existe (incluye una fila con error a propósito)
if not ARCHIVO.exists():
    ARCHIVO.write_text(
        "estudiante,nota1,nota2,nota3\n"
        "Ana,4.5,3.8,4.2\n"
        "Luis,3.0,2.5,3.4\n"
        "Sofía,5.0,cinco,4.8\n",
        encoding="utf-8",
    )

promedios: dict[str, float] = {}
with ARCHIVO.open(encoding="utf-8") as f:

```

```

for numero_linea, fila in enumerate(csv.DictReader(f), start=2):
    try:
        notas = [float(fila[k]) for k in ("nota1", "nota2", "nota3")]
        promedios[fila["estudiante"]] = round(sum(notas) / len(notas), 2)
    except ValueError:
        print(f"Línea {numero_linea}: nota inválida para {fila['estudiante']}, se omite")

for estudiante, promedio in promedios.items():
    estado = "Aprobó" if promedio >= 3.0 else "Reprobó"
    print(f"{estudiante:<8} {promedio:.2f} {estado}")

```

Retos extra (sin solución, para practicar)

1. **Triqui:** juego de tres en línea para dos jugadores en consola, con detección de ganador y empate.
2. **Cajero automático con clases:** usa `CuentaBancaria` del Módulo 12 y un menú; guarda los movimientos en un archivo JSON.
3. **Conversor de moneda con Strategy** (Reto 20.1).
4. **Ahorcado:** palabras aleatorias desde un archivo de texto, dibujo del muñeco en consola y letras usadas en un `set`.
5. **Control de asistencia:** registra la asistencia de un curso en un diccionario `{fecha: [estudiantes]}` y reporta el porcentaje de asistencia de cada estudiante.
6. **Sensor simulado (Robótica):** genera lecturas aleatorias de temperatura cada segundo, detecta valores fuera de rango y guarda las alarmas en un CSV.
7. **Inventario de laboratorio con API:** convierte el inventario del Módulo 9 en una API con FastAPI y agrégle pruebas.

Proyecto integrador: sistema de préstamos de una biblioteca universitaria

Construye, por equipos, un sistema completo que aplique toda la guía:

Componente	Requisitos	Módulos
Modelo	Clases <code>Libro</code> , <code>Estudiante</code> y <code>Prestamo</code> con validaciones (<code>dataclass</code> o <code>Pydantic</code>)	12, 13
Reglas de negocio	Un estudiante no puede tener más de 3 préstamos; un libro prestado no se presta de nuevo; multa de \$1.000 por día de retraso	6, 10, 14
Persistencia	SQLite con consultas parametrizadas	23
Backend	API REST con FastAPI, arquitectura en capas, códigos HTTP correctos (200, 201, 204, 404, 409, 422)	21, 23
Frontend	Dashboard en Streamlit o interfaz en Tkinter	24
Patrones	Al menos dos: Repository, Strategy (cálculo de multas), Observer (avisos) o Factory	20
Pruebas	Mínimo 8 pruebas con pytest, incluyendo casos límite	27

Componente	Requisitos	Módulos
Entrega	Repositorio en GitHub con README.md , requirements.txt y capturas de pantalla	15

Rúbrica sugerida: funcionalidad (30 %), calidad del código y aplicación de principios (25 %), pruebas (15 %), arquitectura y patrones (15 %), documentación y uso de Git (15 %).

Recursos recomendados

Material gratuito para complementar la guía, ordenado según la etapa del curso. Los enlaces fueron revisados en septiembre de 2026; si alguno cambia, busca el nombre del recurso.

Videos en YouTube

Recurso	Idioma	Contenido	Etapas
Curso de Python desde cero para principiantes — MoureDev	Español	Unas 10 horas de fundamentos: variables, operadores, strings, listas, tuplas, sets, diccionarios, condicionales, ciclos, funciones, clases, excepciones y módulos	1 a 3
Python nivel intermedio — MoureDev	Español	Fechas, comprensión de listas, lambdas, funciones de orden superior, archivos, expresiones regulares y paquetes	2 y 3
Python para backend desde cero — MoureDev	Español	API REST con FastAPI, autenticación y base de datos	6
Lista completa del curso de Python — MoureDev	Español	Fundamentos, frontend, backend, testing e IA en una sola lista	Todas
Canal freeCodeCamp Español	Español	Cursos completos gratuitos de Python y otros lenguajes	1 a 3
Python Full Course for free — Bro Code	Inglés	Curso de unas 12 horas con proyectos: calculadora, juegos, Snake con interfaz gráfica	1 a 4

Repositorios en GitHub

Repositorio	Para qué sirve	Etapas
mouredev/Hello-Python	Código de todas las clases del curso de MoureDev, en español	1 a 6
Asabeneh/30-Days-Of-Python	Reto de 30 días con teoría y ejercicios diarios	1 a 3
trekhleb/learn-python	Ejemplos de cada tema con pruebas: se aprende leyendo y ejecutando	1 a 3
TheAlgorithms/Python	Cientos de algoritmos implementados: búsqueda, ordenamiento, estructuras de datos	2
faif/python-patterns	Catálogo de patrones de diseño en Python	5
public-apis/public-apis	Lista de APIs gratuitas para proyectos	6
vinta/awesome-python	Lista curada de librerías y frameworks por categoría	Todas

Documentación y práctica

Recurso	Para qué sirve
Tutorial oficial de Python en español	La referencia oficial, traducida por la comunidad
python.org/downloads	Descarga oficial de Python (Módulo 1)
Python Tutor	Visualiza paso a paso cómo se ejecuta el código y qué hay en memoria. Ideal para entender ciclos, listas y recursividad
CS50's Introduction to Programming with Python (Harvard)	Curso universitario gratuito con problemas calificados automáticamente
Exercism: ruta de Python	Ejercicios con retroalimentación de mentores voluntarios
roadmap.sh/python	Mapa visual de qué aprender y en qué orden
Refactoring.Guru: patrones en Python	Patrones de diseño explicados con ilustraciones, en español
Documentación de FastAPI en español	Tutorial oficial paso a paso (Módulo 23)
Documentación de Streamlit	Componentes y ejemplos (Módulo 24)
Documentación de pygame	Juegos 2D (Módulo 18)
Ursina Engine	Motor 3D (Módulo 26)
Documentación de pytest	Pruebas (Módulo 27)

Anexo: Hoja de referencia rápida

Tema	Sintaxis
Imprimir	<code>print("a", "b", sep="-", end="\n")</code>
Leer	<code>nombre = input("Nombre: ") · n = int(input())</code>
f-string	<code>f"{valor:,.2f}" · f"{texto:>10}" · f"{porc:.1%}"</code>
Condicional	<code>if x > 0: · elif x == 0: · else:</code>
Ternario	<code>a if condicion else b</code>
match	<code>match x: · case 1 2: · case _:</code>
while	<code>while condicion: · break · continue</code>
for	<code>for i in range(inicio, fin, paso): · for i, v in enumerate(lista):</code>
Lista	<code>l = [1, 2] · l.append(3) · l.pop() · l[1:3] · len(l)</code>
Comprensión	<code>[x * 2 for x in l if x > 0]</code>
Tupla	<code>t = (1, 2) · a, b = t</code>
Conjunto	<code>s = {1, 2} · s.add(3) · a & b · a b · a - b</code>
Diccionario	<code>d = {"k": 1} · d.get("k", 0) · for k, v in d.items():</code>
Función	<code>def f(a, b=0, *args, **kwargs) -> int: · return a + b</code>
Lambda	<code>lambda x: x ** 2</code>
Clase	<code>class C(Padre): · def __init__(self, x): · super().__init__()</code>
Excepción	<code>try: · except ValueError as e: · else: · finally: · raise</code>
Archivo	<code>with open("a.txt", "r", encoding="utf-8") as f:</code>
Importar	<code>import math · from random import randint</code>
Entorno virtual	<code>python -m venv .venv · activar · pip install -r requirements.txt</code>
Ejecutar pruebas	<code>pytest -v</code>
Levantar API	<code>uvicorn app.main:app --reload</code>